



KOCAELİ ÜNİVERSİTESİ
Fen Bilimleri Enstitüsü

BİLİŞİM SİSTEMLERİ MÜHENDİSLİĞİ ANABİLİM DALI

**Yapay Zekâ Destekli Otomatik Teleskop Yönlendirme, Gök Cismi
Tanıma ve Gök Cismi Veri Seti Oluşturma Platformu**

ABDULKADİR CAN KİNSİZ

Dr. Öğr. Üyesi Önder Yakut

LİSANS TEZİ

Kocaeli, 2025

ONAY SAYFASI

ETİK BEYAN VE ARAŞTIRMA FONU DESTEĞİ

Kocaeli Üniversitesi Fen Bilimleri Enstitüsü tez yazım kurallarına uygun olarak hazırladığım bu tez/proje çalışmasında,

- Bu tezin/projenin bana ait, özgün bir çalışma olduğunu,
- Çalışmamın hazırlık, veri toplama, analiz ve bilgilerin sunumu olmak üzere tüm aşamalarında bilimsel etik ilke ve kurallara uygun davrandığımı,
- Bu çalışma kapsamında elde edilen tüm veri ve bilgiler için kaynak gösterdiğimi ve bu kaynaklara kaynakçada yer verdiğimi,
- Bu çalışmanın Kocaeli Üniversitesi'nin abone olduğu intihal yazılım programı kullanılarak Fen Bilimleri Enstitüsü'nün belirlemiş olduğu ölçütlere uygun olduğunu,
- Kullanılan verilerde herhangi bir tahrifat yapmadığımı,
- Tezin/Projenin herhangi bir bölümünü bu üniversite veya başka bir üniversitede başka bir tez/proje çalışması olarak sunmadığımı,

beyan ederim.

☒ Bu tez/proje çalışmasının herhangi bir aşaması hiçbir kurum/kuruluş tarafından maddi/alt yapı desteği ile desteklenmemiştir.

☐ Bu tez/proje çalışması kapsamında üretilen veri ve bilgiler tarafından no'lu proje kapsamında maddi/alt yapı desteği alınarak gerçekleştirilmiştir.

Herhangi bir zamanda, çalışmamla ilgili yaptığım bu beyana aykırı bir durumun saptanması durumunda, ortaya çıkacak tüm ahlaki ve hukuki sonuçları kabul ettiğimi bildiririm.

.....
(İmza)

Abdulkadir Can Kinsiz
(Öğrencinin Adı Soyadı)

YAYIMLAMA VE FİKRİ MÜLKİYET HAKLARI

Fen Bilimleri Enstitüsü tarafından onaylanan lisansüstü tezimin/projemin tamamını veya herhangi bir kısmını, basılı ve elektronik formatta arşivleme ve aşağıda belirtilen koşullarla kullanıma açma izninin Kocaeli Üniversitesi'ne verdiğimi beyan ederim. Bu izinle Üniversiteye verilen kullanım hakları dışındaki tüm fikri mülkiyet haklarım bende kalacak, tezimin/projemin tamamının ya da bir bölümünün gelecekteki çalışmalarda (makale, kitap, lisans ve patent vb.) kullanımı bana ait olacaktır. Tezin/projenin kendi özgün çalışmam olduğunu, başkalarının haklarını ihlal etmediğimi ve tezimin/projenin tek yetkili sahibi olduğumu beyan ve taahhüt ederim. Tezimde yer alan telif hakkı bulunan ve sahiplerinden yazılı izin alınarak kullanılması zorunlu metinlerin yazılı izin alarak kullandığımı ve istenildiğinde suretlerini Üniversiteye teslim etmeyi taahhüt ederim.

Yükseköğretim kurulu tarafından yayınlanan "Lisansüstü Tezlerin Elektronik Ortamda Toplanması, Düzenlenmesi ve Erişime Açılmasına İlişkin Yönerge" kapsamında tezim aşağıda belirtilen koşullar haricinde YÖK Ulusal Tez Merkezi/ Kocaeli Üniversitesi Kütüphaneleri Açık Erişim Sisteminde erişime açılır.

☐ Enstitü yönetim kurulu kararı ile tezimin/projemin erişime açılması mezuniyet tarihinden itibaren 2 yıl ertelenmiştir.

☐ Enstitü yönetim kurulu gerekçeli kararı ile tezimin/projemin erişime açılması mezuniyet tarihinden itibaren 6 ay ertelenmiştir.

☒ Tezim/projem ile ilgili gizlilik kararı verilmemiştir.

.....
(İmza)

Abdulkadir Can Kinsiz
(Öğrencinin Adı Soyadı)

ÖNSÖZ VE TEŞEKKÜR

Bu tez, yapay zekâ destekli gök cismi tanıma, otomatik teleskop yönlendirme ve gözlem verilerinin bulut tabanlı bir veri setine dönüştürülmesi amacıyla geliştirilmiştir. Düşük maliyetli donanım bileşenleri ile modern görüntü işleme yöntemlerinin bir araya getirildiği bu çalışma, Türkiye’de amatör astronomi ve otonom gözlem sistemleri için erişilebilir ve sürdürülebilir bir altyapı sunmayı hedeflemektedir. Tez sürecinde teleskop donanımlarının entegrasyonu, sensörlerden konum–yönelim hesaplamaları, görüntülerin yapay zekâ ile sınıflandırılması ve veri kayıt mekanizmalarının oluşturulması gibi aşamalar sistematik biçimde yürütülmüştür.

Bu araştırmanın hazırlanmasında süreç boyunca motivasyon ve desteklerini esirgemeyen, akademik rehberliği ve yönlendirmeleri için danışmanım Dr. Öğr. Üyesi Önder Yakut’a teşekkür ederim.

Kasım – 2025

Abdulkadir Can Kinsiz

İçindekiler

ONAY SAYFASI.....	2
ETİK BEYAN VE ARAŞTIRMA FONU DESTEĞİ.....	3
YAYIMLAMA VE FİKRİ MÜLKİYET HAKLARI.....	4
ÖNSÖZ VE TEŞEKKÜR.....	5
İÇİNDEKİLER.....	6
ŞEKİLLER DİZİNİ.....	8
ÖZET.....	9
ABSTRACT.....	10
1. GİRİŞ.....	11
2. GENEL BİLGİLER.....	12
3. MALZEME VE YÖNETİM.....	13
3.1. KULLANILAN DONANIM BİLEŞENLERİ.....	13
3.1.1. TELESKOP SİSTEMİ.....	14
3.1.1.1. MERCEK AYARI VE OPTİK KOLİMASYON SÜRECİ.....	15
3.1.1.2. NEWTONİAN OPTİK SİSTEMİNİN TEMEL YAPISI VE EKSEN KAVRAMLARI.....	16
3.1.1.3. HATALI KOLİMASYONUN GÖRÜNTÜ KALİTESİNE ETKİLERİ	16
3.1.1.4. KOLİMASYONUN DOĞRULANMASI: YILDIZ TESTİ, GÖRSEL VE FOTOĞRAFİK GÖSTERGELER.....	19
3.1.2. GÖRÜNTÜLEME BİRİMİ.....	21
3.1.3. MİKRODENETLEYİCİ VE KONTROL BİRİMLERİ.....	22
3.1.4. MOTORLAR VE MOTOR SÜRÜCÜLERİ.....	23
3.1.5. KONUM VE YÖNELİM SENSÖRLERİ.....	24
3.1.6. GÜÇ BİLEŞENLERİ VE EK DONANIMLAR.....	25
3.2. YAZILIM ALTYAPISI VE YAPAY ZEKÂ BİLEŞENLERİ.....	26
3.2.1. YAZILIM MİMARİSİ VE KODLAMA YOL HARİTASI: YILDIZ–	

GEZEĞEN TANIMA VE OTONOM GÖZLEM AKIŞI.....	28
3.3. OTONOM YÖNLENDİRME YÖNTEMİ.....	32
3.4. VERİ KAYDI VE DEPOLAMA YÖNTEMİ.....	33
3.4.1. KAYDEDİLEN VERİLERİN VERİ SETİNE TAŞINMASI.....	35
3.4.1.1. OTURUM BAZLI VERİ PAKETLEME VE STANDARTLAŞTIRMA.....	35
3.4.1.2. ÇEVİRİMDIŞI-ÇEVİRİMİÇİ SENARYOLARDA DAYANIKLI VERİ AKTARIMI.....	36
3.4.1.3. SUNUCU ALTYAPISI, DOMAIN PLANI VE DAĞITIM YAKLAŞIMI.....	37
3.4.1.4. SUNUCU TARAĞI SERVİSLER VE TEKNOLOJİ BİLEŞENLERİ	38
3.4.1.5. VERİ DOĞRULAMA, KALİTE KONTROL VE ZENGİNLEŞTİRME ADIMLARI.....	38
3.4.1.6. ETİKETLEME VE VERİ SETİ SÜRÜMLEME YAKLAŞIMI.....	39
3.5. DENEYSEL GÖZLEM YÖNTEMİ.....	40
3.6. MOBİL UYGULAMA VE UZAKTAN KONTROL ALTYAPISI.....	42
3.7 YÖNTEMİN DİYAGRAMLARLA AÇIKLANMASI VE DENEYSEL SONUÇLARIN DEĞERLENDİRİLMESİ.....	45
3.7.1 GENİŞLETİLMİŞ PSEUDO CODE.....	46
4. SONUÇLAR VE ÖNERİLER.....	48
KAYNAKLAR.....	50

ŞEKİLLER DİZİNİ

ŞEKİL 1. KOLİMASYON BOZULMASININ OPTİK EKSENE ETKİSİ.

ŞEKİL 2. KOLİMASYON DÜZELTMESİ SONRASI ELDE EDİLEN DOĞRU HİZALAMA.

ŞEKİL 3. NEO-6M GPS ÖRNEK OKUMA KODU (PYTHON, RPİ TARAFI)

ŞEKİL 4. ÖRNEK SERVO SÜRME KODU

ŞEKİL 5. ÖRNEK KARE YAKALAMA KODU

ŞEKİL 6. RASPBERRYPI'DE TFLITE INFERENCE ÖRNEK İSKELETİ

ŞEKİL 7. BASİTLEŞTİRİLMİŞ ÖRNEK META VERİSİ

ŞEKİL 8. DENEYSEL GÖZLEM DÜZENİNDE KULLANILAN DONANIM BİLEŞENLERİ

ŞEKİL 9. OTONOM TELESKOP SİSTEMİNİN GENEL AKIŞ DİYAGRAMI

ŞEKİL 10. BÜTÇE DOSTU PROTOTİPİN SAHAYA KURULMUŞ HÂLİ: TELESKOP TÜPÜ, TURUNCU 3D BASKI HALKA KELEPÇELER, YEŞİL AZİMUT/ALTITUDE MODÜLÜ, ALÜMİNYUM TRİPOD VE SÜRÜCÜ ELEKTRONİĞİ (LABORATUVAR GÜÇ KAYNAĞI VE BAĞLANTI KABLOLARI).

ŞEKİL 11. 360° AZİMUT ÜST TABLA (MONTAJ) — DIŞLI/TIRTIKLI DIŞ PROFİLLİ DÖNER TABLA KAPAK PARÇASI (360 ALTIN ÜSTÜ PLAKA).

ŞEKİL 12. 360° ÜST PARÇA — AZİMUT TABLASININ DÜZ DİSK/TABLA BİLEŞENİ.

ŞEKİL 13. SERVOLU ALT PLAKA — AZİMUT SERVOSUNUN OTURDUĞU SİLİNDİRİK YATAK GÖVDESİ (ALT PLAKA SERVOLU).

ŞEKİL 14. 360° ALT TABLA VARYANTI (TELESCOPE ALT2 360) — TABAN MONTAJ MODÜLÜ.

ŞEKİL 15. TELESKOP TÜPÜ HALKA KELEPÇE MONTAJI (TELESKOP_MOUNT) — TÜPÜ SABİTLEYEN C PROFİLLİ KELEPÇE VE TABAN PLAKASI.

ŞEKİL 16. TELESKOP ÜST MONTAJI (TELESKOP ÜSTÜ SON) — HALKA KELEPÇE + YÜKSEKLİK (ALTITUDE) BEŞİĞİ BİRLEŞİMİ.

ŞEKİL 17. ALT PARÇA (ALT_PARCA) — SÜRÜCÜ ELEKTRONİĞİ VE KABLOLAMANIN YERLEŞTİĞİ KUTU GÖVDE.

ŞEKİL 18. SERVO TUTUCU 1 — U PROFİLLİ SERVO BRAKETİ.

ŞEKİL 19. SERVO TUTUCU 2 — KÖPRÜ (π) BİÇİMLİ MONTAJ BRAKETİ.

ŞEKİL 20. SERVO UCU ADAPTÖRÜ (SERVO UCU 2) — MİL AKTARIMI İÇİN HAÇ BİÇİMLİ BAĞLANTI PARÇASI.

ŞEKİL 21. TELESKOP MONTAJ KESİTİ — HALKA KELEPÇENİN ALT KUTU GÖVDEYE OTURTULMA GÖRÜNÜMÜ (TELESKOP MONTAJ KESİT).

ŞEKİL 22. ATÖLYEDE SİYAH TABAN/MUHAFAZA PANELİNİN HAZIRLANMASI (WHATSAPP VİDEO, 2026-03-16).

ŞEKİL 23. AZİMUT EKSENİ TİMİNG BELT (ZAMAN KAYIŞI) VE KASNAK AKTARIM MEKANİZMASI — SÜREKLİ DÖNÜŞ SERVOSUNUN TABLAYA BAĞLANDIĞI İÇ MONTAJ.

ŞEKİL 24. 3B BASKI SİYAH GÖVDE VE PAN-TİLT İSKELETİNİN TRİPOD ÜZERİNE BİRLEŞTİRİLMESİ.

ŞEKİL 25. YEŞİL AZİMUT TABLASININ DREMEL İLE İŞLENMESİ (TOLERANS DÜZELTME).

ŞEKİL 26. ATÖLYE TEZGAHINDA METAL FLANŞ, HALKA PARÇALAR VE MONTAJ MALZEMELERİ.

ŞEKİL 27. KURULU PROTOTİP — TELESKOP TÜPÜ, TURUNCU HALKA KELEPÇELER, YEŞİL AZİMUT MODÜLÜ VE ALÜMİNYUM TRİPOD (YOUTUBE TANITIM VİDEOSU, ~6:12).

ŞEKİL 28. KURULU SİSTEM VE ARKA PLANDA WEB TABANLI OTOSKOP GÖZLEM ARŞİVİ ARAYÜZÜNÜN ÇALIŞTIĞI İZLEME İSTASYONU.

ŞEKİL 29. SERVO MİMARİSİ VE PİN ATAMALARI (ARDUİNO MEGA).

ŞEKİL 30. EĞİM-KOMPANZE PUSULA YÖNÜ VE JİROSKOP TÜMLEYİCİ FİLTRESİ (ARDUİNO MEGA).

ŞEKİL 31. SERVO SÜRÜŞ DÖNGÜSÜ VE HEDEF KİLİDİ TESPİTİ (ARDUİNO MEGA).

ŞEKİL 32. ~10 HZ TELEMETRİ JSON PAKETİNİN ÜRETİMİ (ARDUİNO MEGA).

ŞEKİL 33. HTTP UÇ NOKTALARININ YÖNLENDİRİLMESİ VE ÇİFT SUNUCU TASARIMI (ESP32-CAM).

ŞEKİL 34. /STATUS UÇ NOKTASI VE UART BAĞLANTI TEŞHİSİ (ESP32-CAM).

ŞEKİL 35. UART KÖPRÜSÜNDEKİ JSON TELEMETRİ VE KOMUT PROTOKOLÜ ÖRNEKLERİ.

ŞEKİL 36. WEB TABANLI GÖZLEM ARŞİVİNDE BİR MARS GÖZLEMİNİN DETAY SAYFASI: AZİMUT/YÜKSEKLİK, GPS, PARLAKLIK, YAPAY ZEKÂ DOĞRULAMA GÜVENİ VE DİĞER META VERİLER.

ŞEKİL 37. VERİTABANININ OTOMATİK OLUŞTURULMASI VE ŞEMA GÖÇÜ (DB.PHP).

ŞEKİL 38. OTOSKOP GÖZLEM ARŞİVİ GALERİ GÖRÜNÜMÜ: KULLANICILARIN YÜKLEDİĞİ GÖK CİSMİ GÖZLEMLERİ, AI DOĞRULAMA ROZETLERİ VE FİLTRELEME SEÇENEKLERİYLE BİRLİKTE.

ŞEKİL 39. ÇAKIŞMASIZ DOSYA ADI ÜRETİMİ VE GÜVENLİ DEPOLAMA (UPLOAD_LIB.PHP).

ŞEKİL 40. CİHAZ YÜKLEME API'SİNDE ANAHTAR DOĞRULAMASI (API/UPLOAD.PHP).

ŞEKİL 41. TELESKOP KONTROLÜ İÇİN RETROFİT ESP32 API ARAYÜZÜ (ESP32API.KT).

ŞEKİL 42. MDNS/NSD İLE ESP32 CİHAZININ OTOMATİK KEŞFİ (ESP32DISCOVERY.KT).

ŞEKİL 43. ANDROİD UYGULAMASI — MANUEL TELESKOP KONTROL EKRANI: ANLIK VE HEDEF AZ/ALT AÇILARI, SERVO AÇILARI, ADIM BÜYÜKLÜĞÜ VE YÖN TUŞLARI.

ŞEKİL 44. EKVATORAL KOORDİNATLARIN YEREL UFUK KOORDİNATLARINA DÖNÜŞÜMÜ (EPHEMERIS.KT).

ŞEKİL 45. YÖN KALİBRASYONU: TELEFON PUSULASI İLE TELESKOBUN BAKTIĞI AZİMUTUN EŞİTLENMESİ.

ŞEKİL 46. MJPEG AKIŞINDAN JPEG KARELERİN AYIKLANMASI (MJPEGVIEW.KT).

ŞEKİL 47. CANLI KAMERA EKRANI: ESP32-CAM BAĞLANDIĞINDA MJPEG AKIŞI BU ALANDA GÖRÜNTÜLENİR; FOTO ÇEK / VIDEO KAYDI / DOĞRULA İŞLEMLERİ BURADAN YAPILIR.

ŞEKİL 48. CANLI TELEMETRİ/TEŞHİS GÜNLÜĞÜ: AZ/ALT, GPS VE IMU DURUMLARI İLE UART HATTI TANILAMASI ANLIK OLARAK İZLENİR.

ŞEKİL 49. YAKALANAN GÖZLEMLER LİSTESİ: HER KAYIT; HEDEF ADI, ORTAM TÜRÜ, AZİMUT/YÜKSEKLİK VE ÇEKİM ZAMANI META VERİLERİYLE SAKLANIR.

ŞEKİL 50. MOBİL UYGULAMANIN KURULU SİSTEMLE BİRLİKTE SAHADA KULLANIMI.

ŞEKİL 51. ODAK/KOLİMASYON KALİTESİNİN GÖRÜNTÜYE ETKİSİ: İYİ ODAKLANMIŞ (SOLDA) VE ODAĞI KAÇMIŞ (SAĞDA) AY GÖRÜNTÜLERİNİN KARŞILAŞTIRMASI.

ŞEKİL 52. PROTOTİPLE FARKLI OTURUMLARDA ELDE EDİLEN AY GÖZLEMLERİNDEN BİR SEÇKİ.

ŞEKİL 53. PROTOTİP TELESKOBA BAĞLI ESP32-CAM İLE ELDE EDİLEN, YÜZEY DETAYLARININ (KRATERLER, DENİZLER) SEÇİLEBİLDİĞİ BİR AY GÖRÜNTÜSÜ.

ŞEKİL C.1. WEB PLATFORMU OTURUM AÇMA EKRANI (OTURUM TABANLI KİMLİK DOĞRULAMA).

ŞEKİL C.2. GÖZLEM KONUMUNUN HARİTA ÜZERİNDE GÖSTERİMİ (GPS META VERİSİ).

ŞEKİL C.3. PROTOTİPLE ELDE EDİLEN EK AY GÖZLEMLERİ SEÇKİSİ.

ŞEKİL 2. KOLİMASYON DÜZELTMESİ SONRASI ELDE EDİLEN DOĞRU HİZALAMA.

ŞEKİL 5. ÖRNEK KARE YAKALAMA KODU

ŞEKİL 7. BASİTLEŞTİRİLMİŞ ÖRNEK META VERİSİ

ŞEKİL 9. OTONOM TELESKOP SİSTEMİNİN GENEL AKIŞ DİYAGRAMI

Yapay Zekâ Destekli Otomatik Teleskop Yönlendirme, Gök Cismi Tanıma ve Gök Cismi Veri Seti Oluşturma Platformu

ÖZET

Bu tez, düşük maliyetli elektronik bileşenler, yapay zekâ tabanlı görüntü işleme teknikleri ve otonom kontrol mekanizmalarını bir araya getirerek tamamen kendi kendine çalışabilen yenilikçi bir teleskop gözlem sistemi geliştirmeyi

amaçlamaktadır. Sistem; konum, yönelim, görüntü ve sensör verilerini eşzamanlı olarak değerlendirerek gökyüzünü otomatik tarayabilmekte, teleskopu hedef cisimlere yönlendirebilmekte ve tanımlanan nesneleri yapay zekâ algoritmalarıyla sınıflandırabilmektedir. Raspberry Pi merkezli kontrol yapısı, motor sürücülerle birlikte çalışarak teleskopun yatay ve dikey eksenlerde yüksek hassasiyetle hareket etmesini sağlar. Teleskoba bağlı kamera tarafından alınan görüntüler gerçek zamanlı olarak analiz edilir ve sistem, hedef cismin doğru konumda olup olmadığını sürekli kontrol eder.

Gözlemler sırasında kaydedilen görüntüler; tarih, konum, nesne tipi, parlaklık ve model güven oranı gibi ayrıntılı meta verilerle birlikte işlenir. Bu veriler hem yerel depolama birimlerinde tutulur hem de internet bağlantısının bulunduğu durumlarda bulut tabanlı bir veri havuzuna aktarılır. Süreç sonunda geniş kapsamlı, açık erişimli ve sürekli büyüyen bir “Gökyüzü Veri Seti” oluşturulacaktır. Bu veri seti, astronomi araştırmalarında kullanılabileceği gibi makine öğrenmesi model eğitimi, uzaktan eğitim uygulamaları, bilim iletişimi çalışmaları ve dijital sanat çalışmaları için de önemli bir kaynak oluşturmaktadır.

Tez, manuel teleskop kullanımından kaynaklanan hataları ortadan kaldırarak daha tutarlı, kesintisiz ve herkes tarafından erişilebilir bir gözlem deneyimi sunmayı hedefler. Açık kaynaklı yapısı sayesinde öğrenciler, amatör astronomlar ve bağımsız araştırmacılar tarafından geliştirilebilir, uyarlanabilir ve uzun vadede sürdürülebilir bir gözlem altyapısı haline gelmesi amaçlanmaktadır.

Anahtar Kelimeler: Otonom Teleskop Sistemi, Yapay Zekâ, Gök Cismi Tanıma, Görüntü İşleme, Astronomik Veri Seti

Artificial Intelligence–Powered Automated Telescope Pointing, Celestial Object Recognition, and Astronomical Object Dataset Generation Platform

ABSTRACT

This thesis aims to develop an innovative, fully autonomous telescope observation system by integrating low-cost electronic components, artificial intelligence–based image processing methods, and automated control mechanisms. The system is

designed to operate independently, continuously analyzing the sky, detecting celestial objects, and adjusting the telescope's orientation without human intervention. Using a Raspberry Pi as the central control unit, the system processes data from sensors, motor drivers, and the camera module to precisely manage the telescope's horizontal and vertical movements. Real-time image analysis allows the system to verify whether the target object is correctly centered, enabling stable tracking and uninterrupted observation sessions.

Images captured during observations are enriched with detailed metadata such as date, geographic coordinates, object type, brightness level, and AI confidence scores. These data are stored locally and automatically uploaded to a cloud-based repository when an internet connection is available. As the system collects and labels observations over time, it will form a comprehensive and publicly accessible "Sky Object Dataset." This dataset is intended to support a wide range of applications, including astronomy research, machine learning model training, educational tools, remote learning environments, and digital art thesis.

By eliminating the manual effort and potential errors associated with traditional telescope operations, the thesis aims to provide a more consistent, reliable, and user-friendly observation experience. Its open-source structure enables students, amateur astronomers, researchers, and hobbyists to modify, enhance, or expand the system according to their needs. Ultimately, the thesis seeks to democratize access to astronomical observation, create a sustainable and scalable technological framework, and offer a versatile platform that bridges scientific exploration, education, and creative fields.

Keywords: Autonomous Telescope System, Artificial Intelligence, Celestial Object Recognition, Image Processing, Astronomical Dataset

1. GİRİŞ

Astronomik gözlemler, insanlığın evrene dair merakının en temel unsurlarından biri olmuş ve tarih boyunca bilimsel gelişmelerin önünü açmıştır. Geleneksel gözlem yöntemleri, teleskopun tamamen kullanıcı tarafından kontrol edilmesini gerektiren, yüksek dikkat ve deneyim isteyen süreçlere dayanır. Bu yöntemlerde teleskopun hizalanması, gök cisminin kadrada tutulması, poz süresinin ayarlanması ve verilerin kaydedilmesi gibi birçok aşama manuel olarak yapılır. Bu durum, hem zaman alıcıdır hem de özellikle amatör astronomlar veya öğrenciler için gözlem sürecinin verimliliğini ciddi biçimde düşürmektedir. Ayrıca manuel gözlemler, tekrar edilebilir bilimsel çıktı üretmekte zorluk yaşar; çünkü insan faktöründen kaynaklanan hata payı yüksektir.

Modern teknolojilerin gelişmesi ile yapay zekâ, görüntü işleme ve mikrodenetleyici tabanlı otomasyon sistemleri astronomi alanında devrim niteliğinde yenilikler ortaya çıkarmıştır. Özellikle düşük maliyetli bilgisayar kartları (Raspberry Pi vb.), hassas motor kontrol sistemleri ve derin öğrenme tabanlı görüntü sınıflandırma teknikleri, teleskopların insan müdahalesi olmadan çalışabilmesine olanak sağlamıştır. Böylece “otonom teleskop” kavramı ortaya çıkmış; otomatik yönlendirme, hedef takip ve veri kaydı sistemleri astronomi çalışmalarında önem kazanmaya başlamıştır.

Bu tez bu teknolojik dönüşümü doğrudan kullanarak; gök cisimlerini otomatik olarak tanıyabilen, hedefi kadrada tutan, görüntüyü işleyip meta veri üreten ve bunları yerel ya da bulut tabanlı veri tabanına kaydedebilen bir otonom teleskop sistemi geliştirmeyi amaçlamaktadır. Tez, düşük maliyetli, taşınabilir, kullanıcı dostu ve açık kaynaklı bir çözüm sunarak astronomiyle ilgilenen herkesin kendi gözlem ve veri üretim altyapısına sahip olmasını hedeflemektedir.

Geliştirilen sistem yalnızca gözlem sürecini otomatik hâle getirmekle kalmamakta, aynı zamanda uzun yıllar boyunca büyüyecek nitelikte kapsamlı bir “Gökyüzü Veri Seti” oluşturulmasını da sağlamaktadır. Bu veri seti bilimsel araştırmalarda kullanılabileceği gibi eğitim, dijital sanat, medya üretimi ve makine öğrenmesi çalışmaları için de önemli bir kaynak olacaktır. Ayrıca tez, astronomi meraklılarının kendi çevrelerinden veri toplayarak evrensel bir gözlem ağına katkı sunabilmelerine olanak tanımaktadır.

Bu çalışmanın amacı, yapay zekâ destekli otomatik gök cismi tanıma, teleskop yönlendirme, veri kaydı ve bulut entegrasyonu özelliklerine sahip, düşük maliyetli ve sürdürülebilir bir otonom teleskop sistemi tasarlamak, geliştirmek ve performansını değerlendirmektir.

2. GENEL BİLGİLER

Astronomik gözlem teknolojileri son on yılda önemli bir dönüşüm sürecine girmiştir. Bu dönüşüm, özellikle düşük maliyetli bilgisayar platformlarının, açık kaynak donanımların ve kompakt görüntüleme sistemlerinin yaygınlaşmasıyla hız kazanmıştır. Geleneksel teleskop sistemleri çoğunlukla kullanıcıya bağımlı, manuel kontrol gerektiren ve yüksek maliyetli ekipmanlara dayanmaktaydı. Ancak hem donanım maliyetlerinin düşmesi hem de yapay zekâ temelli görüntü analizi yöntemlerinin gelişmesi, amatör astronomiden araştırma amaçlı gözlem topluluklarına kadar geniş bir kitlenin gökyüzü gözlemlerine katkı sunabileceği yeni bir dönemi başlatmıştır.

Uzay nesnelerinin ve gök cisimlerinin optik teleskop görüntülerinden gerçek zamanlı olarak tespiti ve izlenmesi, son yıllarda makine öğrenmesi yöntemlerinin yaygınlaşmasıyla birlikte önemli bir araştırma alanı hâline gelmiştir [1], [7], [10]. Bu çalışmalarda; uzay-temelli gözetim sistemlerinden yer-temelli tarama teleskoplarına kadar geniş bir yelpazede, hareketli hedeflerin arka plandaki sabit yıldız alanından ayrıştırılması ve iz (tracklet) çıkarımı amaçlanmaktadır. Derin öğrenme tabanlı nesne tespiti modellerinin (ör. YOLO ailesi) astronomik görüntülere uyarlanması, özellikle küçük ve sönük nesnelerin saptanmasında klasik eşikleme yöntemlerine kıyasla belirgin başarı artışı sağlamıştır [2], [16].

Bu alanın güncel durumunu derleyen kapsamlı tarama çalışmaları, yörüngedeki nesnelerin (resident space objects) tespiti ve karakterizasyonunda makine öğrenmesinin rolünü sistematik biçimde ele almaktadır [11]. Bununla birlikte, geniş alanlı ve küçük açıklıklı teleskoplarla yapılan gözlemlerde gerçek zamanlı geçici (transient) olay tespiti, hesaplama kaynaklarının sınırlı olduğu sahalarda dahi uygulanabilir akıllı gözlem yöntemlerini gündeme getirmiştir [15]. Bu yaklaşımlar, pahalı profesyonel altyapılar yerine erişilebilir donanımlarla çalışan, uçta (edge) işleme yapabilen sistemlerin önemini vurgulamaktadır.

Erişilebilir donanımlar bağlamında, Raspberry Pi gibi tek kart bilgisayarların astronomik uygulamalarda kullanımı literatürde sıkça incelenmiştir. Düşük maliyetli akış cihazları [4], meteor kamera sistemleri [5] ve ultra-düşük maliyetli yıldız izleyici (star tracker) konseptleri [6], hobici ve eğitim amaçlı astronomiyi demokratikleştirmiştir. Benzer biçimde PANOPTES projesi, düşük maliyetli dijital kameralarla ötegezegen keşfini hedefleyen dağıtık bir vatandaş-bilim girişimi olarak öne çıkmaktadır [17]. Bu çalışmalar, bu tezde benimsenen “maliyet-erişilebilirlik” odaklı tasarım felsefesinin literatürdeki sağlam dayanaklarını oluşturur.

Robotik ve otonom teleskop kontrolü tarafında, gözlemevi denetim sistemleri (observatory control systems) teleskobun konumlandırılmasından veri toplamaya kadar tüm süreci otomatikleştiren yazılım çerçeveleri sunmaktadır [9]. Tüketici sınıfı kameralarla yapılan ötegezegen geçiş (transit) fotometrisi çalışmaları ise [3], [12], [13]; DSLR ve CMOS sensörlerin amatör koşullarda dahi bilimsel değer taşıyan veriler üretebildiğini göstermiştir. Dar alanlı astrometri için öğrenme tabanlı yıldız konumlandırma yöntemleri [14] de, görüntüden hassas konum çıkarımının otomatikleştirilmesine katkı sunmaktadır. Tüm bu birikim, bu tezde önerilen bütünsel ve otonom gözlem sisteminin hem teknik hem de bilimsel gerekçesini güçlendirmektedir.

Günümüzde astronomik gözlemlerde sıklıkla başvuru alan yapay zekâ temelli görüntü işleme yöntemleri, gök cismi tespiti, sınıflandırması ve hareket takibi gibi süreçlerde önemli avantajlar sunmaktadır. Örneğin [1], [2] ve [7] gibi çalışmalarda optik teleskop

görüntülerinin otomatik olarak işlenmesi, gök cisimlerinin gerçek zamanlı tespiti ve gözlem akışının operatör müdahalesine gerek kalmadan sürdürülebilmesi gösterilmiştir. Bu çalışmalar, gök cisimlerinin otomatik takibini mümkün kılarak hem insan hatasını azaltmakta hem de uzun süreli gözlem görevlerinde performansı artırmaktadır [3], [11].

Bununla birlikte literatürde yer alan mevcut sistemlerin büyük bir kısmı ya profesyonel gözlemevlerinde kullanılan yüksek maliyetli donanımlara bağlıdır ya da belirli tekil görevlerle sınırlıdır. Örneğin meteor izleme sistemleri [4], [5] yalnızca meteor olaylarına odaklanırken; yıldız takip sistemleri [6] düşük maliyetli yapılarıyla dikkat çekse de veri paylaşımı, sınıflandırma veya bulut tabanlı arşivleme gibi özellikler sunmamaktadır. Benzer şekilde optik teleskoplarla yapılan uzay nesnesi tespit çalışmalarında [2], [7], [10] daha çok yapay zekâ modellerinin performans analizleri ön plandadır; ancak bu sistemlerin çoğu gerçek zamanlı otonom yönlendirme kabiliyetine sahip değildir.

Bu bağlamda geliştirilen tez, mevcut literatürde gözlemlenen eksikliklere doğrudan karşılık gelen bütünsel bir yaklaşım sunmaktadır. Düşük maliyetli donanım kullanılarak otonom bir teleskop sistemi kurulması; görüntülerin yapay zekâyla yorumlanması, teleskop yönünün otomatik olarak ayarlanması, gözlem sonucunda elde edilen görüntülerin ve meta verilerin kayıt altına alınması; bunların yerel veya çevrimiçi ortamlarda saklanarak açık bir veri setine dönüştürülmesi; literatürde birlikte ele alınmayan, ancak modern gözlem sistemlerinin ihtiyaç duyduğu bütünlüklü bir yapıdır. Sistem; teleskop, görüntüleme birimi, yapay zekâ modelleri, motor kontrol mekanizması ve veri depolama altyapısını bir araya getiren bütünsel bir mimariye sahiptir.

Bu yaklaşımın en önemli yeniliklerinden biri, verinin yalnızca analiz edilmek üzere tüketilmesi değil; doğrudan kullanıcı tarafından üretilerek paylaşıma açılmasıdır. Literatürdeki büyük ölçekli veri setlerinin çoğu uluslararası ajanslar tarafından üretilmiş olup genellikle lisans kısıtları taşır veya güncel gözlem koşullarını yansıtmaz. Mevcut veri setlerinin çoğu uzay teleskoplarından, uydulardan veya yüksek çözünürlüklü profesyonel sensörlerden elde edildiği için amatör gözlem koşullarıyla uyumlu değildir. Bu nedenle [6], [9], [13] gibi çalışmalarda düşük maliyetli donanım kullanılarak gözlem yapılmasının önemi vurgulanmış; geniş katılımlı veri üretiminin, özellikle yerde konuşlu küçük açıklıklı teleskoplarla yürütülen gözlem çalışmalarında önemli bir rol oynadığı ifade edilmiştir.

Bu tez kapsamındaki otonom gözlem sistemi, kamera sensörü olarak düşük maliyetli modüllerle uyumlu çalışacak şekilde tasarlanmıştır. Tez için IMX477 gibi yüksek performanslı sensörler önerilse de, maliyet kısıtları nedeniyle ESP32-CAM gibi daha kompakt ve erişilebilir kameralar da sistem mimarisine dâhil edilebilmektedir. Bu durum, literatürde [4], [5] ve [6] kapsamındaki çalışmalarla paraleldir; bu çalışmalar düşük maliyetli görüntüleme cihazlarının bilimsel gözlem için yeterli seviyede veri üretebildiğini göstermektedir.

Otonom teleskop sistemlerinde yalnızca görüntüleme bileşenleri değil, aynı zamanda yönlendirme mekanizması da hayati önem taşır. Sistem; step motorlar, motor sürücü devreleri ve mikrodenetleyici birimleriyle donatılarak teleskopun hedef cisimlere otomatik olarak yönlendirilebilmesini sağlar. Bu yönlendirme süreci sırasında kullanıcının yalnızca gözlem yapılacak gök cismini seçmesi yeterlidir; sistem, mevcut konum bilgisi, zamanı ve sensör verilerini kullanarak önceden belirlenen nesnenin açı

farkını hesaplar, motorları doğru şekilde çalıştırır ve teleskopu istenen konuma getirir. Bu tür otomatik yönlendirme mekanizmaları [7], [8] gibi çalışmalarda farklı algoritmalarla gösterilmiş olsa da, düşük maliyetli donanımlarla bütünleşik açık veri üretimi ile sunulan bir sistem literatürde sınırlıdır.

Elde edilen görüntülerin saklanması ve işlenmesi süreci, modern astronomik gözlem mimarilerinin en kritik yapı taşlarından biridir. Sistem; görüntüleri tarih, konum, parlaklık bilgisi, tespit edilen gök cisminin türü ve yapay zekâ modelinin güven oranıyla birlikte kaydederek bunları yerel ve çevrimiçi veri tabanlarına aktarmaktadır. Bu yaklaşım, [14], [15] ve [16] gibi çalışmalarda vurgulanan astronomik veri yönetimi pratikleriyle uyumludur ve geniş ölçekli açık veri setlerinin oluşturulmasına katkı sağlar.

2.1. DÜŞÜK MALİYETLİ VE ERİŞİLEBİLİR GÖZLEM SİSTEMLERİ

Astronomik gözlem donanımlarının yüksek maliyeti, uzun yıllar boyunca bu alanı büyük ölçüde profesyonel gözlemcileri ve iyi finanse edilen araştırma gruplarıyla sınırlamıştır. Ancak tek kart bilgisayarların (single-board computers), mikrodenetleyici kartların ve düşük maliyetli CMOS görüntüleyicilerin yaygınlaşması, gözlem sistemlerinin amatörler ve eğitim kurumları için de erişilebilir hâle gelmesini sağlamıştır. Maulana ve arkadaşları, Raspberry Pi tabanlı bir denetleyicinin astronomik bir düzeneğin kontrolünde başarıyla kullanılabileceğini göstererek, tek kart bilgisayarların bu alandaki potansiyelini ortaya koymuştur [4]. Bu yaklaşım, hem donanım maliyetini düşürmekte hem de açık kaynaklı yazılım ekosisteminden yararlanma imkânı sunmaktadır.

Gutiérrez ve arkadaşlarının geliştirdiği SOST (ultra-low-cost) sistemi, gerçekten düşük bütçeyle dahi işlevsel bir gözlem aracının kurulabileceğini kanıtlayan dikkat çekici bir çalışmadır [6]. Benzer biçimde, geniş alan gözlemlerinde maliyet etkin çözümlerin kullanılması, hem amatör topluluğun hem de bilimsel projelerin veri toplama kapasitesini önemli ölçüde artırmıştır [17]. Bu çalışmalar, pahalı endüstriyel bileşenlerin yerine dikkatli bir mühendislik ve yazılım tasarımıyla benzer işlevlerin elde edilebileceğini göstermektedir. Bu tez kapsamında geliştirilen prototip de tam olarak bu felsefeyi benimsemekte; Raspberry Pi tabanlı ideal mimarının yerine, bütçe kısıtları altında Arduino Mega 2560 ve ESP32-CAM ikilisini kullanan çalışır bir sistem ortaya koymaktadır.

Düşük maliyetli sistemlerin temel zorluğu, sınırlı donanım kaynaklarına rağmen kabul edilebilir bir gözlem ve izleme doğruluğunu korumaktır. Bu zorluk genellikle iki şekilde aşılar: birincisi, hesaplama yükünün bir kısmının bulut servislerine veya daha güçlü ikincil cihazlara aktarılması; ikincisi ise sensör füzyonu ve kapalı çevrim denetim gibi algoritmik tekniklerle donanım eksikliklerinin telafi edilmesidir. Bu tez, ikinci yaklaşımı öne çıkararak, sürekli dönüş servosunun açısız konum geri beslemesi olmadan dahi inersiyal ölçüm birimi (IMU) verisiyle kapalı çevrim içinde sürülebileceğini göstermektedir.

2.2. YAPAY ZEKÂ TABANLI GÖK CİSMİ TESPİTİ VE GÖRÜNTÜ İŞLEME

Son yıllarda derin öğrenme yöntemleri, astronomik görüntülerden gök cismi tespiti ve sınıflandırması alanında geleneksel görüntü işleme tekniklerinin önüne geçmiştir. Su ve arkadaşları, optik teleskop görüntülerinde uzay nesnelerinin gerçek zamanlı tespiti için evrimsel sinir ağı (CNN) tabanlı bir yaklaşım önererek, bu tür modellerin gömülü sistemlerde dahi çalıştırılabilecek kadar verimli hâle getirilebileceğini göstermiştir [1]. Zhou ve arkadaşları ise iyileştirilmiş bir uzay nesnesi tespit yöntemiyle, özellikle zayıf ve küçük hedeflerin tespitinde elde edilen başarıyı artırmıştır [2].

Zayıf ve hareketli nesnelerin tespiti, astronomik görüntü işlemenin en zorlu

problemlerinden biridir; çünkü bu nesneler genellikle düşük sinyal-gürültü oranına sahiptir ve gökyüzü arka planından ayırt edilmeleri güçtür. Jiang ve arkadaşları bu probleme “samanlıkta iğne aramak” benzetmesiyle yaklaşmış ve zayıf nesnelerin tespiti için özelleştirilmiş yöntemler geliştirmiştir [16]. Felt ve Fletcher ise dar alan görüntülerinde yıldız konumlarının öğrenme tabanlı yöntemlerle hassas biçimde belirlenmesini ele almıştır [14]. Bu çalışmalar, yapay zekânın yalnızca sınıflandırma değil, aynı zamanda hassas konum belirleme (astrometri) görevlerinde de kullanılabileceğini ortaya koymaktadır.

Jia ve arkadaşlarının önerdiği akıllı gözlem sistemleri, görüntü işleme ile gözlem planlamasını bütünleştirerek, sistemin yalnızca veri toplayan değil, aynı zamanda topladığı veriyi yorumlayıp gözlem stratejisini buna göre uyarlayan bir yapıya kavuşmasını hedeflemektedir [15]. Bu tez kapsamında geliştirilen prototip, yerel bir CNN/TFLite çıkarımı yerine bulut tabanlı çok-kipli bir büyük dil modeli (Google Gemini) kullanarak yakalanan gök cisminin doğrulanmasını gerçekleştirmektedir; bu, sınırlı gömülü kaynaklarla yüksek seviyeli anlamsal doğrulama elde etmenin pratik bir yolunu temsil etmektedir.

2.3. UZAY NESNELERİNİN GERÇEK ZAMANLI İZLENMESİ

Yörüngedeki nesnelerin (resident space objects) izlenmesi, uzay durumsal farkındalığı (space situational awareness) açısından kritik bir araştırma alanıdır. De Vittori ve arkadaşları, gerçek zamanlı uzay nesnesi izleme için optik sistemlerin nasıl tasarlanabileceğini ayrıntılı biçimde ele almış ve izleme doğruluğunu artıran yöntemler sunmuştur [10]. Baranova ve arkadaşları ise otonom akış (streaming) tabanlı bir gözlem yaklaşımıyla, sürekli veri akışından nesne tespitini gerçekleştiren bir mimari önermiştir [8].

Bu alandaki güncel durumu derleyen kapsamlı tarama çalışmaları, optik gözlem sistemlerinin yörünge takibinde giderek daha fazla otomasyon ve yapay zekâ entegrasyonu içerdiğini göstermektedir [11]. Bu eğilim, gözlem sistemlerinin manuel müdahaleye olan bağımlılığını azaltmakta ve sürekli, kesintisiz gözlem olanağı sağlamaktadır. Bu tezde geliştirilen sistemin otonom yönlendirme ve hedef kilidi mekanizmaları, benzer bir otomasyon felsefesini, amatör ölçekte ve düşük maliyetli donanım ile hayata geçirmeyi amaçlamaktadır.

2.4. AMATÖR ASTROFOTOĞRAFİ VE EKZOGZEĞEN GÖZLEMLERİ

Amatör donanımlarla yürütülen bilimsel gözlemler, özellikle ekzogezen geçişlerinin (transit) tespitinde önemli katkılar sağlamaktadır. Murawski, CMOS kameraların ekzogezen geçiş fotometrisinde kullanımını inceleyerek, görece mütevazı donanım ile dahi geçiş eğrilerinin elde edilebileceğini göstermiştir [12]. Littlefield ise dijital SLR fotoğraf makineleriyle ekzogezen geçişlerinin gözlemlenebileceğini ortaya koyarak, tüketici sınıfı görüntüleyicilerin bilimsel değerini vurgulamıştır [13].

Schanche ve arkadaşlarının çalışması, gözlem verilerinin profesyonel düzeyde işlenmesiyle elde edilebilecek bilimsel sonuçların kapsamını göstermekte ve amatör/yarı-profesyonel gözlemlerin daha büyük araştırma çerçevelerine nasıl entegre edilebileceğine ışık tutmaktadır [3]. Bu çalışmalar topluca, görüntü kalitesinin yalnızca donanımın fiyatına değil, aynı zamanda optik hizalama, odaklama, pozlama stratejisi ve sonraki işleme adımlarının dikkatle uygulanmasına bağlı olduğunu göstermektedir. Bu tezde Ay gözlemleri üzerinden elde edilen sonuçlar da, doğru kolimasyon ve odaklama ile düşük maliyetli bir kamerayla dahi yüzey detaylarının (kraterler, denizler) görünür kılınabildiğini doğrulamaktadır.

2.5. OTONOM VE ROBOTİK TELESKOP KONTROL SİSTEMLERİ

Robotik teleskoplar, gözlem planlamasından veri toplamaya kadar tüm süreci insan müdahalesi olmaksızın yürütebilen sistemlerdir. Husser ve arkadaşları, modüler bir gözlemevi denetim sistemi mimarisi sunarak, teleskop konumlandırma, kubbe kontrolü, kamera yönetimi ve gözlem zamanlaması gibi alt sistemlerin nasıl koordine edilebileceğini ayrıntılı biçimde tartışmıştır [9]. Bu tür mimarilerde temel ilke, her bir donanım bileşeninin iyi tanımlanmış bir yazılım arayüzü ardına soyutlanması ve üst seviye bir denetleyicinin bu arayüzler üzerinden tüm sistemi yönetmesidir.

Bu tez kapsamında geliştirilen prototip, aynı katmanlı soyutlama ilkesini benimser: Arduino Mega düşük seviyeli gerçek zamanlı denetimi (sensör füzyonu ve servo sürüşü) üstlenirken, ESP32-CAM ağ geçidi ve kamera görevini, Android uygulaması ise üst seviye kullanıcı denetimi ve gözlem planlamasını üstlenmektedir. Böylece, profesyonel gözlemevi denetim sistemlerinde görülen görev ayrıştırması, çok daha küçük ve uygun maliyetli bir ölçekte yeniden üretilmiştir. Bu yapı, sistemin gelecekte yerel CNN çıkarımı veya bulut tabanlı görev kuyruğu gibi bileşenlerle genişletilmesine de olanak tanımaktadır.

2.6. OPTİK HİZALAMA (KOLİMASYON) VE GÖRÜNTÜ KALİTESİ

Newtonian yansıtımlı teleskoplarda görüntü kalitesini belirleyen en kritik etkenlerden biri, aynaların optik eksene göre doğru hizalanması, yani kolimasyondur. Seronik, kolimasyon sürecini adım adım ele alan kapsamlı bir başlangıç rehberi sunarak, hatalı hizalamanın görüntüde yarattığı bozulmaları ve bunların nasıl düzeltilileceğini açıklamıştır [19]. Sky & Telescope editörleri de Newtonian bir teleskobun hizalanmasına dair pratik bir yöntem sunmaktadır [20].

Hatalı kolimasyon, optik eksenin kayması nedeniyle yıldız görüntülerinin asimetrik (koma benzeri) bir biçim almasına ve genel keskinliğin düşmesine yol açar. Bu nedenle, düşük maliyetli bir görüntüleyici kullanılsa dahi, doğru kolimasyon ve hassas odaklama, elde edilen görüntü kalitesini belirgin biçimde artırmaktadır. Bu tezde de gözlemler öncesinde kolimasyon ve odak kontrolü yapılmış; ilgili bölümlerde iyi odaklanmış ve odağı kaçmış görüntülerin karşılaştırması sunularak bu etkinin niteliksel kanıtı verilmiştir.

2.7. EFEMERİS HESABI VE KOORDİNAT KÜTÜPHANELERİ

Bir gök cisminin belirli bir zamanda ve konumda gökyüzündeki yerini belirlemek, otonom yönlendirmenin temelini oluşturur. Bu hesaplama için literatürde ve uygulamada çeşitli olgun kütüphaneler ve servisler bulunmaktadır. Astropy, zaman ve koordinat dönüşümleri için yaygın olarak kullanılan kapsamlı bir Python kütüphanesidir [24]; Skyfield ise yüksek doğruluklu efemeris hesapları için tercih edilen bir başka araçtır [34]. Bulut tabanlı çözümler arasında AstronomyAPI gibi servisler, gök cisimlerinin konumlarını bir REST arayüzü üzerinden sunmaktadır [23].

Bu tezde geliştirilen Android uygulaması, dış bir servise veya ağ bağlantısına bağımlı kalmamak için, ekvatorial koordinatları yerel ufuk koordinatlarına (azimut/yükseklik) dönüştüren hafif bir efemeris modülünü cihaz üzerinde (on-device) çalıştırmaktadır. Bu tasarım tercihi, sahada internet erişiminin olmadığı durumlarda dahi sistemin hedef hesaplarını yapabilmesini sağlamak ve gecikmeyi en aza indirmektedir. Böylece olgun kütüphanelerin sunduğu doğruluk ilkeleri korunurken, gömülü/mobil bir bağlamda çalışabilen bağımsız bir çözüm elde edilmiştir.

2.8. LİTERATÜRDEKİ BOŞLUK VE BU ÇALIŞMANIN KATKISI

Yukarıda özetlenen çalışmalar incelendiğinde, mevcut sistemlerin büyük ölçüde iki uça yoğunlaştığı görülmektedir: bir yanda yüksek maliyetli, profesyonel gözlemevlerine yönelik tam otomatik sistemler [9], [10], [11]; diğer yanda ise belirli bir göreve (örneğin

yalnızca ekzogezen geiři veya yalnızca meteor tespiti) odaklanan, görece dar kapsamlı amatör çözümler [5], [12], [13]. Düşük maliyetli donanımı, otonom yönlendirmeyi, yapay zekâ destekli doğrulamayı ve kullanıcı tarafından büyütölen bir veri arşivini tek bir uçtan uca akışta birleřtiren bütönlöřik çalıřmalar görece sınırlıdır.

Bu tez, söz konusu boşluęa doğrudan karřılık veren bütönlöřik bir mimari önermektedir. Sistem; (i) düşük maliyetli ve erişilebilir bileřenlerle [4], [6], [17] kurulan iki eksenli bir mekanik düzeneęi, (ii) IMU ve GPS verisine dayalı kapalı çevrim otonom yönlendirmeyi, (iii) yakalanan görüntünün yapay zekâ ile doğrulanmasını [1], [2] ve (iv) doğrulanmış gözlemlerin kullanıcılar tarafından paylaşıldığı, zaman içinde büyüyen bir web tabanlı arşivi tek bir çalıřır prototipte bir araya getirmektedir. Böylece çalıřma, hem teknik gerekleřtirme hem de topluluk temelli veri üretimi boyutuyla literatüre özğün bir katkı sunmayı hedeflemektedir.

Sonuç olarak bu bölümde ele alınan tarihsel ve teknik çereve, geliştirilen sistemin neden gerekli olduęunu açıka ortaya koymaktadır. Manuel kontrol gerektiren teleskop sistemleri artık modern astronominin taleplerine yanıt verememekte; yapay zekâ, otomasyon ve açık veri kavramlarıyla zenginleşen yeni nesil yapılar, hem bilimsel üretimi hem de topluluk astronomisini destekleyecek düzeye ulaşmıştır.

3. MALZEME VE YÖNTEM

Bu bölümde arařtırmada kullanılan donanımlar, yazılım bileřenleri, veri toplama yöntemi, yapay zekâ temelli sınıflandırma süreci, teleskop yönlendirme mekanizması, cihazlar arası iletişim yapısı ve gözlem deneyinin nasıl yürütöldüğü ayrıntılı biçimde açıklanmaktadır. Kullanılan tüm malzemeler, sistemin bir bütönlörlönde çalıřmasını saęlayan temel bileřenler olarak ele alınmış; her bir bileřenin arařtırma sürecindeki işlevi, görev kapsamı ve dięer bileřenlerle ilişkisi sistematik bir çereve içinde sunulmuştur.

3.1. KULLANILAN DONANIM BİLEŐENLERİ

Arařtırmada kullanılan donanımlar, hem teleskop tabanlı gözlem mekanizmasını hem de otonom kontrolö mümkün kılan elektronik bileřenleri içermektedir. Bu donanımlar, optik sistemler, sensörler, mikrodenetleyiciler, motor ve motor sürücö devreleri ile güç bileřenlerinden oluşmaktadır.

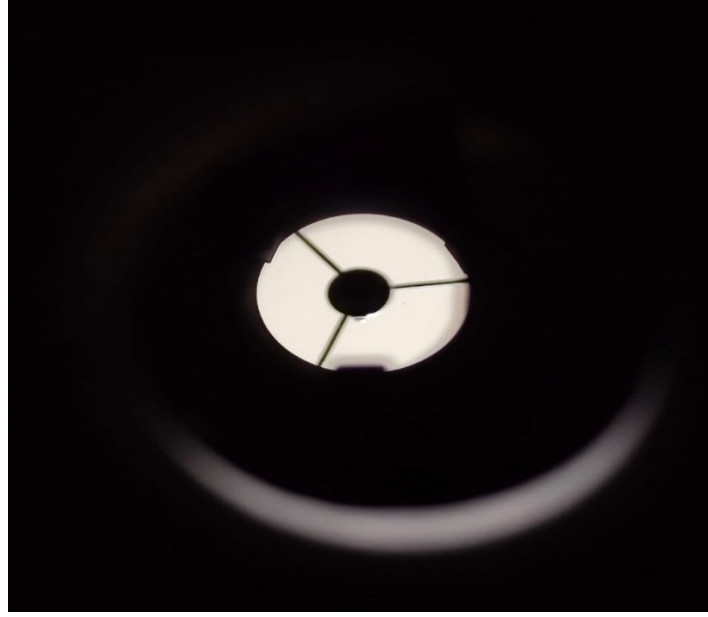
3.1.1. TELESKOP SİSTEMİ

Gözlem mekanizmasının temelini oluşturan teleskop, 76 mm açıklığa ve 700 mm odak uzaklığına sahip Newton tipi bir reflektör modelidir. Bu teleskop gök cisimlerinin görüntölenmesi için temel optik sistemi saęlamaktadır. Kullanılan teleskop, küçük açıklığına rağmen gezegen gibi parlak gök cisimlerinin kaydedilmesine uygun olup, geniş amatör topluluklarda bilinen başlangıç seviyesi bir modeldir. Farklı odak uzaklıklarına sahip göz merceklöri (SR4, H6, H12.5, H20 mm ve Plössl marka lensler) gözlem koşullarına göre deęiřtirilerek hedef cismin büyötmö düzeyi ayarlanmıştır. Bu lens seti, özellikle gezegenlerin farklı parlaklık ve konum koşullarına göre gerektiğinde daha geniş veya daha dar görüntö alanı elde edebilmek için kullanılmıştır.

Proje sürecinde teleskop, farklı ortamlara taşınması ve saha kullanımı sırasında maruz kaldığı küçük çarpmalar ve titreşimler nedeniyle optik hizasını (kolimasyon/mercek–odak doğrultusu) kısmen kaybetmiştir. Bu durum; görüntüde netlik azalması, odaklamada güçlük ve özellikle parlak cisimlerin çevresinde dağılma/bozulma şeklinde gözlemlenmiştir. Ayrıca uzun pozlama veya ardışık çekimlerde yıldızların noktasal görünümü bozulmuş, ışık yoğunluğu sensör üzerine homojen dağılmadığı için görüntü kalitesi tutarsız hale gelmiştir. Bu tür hizalama sorunları, hem tespit (gök cismi tanıma) doğruluğunu hem de veri seti üretiminde elde edilen görüntülerin karşılaştırılabilirliğini olumsuz etkileyebilmektedir. Bu nedenle optik bileşenlerin mekanik bağlantıları kontrol edilerek kolimasyon ayarları yeniden yapılmış ve odak doğrultusu iyileştirilmiştir. Aşağıdaki iki görselde, hizalama bozulmasının mercek hizasına etkisi ile gerekli kolimasyon ayarları yapıldıktan sonra elde edilen iyileşme örnek olarak sunulmaktadır.



Şekil 1. Kolimasyon bozulmasının optik eksene etkisi.



Şekil 2. Kolimasyon düzeltmesi sonrası elde edilen doğru hizalama.

3.1.1.1. MERCEK AYARI VE OPTİK KOLİMASYON SÜRECİ

Amatör astronomi ve uygulama alanlarında “mercek ayarı” ifadesi çoğu zaman tek bir işlemi değil, teleskobun görüntü kalitesini doğrudan etkileyen iki temel ayar türünü birlikte kapsayan genel bir tanım olarak kullanılır. Bu kapsamın ilk bileşeni kolimasyondur; yani birincil ve ikincil aynaların (ve buna bağlı olarak odaklayıcı ekseninin) ortak bir optik eksen üzerinde doğru şekilde hizalanmasıdır. İkinci bileşen ise odaklamadır; yani oluşan görüntünün odak düzleminin göz merceği ya da kamera sensörü üzerinde en doğru noktaya getirilmesidir. Özellikle Newton tipi yansıtmalı teleskoplarda görüntüyü oluşturan ana eleman bir mercek değil, tüpün arka kısmında yer alan parabolik birincil aynadır. Bu ayna gökyüzünden gelen ışığı belirli bir odak düzleminde toplar; tüp ağzına yakın konumlanan ikincil (diyagonal) ayna ise birincil aynadan gelen ışığı odaklayıcıya yönlendirerek görüntünün oküler veya sensör üzerinde oluşmasını sağlar. Dolayısıyla Newtonian sistemlerde günlük kullanımda “mercek ayarı” denildiğinde pratikte çoğunlukla ayna hizalaması (kolimasyon) ve bunu takip eden fokus optimizasyonu kastedilir.

Kolimasyonun temel hedefi, teleskobun en keskin görüntüyü verdiği optik eksenin gözlemci/sensör eksenineyle çakışmasını sağlamaktır. Kolimasyon doğru değilse odak “yakalanmış gibi” görünse bile, özellikle yüksek büyütmelede yıldızlar noktasal yapısını kaybeder; gezegen ayrıntıları yumuşar, kontrast düşer ve görüntüde yönlü/asimetrik bozulmalar artar. Bu nedenle kolimasyon, Newtonian teleskoplarda performansı belirleyen en kritik bakım ve kalibrasyon adımlarından biri olarak değerlendirilir. Odaklama ise kolimasyon doğru şekilde tamamlandıktan sonra görüntü keskinliğini maksimize eden son aşamadır; bir anlamda sistemin potansiyelini görünür hale getirir. Bu tez kapsamında “mercek ayarı” konusu yalnızca görsel gözlem kalitesi açısından değil,

aynı zamanda görüntü işleme ve yapay zekâ tabanlı tanıma algoritmalarının kararlılığı, veri seti üretiminde standart görüntü kalitesinin korunması ve sistemin otonom çalışmasındaki güvenilirlik açısından da kritik görülmüştür. Bu sebeple mercek ayarı başlığı altında kolimasyon ve odaklamanın yöntemi, uygulanışı, kontrol adımları ve sahada doğrulanması ayrıntılı biçimde ele alınmıştır.

3.1.1.2. NEWTONIAN OPTİK SİSTEMİNİN TEMEL YAPISI VE EKSEN KAVRAMLARI

Newton tipi reflektör teleskoplar temel olarak birincil ayna, ikincil ayna, odaklayıcı ve ikincil aynayı taşıyan spider (örümcek) yapısından oluşur. Birincil ayna, ışığı toplayıp odak düzleminde görüntüyü oluşturan parabolik yüzeydir. İkincil (diyagonal) ayna ise birincil aynadan gelen ışığı yaklaşık 90 derece saptırarak odaklayıcıya taşır ve görüntünün oküler ya da sensöre iletilmesini sağlar. Odaklayıcı, oküler veya kamerayı odak düzlemine göre ileri-geri hareket ettirerek fokus ayarının yapılmasına imkân veren mekanizmadır. Spider yapısı da ikincil aynayı tüp merkezinde konumlandıran ince kollardan oluşur ve sistemin mekanik stabilitesinde rol oynar.

Bu optik düzenekte “hizalama” ifadesi, gerçekte üç ayrı geometrik/optik referansın birbirleriyle doğru ilişkiye getirilmesini amaçlar. Bunlardan ilki odaklayıcı eksenidir; oküler veya kamera doğrultusunu temsil eder. İkincisi birincil aynanın optik eksenidir; parabolik aynanın simetri doğrultusu olup en iyi görüntü bu eksen üzerinde elde edilir. Üçüncüsü ise ikincil aynanın yönlendirme doğrultusudur; birincilden gelen ışığın odaklayıcıya eksele ve simetrik biçimde taşınmasını sağlayan ayna konumu ve eğimini ifade eder. Kolimasyonun başarısı bu üç bileşenin birlikte doğru hizalanmasına bağlıdır ve Newtonian teleskoplarda genellikle sistematik bir sıra izlenir. Önce ikincil aynanın odaklayıcı altında geometrik konumu düzeltilir; ardından ikincil aynanın eğimi ayarlanarak birincil aynanın merkezi “hedeflenir”; son aşamada ise birincil aynanın hassas eğimi yapılarak optik eksen odaklayıcı eksenine eşleştirilir. Bu sıralama, bir adımda yapılan hatanın sonraki adımlarda büyümesini engeller ve tekrarlanabilir, kontrollü bir kolimasyon süreci sağlar.

3.1.1.3. HATALI KOLİMASYONUN GÖRÜNTÜ KALİTESİNE ETKİLERİ

Kolimasyon bozulduğunda ortaya çıkan etkiler hem görsel gözlemlerde hem de astrofotoğrafik kayıtlarda belirgin hale gelir. Bu etkiler özellikle yüksek büyütme oranlarında ve hızlı optik sistemlerde (düşük f oranlarında) daha kolay fark edilir, çünkü optik eksenindeki küçük kaçıklıklar dahi görüntüye daha güçlü şekilde yansır. Kolimasyon hatasının en tipik sonucu, sistemin en net olması beklenen bölgesinin kaymasıdır; gözlemci hedefi görüş alanının merkezine alsa bile optimum keskinliğe ulaşmak zorlaşır. Yıldız görüntülerinde noktasal yapı yerine asimetri ve kuyruklanma görülebilir; bu bozulma yalnızca odak ayarıyla giderilemeyen “kalıcı bir bulanıklık” hissi oluşturur. Benzer biçimde gezegen gözlemlerinde ince detaylar yumuşar; Jüpiter’in bantları, Satürn’ün halka yapısı veya Ay üzerindeki küçük krater ayrıntılarında kontrast azalır ve beklenen

keskinlik kaybolur. Astrofotoğraf tarafında ise alan dengesizliği dikkat çeker: Çerçevenin bir köşesinde yıldızlar farklı yönde bozulurken diğer köşede farklı bir deformasyon görülebilir ve bazı bölgelerde odak iyi görünürken bazı bölgelerde “odak kaçmış” izlenimi oluşur. Bu durum yalnızca görsel kaliteyi düşürmekle kalmaz, görüntü işleme ve yapay zekâ tabanlı analizlerde de kararsızlığa neden olabilir; kenar/özellik çıkarımı zorlaşır, aynı sınıftaki nesnelerin görünümü tutarsızlaşır ve yanlış pozitif/yanlış negatif oranlarında artış gözlenebilir. Bu nedenle kolimasyon, yalnızca “görüntüyü güzelleştirme” amacı taşıyan bir işlem değil, sistemin ölçüm ve algoritma tabanlı çıktı kalitesini doğrudan belirleyen kritik bir kalibrasyon adımı olarak değerlendirilmelidir.

Kolimasyon işlemi optik yüzeylere yakın çalışmayı gerektirdiğinden, hem mekanik güvenlik hem de optik yüzeylerin korunması önem taşır. İşleme başlamadan önce teleskobun devrilmeyecek ve ani hareket etmeyecek şekilde sabitlenmesi gerekir; Dobson taban ya da tripod kullanılıyorsa kilit mekanizmaları kontrol edilmelidir. Kolimasyonun ilk kurulum aşamasında gündüz veya iyi aydınlatılmış bir ortam tercih edilmesi, optik elemanları daha rahat görmeyi sağlar; gece yapılacak uygulamalarda ise göz kamaştırmayan, yumuşak bir ışık kaynağı kullanmak daha uygundur. Aynalar ve kaplamalar hassas olduğundan, parmak izi, çizik veya kimyasal temas görüntü kalitesini kalıcı biçimde düşürebilir; bu yüzden doğrudan ayna yüzeyine temas edilmemeli ve temizlik/koruma konusunda dikkatli olunmalıdır. Kolimasyon aracı kullanılacaksa oküler çıkarılır, odaklayıcı bölgesi temizlenir ve odaklayıcıdaki mekanik gevşeklikler kontrol edilir; çünkü odaklayıcıda boşluk olması hizalama okumasını yanıltabilir. Birincil aynada merkez işaretinin bulunması da kritik bir noktadır; merkez işareti yoksa veya hatalı konumlandırılmışsa kolimasyonun güvenilirliği azalır ve yapılan ayarlar tekrarlanabilir sonuç vermeyebilir. Son olarak ayar vidalarının işlevi anlaşılmadan işleme başlanmamalıdır; birincil ayna kolimasyon vidaları ile varsa kitleme vidaları (lock) ve ikincil aynanın merkez vidası ile eğim vidaları birbirine karıştırıldığında ayar daha da bozulabilir. Kolimasyonun “hızlıca çevirme” yaklaşımıyla yapılması hatayı büyütebildiği için, her adımın küçük hareketlerle ve arada kontrol edilerek ilerletilmesi doğru yöntemdir.

Kolimasyon farklı hassasiyet seviyelerinde farklı araçlarla gerçekleştirilebilir ve her aracın güçlü olduğu bir kullanım alanı vardır. Bu tez çalışmasında hedef, sahada da uygulanabilir ve tekrarlanabilir bir yöntem sunmak olduğundan araçlar işlevlerine göre değerlendirilmiştir. Basit bir kolimasyon kapağı (pinhole cap) ile odaklayıcı eksenini kabaca referans almak mümkündür; bu yöntem temel hizalama ve hızlı kontrol için pratik bir başlangıç sağlar. İkincil aynanın odaklayıcı altında doğru konumlandırılmasında sight tube (bakış tüpü) gibi araçlar öne çıkar; ikincilin geometrik merkezlemesi ve uygun yerleşimi açısından oldukça etkilidir. Birincil aynanın hassas eğim ayarı için Cheshire, güvenilir bir standart olarak kabul edilir ve “son ayar” aşamasında yüksek doğruluk sağlar. Lazer kolimatör ise hız avantajı nedeniyle sahada zaman kazandırabilir; ancak cihazın kendi doğruluğu, odaklayıcıya tam oturması ve mekanik toleranslar sonucu ciddi şekilde etkileyebildiğinden tek başına nihai doğrulama aracı olarak görülmemelidir. Nihai

doğrulamada yıldız testi çok güçlü bir yöntemdir; bununla birlikte atmosfer koşulları ve teleskobun termal dengesi sonuçları etkileyebileceği için uygun şartlarda uygulanmalıdır. Uygulamada en güvenilir yaklaşım, ikincil aynayı sight tube ile doğru konuma getirmek, birincil aynayı Cheshire ile hassas ayarlamak ve ardından yıldız testiyle son doğrulamayı yapmaktır.

Kolimasyon sürecinin ilk aşaması, ikincil aynanın odaklayıcı altında geometrik olarak doğru konumda olduğundan emin olmaktır. Bu aşamada amaç, odaklayıcıdan bakıldığında ikincil aynanın mümkün olduğunca dairesel ve merkezde görünmesini sağlamaktır. Kolimasyon kapağı veya sight tube odaklayıcıya takılarak içeri bakıldığında ikincil ayna çoğu zaman elips formunda görülür ve bu elipsin odaklayıcı açıklığına göre ortalanması hedeflenir. İkincil ayna merkezden kayıksa, örümcek kollarının gerginliği kontrol edilmeli ve kolların eşit gerginlikte olduğundan emin olunmalıdır; çünkü dengesiz gerginlik ikincilin tüp içinde kaymasına neden olabilir. Gerekirse ikincil aynayı taşıyan yapının tüp içindeki ileri-geri konumu çok küçük miktarlarda ayarlanarak merkezleme yapılır. Bazı durumlarda ikincil aynanın aşırı sağ/sol ya da yukarı/aşağı kayması, odaklayıcının tüpe montaj açısının da kontrol edilmesini gerektirebilir; odaklayıcı hafif yamuk monte edilmişse ikincili “doğru” göstermeye çalışırken bir noktada ilerleme sağlanamayabilir. Bu aşama genellikle bir kez doğru yapıldığında uzun süre korunur; ancak teleskop darbe aldıysa, mekanik parçalar değiştirildiyse veya yeni bir odaklayıcı takıldıysa yeniden kontrol edilmesi gerekir.

İkincil aynanın konumu doğruya getirildikten sonraki aşama, ikincil aynanın eğimini ayarlayarak odaklayıcı ekseninin birincil aynanın merkez işaretine bakmasını sağlamaktır. Sight tube üzerinde nişangâh çizgileri bulunuyorsa bu çizgiler birincil aynanın merkez işaretiyle karşılaştırılır ve ikincil aynanın eğim vidaları çok küçük adımlarla ayarlanır. Birincil aynanın merkez işareti, odaklayıcı ekseninde tam ortada görünene kadar bu işlem sürdürülür. Bu aşamada sık yapılan hatalardan biri ikincil aynanın rotasyonunun (kendi eksenini etrafındaki dönüklüğünün) yanlış olmasıdır. Rotasyon hatalıysa, eğim vidalarıyla merkez işareti belirli bir noktaya getiriliyor gibi görünse bile birincil ayna yansıması simetrik bir geometri oluşturmaz; bu durumda ikincil ayna çok hafif gevşetilerek rotasyon düzeltilir ve eğim ayarı yeniden yapılır.

Newtonian sistemin performansını asıl belirleyen adım ise birincil aynanın optik eksenini odaklayıcı eksenine karşılaştıran hassas eğim ayarıdır. Bu aşamada Cheshire odaklayıcıya takılır ve görüntüde birincil aynanın merkez işareti ile Cheshire’in referans deseni birlikte izlenir. Birincil aynanın arkasındaki kolimasyon vidaları küçük hareketlerle çevrilerek merkez işaretinin referans desenle üst üste gelmesi sağlanır. Ayar yapılırken bir vidayı aşırı uçlara zorlamak yerine, iki veya üç vida arasında dengeli bir düzenleme yapmak daha stabil sonuç verir. İşlem tamamlandığında desenlerin eşmerkezli görünmesi ve merkez işaretinin tam ortada yer alması beklenir.

Kolimasyon doğru yapılsa bile doğru odaklama gerçekleştirilmeden beklenen keskinliğe ulaşmak mümkün değildir. Görsel gözlemde odaklama için parlak bir yıldız veya Ay kenarı gibi yüksek kontrastlı bir hedef seçilir; odak yavaşça ayarlanır ve yıldız görüntüsünün en küçük, en keskin noktaya ulaştığı konum optimum odak olarak kabul edilir. Kamera ile yapılan uygulamalarda ise sensör üzerinden büyütme/zoom kullanılarak yıldız görüntüsünün boyutu ve simetrisi izlenir; yıldızın en küçük ve en simetrik olduğu noktada odak sabitlenir ve mümkünse odak kilidi uygulanır. Odaklamada mekanik boşluklar kritik bir etkindir; odaklayıcıda oynama varsa odak kısa sürede kaçabilir ve oturumlar arasında görüntü kalitesi tutarsız hale gelebilir. Bu nedenle odaklayıcının mekanik sağlamlığı, pratikte kolimasyon kadar önemli bir unsurdur ve veri seti üretiminde standardizasyon için özellikle dikkate alınmalıdır.

3.1.1.4. KOLİMASYONUN DOĞRULANMASI: YILDIZ TESTİ, GÖRSEL VE FOTOĞRAFİK GÖSTERGELER

Kolimasyonun gerçek gözlem koşullarında doğru yapıp yapılmadığını anlamamanın en güvenilir yollarından biri yıldız testidir. Bu yöntemde gökyüzünde parlak ve tekil bir yıldız seçilerek yüksek büyütmeye geçilir. Ardından yıldız çok az odak dışına alınır ve oluşan dairesel halkaların simetrisi incelenir. Kolimasyon iyi ise halkalar merkezlenmiş ve simetrik görünür; kolimasyon kötü ise merkezdeki gölge kayar ve halkalar belirgin şekilde asimetrikleşir. Yıldız testi uygulanırken atmosferik koşulların (seeing) önemli bir değişken olduğu unutulmamalıdır. Seeing kötü olduğunda halkalar “zıplıyor” gibi görünebilir ve bu durum kolimasyon hatasıyla karıştırılabilir. Bu nedenle yıldız testi, teleskop termal dengeye geldikten sonra ve mümkünse gökyüzünün daha stabil olduğu zaman aralıklarında yapılmalıdır.

Kolimasyon sorunları çoğu zaman yalnızca testle değil, gözlemsel ipuçlarıyla da kendini belli eder. Örneğin görüntü alanının merkezindeki yıldızlar dahi kuyruklu veya uzamış görünüyorsa kolimasyonun kontrol edilmesi gerekir. Benzer şekilde gezegen detayları beklenenden silikse, odak doğru görünse bile optik hizalama yeniden değerlendirilmelidir. Ay gözlemlerinde terminatör çizgisi gibi yüksek kontrastlı kenarlarda çiftlenme ya da belirgin bir yumuşama görülmesi de hizalama veya odaklama ile ilişkili bir probleme işaret edebilir.

Fotoğrafik çıktılar ise kolimasyonun daha sistematik şekilde yorumlanmasına imkân verir. Çerçevenin bir tarafındaki yıldız şekilleri diğer tarafa göre belirgin biçimde farklılaşıyorsa, yalnızca kolimasyon değil sensör düzlemi ile optik eksen ilişkisi de birlikte ele alınmalıdır. Bunun yanında, görüntü merkezinde bile yıldızların noktasal olmaktan uzak olması kolimasyon hatasına veya odaklayıcı mekanizmada bir mekanik probleme işaret edebilir.

Bu tez kapsamında geliştirilen teleskop tabanlı sistemde kolimasyon, yalnızca ilk kurulum aşamasında yapılan tek seferlik bir işlem olarak değil, veri kalitesini standardize eden düzenli bir saha rutini olarak ele alınmıştır. Önerilen yaklaşımda öncelikle

laboratuvar veya ev ortamında ayrıntılı kolimasyon yapılır; ikincil ayna merkezleme ve hedefleme adımlarının ardından birincil aynada hassas ayar gerçekleştirilir. Sahaya çıkmadan önce kısa bir kontrol ile birincil aynada hızlı bir Cheshire doğrulaması yapılır. Saha kurulumunda teleskop kurulduktan sonra kısa bir yıldız testi uygulanarak hizalamanın gerçek gözlem koşullarında da doğru sonuç verdiği doğrulanır. Veri toplamaya geçmeden önce, özellikle ortam sıcaklığında belirgin bir değişim varsa, teleskobun termal dengeye gelmesi için kısa bir süre beklenir ve ardından yeniden kısa bir kontrol yapılır. Ayrıca farklı günlerde toplanan verilerin karşılaştırılabilirliğini artırmak için her oturumun başlangıcında aynı kontrol adımlarının tekrarlanması hedeflenir. Bu disiplin, yapay zekâ tabanlı tanıma veya veri seti üretimi süreçlerinde görüntü kalitesindeki dalgalanmaların modele yansımaları azaltır ve aynı sınıftaki nesnelerin daha tutarlı görünmesine katkı sağlar.

Uygulamada sık karşılaşılan durumlardan biri, optik bileşenlerin “ortalanmış” görünmesine rağmen görüntünün hâlâ yeterince net olmamasıdır. Bu durumda teleskop termal dengeye gelmemiş olabilir, seeing koşulları kötü olabilir veya odaklayıcıda mekanik boşluk bulunabilir. Ayrıca birincil ayna çok sıkı tutuluyorsa yüzey gerilimi oluşarak görüntü kalitesini düşürebilir. Bu tür durumlarda termal denge için zaman tanınması, odaklayıcıdaki boşluğun kontrol edilmesi ve ayna klemplerinin aşırı sıkıysa gevşetilmesi önerilir; sonrasında yıldız testi tekrar edilerek durum doğrulanır.

Bir diğer yaygın sorun, ikincil aynanın bir türlü simetrik görünmemesidir. Bunun nedeni ikincil aynanın rotasyonunun hatalı olması, örümcek (spider) kollarının eşit gerginlikte olmaması veya odaklayıcının tüpe tam dik monte edilmemiş olması olabilir. Bu senaryoda ikincil aynanın rotasyonu yeniden kontrol edilir, spider gerginliği dengelenir ve gerekirse odaklayıcı montaj açısı incelenir.

Bazı durumlarda lazer kolimatör “doğru” bir hizalama gösterirken yıldız testinin “kötü” sonuç vermesi mümkündür. Bu çelişkinin kaynağı lazer kolimatörün kendi doğruluğunun bozuk olması, odaklayıcıya oturma toleranslarının lazeri yanıltması veya ikincil hedefleme aşamasındaki küçük bir hatanın büyüyerek etkili hale gelmesi olabilir. Bu nedenle lazer yönteminin daha çok hızlandırıcı bir araç olarak görülmesi, nihai doğrulamanın Cheshire ve yıldız testi gibi pasif yöntemlerle yapılması daha güvenilir bir yaklaşım sağlar.

Kolimasyonun sık sık bozulması ise genellikle mekanik kaynaklıdır. Mekanik gevşeklik, ayna hücresi esnemesi veya odaklayıcının oynaması gibi problemler buna sebep olabilir; ayrıca tüp farklı açılarda eğilme (fleks) gösterebilir. Bu durumda bağlantı elemanları kontrol edilir ve kritik gözlem öncesinde teleskop, hedefe bakılan açığa yakın bir konumdayken kolimasyonun yeniden doğrulanması önerilir.

Newtonian teleskoplarda ikincil ayna, spider adı verilen ince kollarla taşınır. Bu yapı özellikle parlak yıldızlarda görülen “çapraz ışınlar” (diffraction spikes) üretir ve bu izler zaman zaman kolimasyon hatası sanılabilir. Ancak bu durum, optik hizalamadan ziyade

mekanik destek geometrisinden kaynaklanan fiziksel bir optik etkidir. Dolayısıyla kolimasyon mükemmel olsa bile difraksiyon izleri tamamen kaybolmayabilir. Görüntü kalitesini artırmak için spider geometrisinin optimize edilmesi, yani kolların kalınlığı, sayısı ve konumlandırmasının düzenlenmesi bir yaklaşım sunar. Bunun yanında bazı tasarımlarda spider kollarına eklenen özel profiller veya apodizer benzeri çözümlerle difraksiyon izlerinin şiddeti azaltılabilir. Bu tez bağlamında konu iki açıdan önem taşır: Görüntüdeki çizgisel izlerin yanlışlıkla “kolimasyon sorunu” olarak yorumlanmasını önlemek ve görüntü işleme ile veri seti üretimi aşamalarında parlak yıldız çevresindeki çizgisel yapıların algoritmaları etkileme riskine dikkat çekmek. Bu nedenle kolimasyon süreci doğru yapıldıktan sonra görüntü hâlâ belirgin difraksiyon izleri içeriyorsa, bunun hizalama hatası değil, spider kaynaklı difraksiyon etkisi olabileceği değerlendirilmelidir.

Newtonian teleskoaplarda “mercek ayarı” olarak ifade edilen uygulama, pratikte kolimasyon + odaklama süreçlerinin birleşimidir. Kolimasyon; ikincil aynanın odaklayıcı altında doğru konumlandırılması, ikincil eğimin birincil merkeze yönlendirilmesi ve birincil aynanın hassas eğim ayarı adımlarından oluşur. Bu adımlar doğru uygulandığında teleskobun keskinlik ve kontrast performansı maksimuma çıkar; aynı zamanda görüntü işleme ve yapay zekâ tabanlı tanıma süreçleri için daha tutarlı ve kaliteli veri üretimi sağlanır. Son doğrulamanın yıldız testiyle yapılması, saha koşullarında güvenilir bir performans standardı oluşturur. Ek olarak, kolimasyon dışında kalan spider difraksiyonu gibi mekanik kaynaklı optik etkiler de görüntü değerlendirmesinde ayrıca dikkate alınmalıdır.

3.1.2. GÖRÜNTÜLEME BİRİMİ

Tez başlangıç aşamasında IMX477 gibi yüksek performanslı kamera modülleri değerlendirilmiş olsa da, maliyet-etkinlik kriterleri doğrultusunda ESP32-CAM modülü sisteme entegre edilmiştir. IMX477 sensörünün sunduğu yüksek çözünürlük, geniş dinamik aralık ve düşük gürültü değerleri göz önünde bulundurulmuş; ancak toplam sistem maliyetini önemli ölçüde artırması, güç tüketiminin daha yüksek olması ve özellikle taşınabilir bir gözlem düzeni hedefleyen araştırma kapsamında ek donanım gereksinimleri doğurması nedeniyle sistem tasarımından çıkarılması uygun görülmüştür. Buna karşın ESP32-CAM modülü, düşük çözünürlüklü olmasına rağmen gezegen gibi parlak gök cisimlerinin görüntülenmesi için yeterli performans sunmakta, özellikle parlaklık seviyesi yüksek hedeflerde net görüntüler oluşturabilmekte ve sistemin kompakt, düşük maliyetli, enerji verimli bir yapı kazanmasına katkı sağlamaktadır [12]. ESP32-CAM modülünün tercih edilme nedenlerinden biri de sistemin taşınabilirlik ve erişilebilirlik prensipleri gözetilerek tasarlanmasıdır. Modülün küçük form faktörü sayesinde hem teleskop üzerine ek yük bindirilmemekte hem de kullanım sırasında denge problemleri oluşmamaktadır.

Ayrıca modülün yerleşik Wi-Fi özelliği, verilerin anlık olarak Raspberry Pi’ye veya yerel ağa aktarılmasını kolaylaştırmakta; kablo yoğunluğunu azaltarak teleskop çevresindeki düzeneklerin sade kalmasını sağlamaktadır. Bu özellik, özellikle gece gözlemlerinde sistemin pratik kullanımını artırmakta ve kurulum süresini kısaltmaktadır.

Kameranın gövdeye entegrasyonu, optik eksen ile sensör düzlemi arasında hizalama hatalarını en aza indirmek amacıyla 3D yazıcı ile üretilmiş özel bir adaptör aracılığıyla yapılmıştır. Bu adaptör, teleskopun odak düzlemi ile ESP32-CAM'in sensör yüzeyi arasında doğru mesafenin korunmasını sağlayacak şekilde tasarlanmış, optik ekseninde oluşabilecek sapmaların azaltılması için tolerans değerleri göz önünde bulundurularak üretilmiştir. Üretim sırasında teleskopun odak noktasına ulaşabilmesi için gerekli geri odak aralığı dikkate alınmış; modül gövdeye mekanik olarak sağlam bir biçimde bağlanırken titreşim kaynaklı görüntü kayıplarını en aza indirecek bir yapıda monte edilmiştir.

Ayrıca adaptörün 3D baskı yöntemiyle üretilmiş olması, farklı sensörlerin veya farklı lens yapılandırmalarının gelecekte sisteme entegre edilebilmesine olanak tanıyan esnek bir platform sağlamıştır. Bu durum, tez kapsamında donanımın kolayca değiştirilebilir, geliştirilebilir ve çeşitli gözlem koşullarına uyarlanabilir olmasını mümkün kılmıştır. Böylece ESP32-CAM, yalnızca ekonomik ve pratik bir çözüm sunmakla kalmamış, aynı zamanda araştırma altyapısı üzerinde yapılabilecek gelecekteki genişletme çalışmalarının da önünü açmıştır.

Prototipte görüntüleme birimi olarak ESP32-CAM modülüne entegre OV2640 sensörü kullanılmıştır. OV2640, düşük maliyetli ve düşük güç tüketimli bir CMOS sensör olup, JPEG sıkıştırma donanım düzeyinde destekleyerek mikrodenetleyici üzerindeki işlem yükünü azaltır. Bu özellik, sınırlı belleğe sahip ESP32 üzerinde canlı görüntü akışının (MJPEG) pratik biçimde gerçekleştirilebilmesini sağlamıştır. Ne var ki sensörün küçük piksel boyutu ve sınırlı dinamik aralığı, özellikle düşük ışıktaki gözlemlenecek gök cisimlerinin çeşitliliğini kısıtlamaktadır.

İdeal tasarımda öngörülen IMX477 gibi daha büyük sensörlü kameralar [32], [33], hem daha yüksek çözünürlük hem de belirgin biçimde daha iyi düşük ışık başarımı sunar. Bu nedenle görüntüleme birimi, prototipin en belirgin yükseltme noktalarından biri olarak değerlendirilmektedir. Bununla birlikte, parlaklığı ve açısal boyutu yüksek olan Ay gibi hedeflerde OV2640 ile dahi yüzey detaylarının (kraterler, denizler) seçilebildiği saha testlerinde gösterilmiştir; bu durum, düşük maliyetli sensörlerin uygun hedeflerde tatmin edici sonuç verebildiğini doğrulamaktadır.

3.1.3. MİKRODENETLEYİCİ VE KONTROL BİRİMLERİ

Sistemde teleskopun otonom olarak hareket edebilmesi için iki temel mikrodenetleyici birimi kullanılmıştır. Bu birimler, hem farklı görevlerin dağıtılmasını sağlamakta hem de sistemin çalışma sürekliliğini güvence altına alan yedekli bir yapı oluşturmaktadır.

Birinci kontrol birimi olan Raspberry Pi, görüntü işleme, sınıflandırma, veri kaydı, karar verme mekanizmaları ve yönlendirme komutlarının üretilmesinden sorumludur. Bu birim, sistemin “üst seviye kontrol merkezi” olarak çalışmakta; kamera modülünden gelen görüntüleri analiz etmekte, yapay zekâ modelini çalıştırmakta, tespit edilen gök cisminin konumunu yorumlamakta ve motorlara iletilmesi gereken yönlendirme talimatlarını hesaplamaktadır. Raspberry Pi'nin çok çekirdekli işlem yapısı, sistemin aynı anda hem görüntü işleme hem de sensör verisi değerlendirme işlemlerini gerçekleştirmesine olanak tanımakta; böylece gerçek zamanlı karar verme süreci kesintisiz sürdürülebilmektedir. Ayrıca bu birim, veri kaydının düzenli tutulması, meta

verilerin dosyalanması ve ağ bağlantısı sağlandığında bu verilerin dış ortama aktarılması gibi operasyonel süreçlerde de merkezi rol üstlenmektedir.

İkinci kontrol birimi olan Arduino Mega veya ESP32, düşük seviye görevler için sistemde görev almaktadır. Motor kontrolü, step motorların çalıştırılması, açısal hareket komutlarının uygulanması, hız kontrolü, yön değişikliklerinin senkronize edilmesi ve teleskopun fiziksel hareketlerinin doğru şekilde gerçekleştirilmesi bu birim tarafından yürütülmektedir. Aynı zamanda IMU ve GPS gibi sensörlerden gelen verilerin okunması, temel filtreleme işlemlerinin yapılması ve bu verilerin Raspberry Pi'ye güvenli şekilde iletilmesi de bu mikrodenetleyicinin sorumluluk alanındadır. Arduino Mega'nın kararlı ve deterministik çalışma yapısı ile ESP32'nin kablosuz iletişim ve yüksek hız avantajları göz önüne alındığında, bu kontrol biriminin sistem için esnek bir seçenek sunduğu görülmektedir.

Bu bileşen, Raspberry Pi'nin görev dışı kalması, kapanması veya kritik bir hata vermesi durumunda sistemi minimum işlevsellikle sürdürebilmek amacıyla aynı zamanda yedek bir kontrol modülü olarak yapılandırılmıştır. Böyle bir durumda sistem en azından teleskopun güvenli konuma alınmasını, motorların durdurulmasını veya temel bir yönlendirme işleminin gerçekleştirilmesini sağlayabilmektedir. Bu özellik, uzun gözlem oturumlarında sistem güvenliğini ve donanım bütünlüğünü korumak adına önemli bir tasarım tercihidir.

Her iki mikrodenetleyici de step motorlara gönderilecek hareket komutlarının üretilmesinden sorumludur ve ortak çalışma yapıları belirli bir görev paylaşımı üzerinden kurulmuştur. Raspberry Pi, hedef gök cisminin koordinatlarını hesaplayarak motorlara iletilmesi gereken hedef açıları belirlerken; Arduino veya ESP32 bu açılara ulaşmak için gerekli adım miktarını, hız profilini ve mikrostopping ayarlarını yönetmektedir. Böylece yönlendirme sürecinde hem yüksek seviye algoritmaların hem de düşük seviye kontrolün verimli biçimde entegre edilmesi sağlanmış; sistem hem doğru hem de tekrarlanabilir açı hareketleri üretebilen bir yapıya kavuşturulmuştur.

Bu iki mikrodenetleyici arasında veri alışverişi seri iletişim (UART), I2C veya Wi-Fi tabanlı protokoller aracılığıyla gerçek zamanlı olarak gerçekleştirilmiştir. Bu iletişim, tüm sistemin senkronize çalışması açısından kritik öneme sahiptir. Görüntü analizi sürecinde tespit edilen konum ile sensörlerden okunan gerçek yönelim arasındaki farklar sürekli olarak karşılaştırılmış; böylece teleskopun hedef gök cismini görüntünün merkezinde tutacak biçimde gerekli düzeltmeleri yapması mümkün olmuştur.

Sonuç olarak, bu iki birimin birlikte kullanılması sistemin hem esneklik hem de dayanıklılık kazanmasını sağlamış; otonom teleskop mimarisinin temelini oluşturan karar verme, konumlandırma ve hareket kontrolü süreçleri bu iki kontrol birimi tarafından sağlıklı şekilde yönetilmiştir.

3.1.4. MOTORLAR VE MOTOR SÜRÜCÜLERİ

Teleskop hareketi, iki eksenli alt-azimut yapıda çalışan step motorlar tarafından sağlanmıştır. Sistemde 17HS4401 model Nema17 step motorlar kullanılmış; motorların sürülmesi için TMC2208 model sürücü kartlarından yararlanılmıştır. Nema17 sınıfı

motorlar kompakt yapıları, yüksek tork değerleri ve düşük maliyetleri nedeniyle amatör astronomi uygulamalarında yaygın olarak kullanılmakta olup, teleskoplarda gerekli olan yavaş ve kontrollü hareketleri üretmek için uygun bir çözüm sağlamaktadır. Bu motorların teleskop gövdesinde kullanılmasıyla her iki ekseninde hassas kontrol mümkün olmuş; sistemin hedef cismi otomatik takip edebilmesi için gerekli mekanik temel oluşturulmuştur.

TMC2208 motor sürücülerini ise seçildiği anda sistemin mekanik performansını belirgin şekilde iyileştiren bir bileşen olarak öne çıkmıştır. Bu sürücüler, motorların sessiz çalışması, akıcı hareket üretmesi, titreşim azaltma özellikleri ve mikrosteyping desteği ile bilinir. Bu avantajlar astronomik gözlem gibi uzun pozlama veya düşük ışıqla gerçekleştirilen çalışmalarda kritik öneme sahiptir. Titreşimlerin minimuma indirilmesi, görüntülerin bulanıklaşmasını engelleyerek kameranın daha net kareler yakalamasını sağlamış; dolayısıyla modelin nesne tespit başarısını doğrudan olumlu yönde etkilemiştir. Motor ve sürücü bileşenlerinin birlikte kullanılması, teleskop hareketlerinde doğruluk ve stabiliteyi artırmıştır. Özellikle otomatik takip sürecinde step motorların düzgün hız geçişleri ve kontrollü dur-kalk tepkileri, hedef cismin kadrada kaymasını engellemiş ve uzun süreli gözlemlerde teleskopun sürekli yeniden hizalanması gereksinimini azaltmıştır. Alt-azimut yapının doğal sınırlamaları bulunsa da kullanılan motor-sürücü kombinasyonu sayesinde küçük düzeltmeler kesintisiz şekilde yapılabilmiş; bu durum, özellikle gezegenlerin görüntülenmesi sırasında hedefin kare merkezinde tutulmasına yardımcı olmuştur.

Motor sürücülerinin düşük gürültü ve yüksek hassasiyet sunması, sistemin gece gözlem ortamlarında dikkat dağıtıcı mekanik sesler üretmesini de engellemiştir. Bu özellik yalnızca kullanıcı deneyimini iyileştirmekle kalmamış, aynı zamanda motor titreşimlerinin teleskop borusu üzerinde oluşturabileceği küçük salınımların önüne geçmiştir. Sessiz çalışma modu, özellikle yıldız ve gezegen kaydı sırasında sensörün stabil bir görüntü yakalamasına katkıda bulunmuştur.

TMC2208'in sunduğu koruma devreleri de sistem güvenliği açısından önemli bir avantaj sağlamıştır. Akım sınırlama, ısınma kontrolü ve arıza durumunda otomatik kapanma gibi güvenlik önlemleri, motorların aşırı yük altında hasar görmesini engellemiş; uzun süreli kullanımda donanım dayanıklılığını artırmıştır. Bu koruma mekanizmaları, açık alanda yapılan gözlemlerde değişken sıcaklık ve nem koşullarına maruz kalan sistem bileşenlerinin daha güvenilir şekilde çalışmasına yardımcı olmuştur.

Ayrıca, 3D baskı parçaların hızlı prototipleme imkânı sayesinde saha testleri sırasında tespit edilen mekanik boşluk, hizalama sapması veya titreşim kaynakları kısa sürede giderilebilmiş; böylece iteratif iyileştirme döngüsü hızlandırılmıştır. Bununla birlikte, bağlantı elemanlarının parametrik tasarım yaklaşımıyla modellenmesi, farklı servo/motor modellerine ve farklı tork gereksinimlerine uyarlanabilirliği artırarak sistemin uzun vadeli bakım ve yükseltme süreçlerini de kolaylaştırmıştır.

Sonuç olarak kullanılan motorlar ve motor sürücülerini, sistemin otonom yönlendirme kabiliyetinin temel bileşenlerinden birini oluşturmuş; teleskopun doğruluk, kararlılık ve titreşimsiz hareket gereksinimlerini karşılayarak hem görüntü kalitesini hem de yapay zekâ tabanlı analiz süreçlerinin güvenilirliğini önemli ölçüde desteklemiştir.

Prototipin tahrik sistemi, ideal tasarımıdaki yüksek torklu servo + PCA9685 sürücü [21] kombinasyonu yerine, doğrudan Arduino Mega'nın donanımsal PWM çıkışlarından sürülen hobi servolarla gerçekleştirilmiştir. İki eksen için iki farklı servo türü kullanılır: azimut ekseninde, 360° kesintisiz dönebilen bir sürekli dönüş servosu; yükseklik ekseninde ise belirli bir açı aralığında konumlanan standart bir konum servosu. Bu seçim, maliyeti düşürmesinin yanı sıra, harici bir sürücü kartına olan bağımlılığı da ortadan kaldırmıştır.

Sürekli dönüş servosunun temel dezavantajı, açısal konum geri beslemesi sunmamasıdır; bu servoya verilen komut, gidilecek açıyı değil, dönüş hızını ve yönünü ifade eder. Bu sınırlama, IMU tabanlı yön ölçümüyle kurulan kapalı çevrim bir denetim sayesinde aşılmıştır: sistem, ölçülen azimut ile hedef azimut arasındaki farka göre servoyu uygun yönde döndürür ve hedefe yaklaşıldığında durdurur. Konum servosu ise doğrudan açı yazılabildiğinden, yükseklik eksenini için ek bir geri besleme gerektirmez. Bu hibrit yaklaşım, düşük maliyetli bileşenlerle kabul edilebilir bir yönlendirme doğruluğu elde etmenin pratik bir yolunu temsil etmektedir.

3.1.5. KONUM VE YÖNELİM SENSÖRLERİ

Sistemin bulunduğu coğrafi konumun ve teleskopun yönelim açılarının doğru bir şekilde belirlenmesi, otonom gözlem sürecinin temel gerekliliklerinden biridir. Bu amaç doğrultusunda sistemde Neo-6M GPS modülü ile MPU6050 veya MPU9250 ivmeölçer–jiroskop birimleri birlikte kullanılmıştır. Neo-6M modülü, GPS uydularından elde ettiği sinyaller aracılığıyla teleskopun enlem, boylam ve yükseklik bilgilerini hassas biçimde belirlemekte; böylece gök cisimlerinin o anda bulunması beklenen konumlarının hesaplanmasında gerekli olan temel coğrafi koordinatları sağlamaktadır. GPS verilerinin düzenli güncellenmesi, özellikle ilk kalibrasyon aşamasında teleskopun dünya üzerindeki doğru konumunun sisteme tanıtılması açısından kritik öneme sahiptir.

Yönelim sensörü olarak kullanılan MPU6050 veya MPU9250 bileşenleri ise teleskopun mevcut eğim, dönüş ve ivme değerlerini ölçerek gerçek zamanlı yönelim bilgisinin elde edilmesini sağlamaktadır. Bu sensörler, ivmeölçer ve jiroskop verilerini bir araya getirerek teleskopun hem yatay hem de dikey eksenindeki açısal durumunu belirler. Böylece sistem, teleskop başlangıç pozisyonunda iken gerçek bir referans noktası oluşturabilir ve hareket sırasında konum kaymalarını algılayarak gerekli düzeltmeleri gerçekleştirebilir.

Bu sensörlerden elde edilen konum ve yönelim verileri araştırma sürecinde üç temel alanda kullanılmıştır. İlk olarak, teleskopun başlangıç konumunun tanımlanmasında önemli rol oynamış; GPS ve IMU verilerinin birleştirilmesiyle sistemin otonom modda çalışabilmesi için gerekli temel referans oluşturulmuştur. İkinci olarak, gözlem yapılacak gök cisminin teorik konumu ile teleskopun mevcut yönelimi kıyaslanarak açısal fark hesaplanmış; böylece motorların ne kadar ve hangi yönde hareket etmesi gerektiği belirlenmiştir. Üçüncü olarak ise otonom takip sürecinde oluşabilecek yönelim kaymaları, titreşimler veya mekanik tolerans kaynaklı sapmalar sensör verileri ile tespit edilmiş ve sistem gerektiğinde mikro düzeltmeler yaparak hedef cismin merkezde tutulmasını sağlamıştır.

Neo-6M ve MPU serisi sensörlerin birlikte kullanılması, düşük maliyetli teleskop sistemlerinde sık karşılaşılan yönelim belirsizliklerinin büyük ölçüde giderilmesini sağlayarak otonom yönlendirme performansını artırmıştır. Bu yapı, sistemin her gözlem oturumunda doğruluğunu korumasını kolaylaştırmış, özellikle uzun süreli planet takip deneylerinde stabil bir çalışma elde edilmesine katkı sağlamıştır.

Konum ve yönelim algılaması, iki ayrı sensörle sağlanır: yönelim (azimut/eğim) için dokuz eksenli bir inersiyal ölçüm birimi (MPU9250 sınıfı IMU) ve coğrafi konum için bir NEO-6M GPS modülü. IMU; ivmeölçer, jiroskop ve manyetometre verilerini bir araya getirerek, düzeneğin gökyüzüne göre yönelimini kestirir. GPS modülü ise gözlemcinin enlem, boylam bilgisini ve hassas zaman referansını sağlar; bu veriler, efemeris hesaplarında gök cisminin yerel ufuk koordinatlarının belirlenmesi için zorunludur. GPS modülü, NMEA 0183 protokolüne uygun cümleler üretir ve bu cümleler denetleyici tarafında ayrıştırılarak konum/zaman bilgisine dönüştürülür [27], [36]. Konum kilidinin (fix) sağlanması, açık gökyüzü görüşü gerektirdiğinden, gözlem düzeneğinin kurulumunda bu husus göz önünde bulundurulur. IMU tarafında ise manyetometre okumalarının çevredeki metalik kütlelerden ve elektriksel gürültüden etkilenebileceği dikkate alınmış; bu nedenle yön kalibrasyonu ve eğim kompanzasyonu gibi düzeltici yordamlar uygulanmıştır.

3.1.6. GÜÇ BİLEŞENLERİ VE EK DONANIMLAR

Sistem, açık alanda uzun süreli gözlem yapılabilmesine olanak tanıyan taşınabilir bir güç altyapısıyla donatılmıştır. Gözlem istasyonlarının çoğu şehir ışıklarından uzak bölgelerde kurulduğundan, enerji kaynağının güvenilir ve yeterli kapasiteye sahip olması sistemin kesintisiz çalışması için temel gerekliliktir. Bu doğrultuda kullanılan 3S Li-Po batarya, hem yüksek akım sağlayabilmesi hem de ağırlık açısından taşınabilirliğe uygun olması nedeniyle tercih edilmiştir. Li-Po bataryaların yüksek enerji yoğunluğu, motorlar, sensörler ve diğer bileşenlerin yaklaşık birkaç saat boyunca aralıksız çalışmasına imkân tanımıştır.

Bataryadan gelen enerjinin sistem bileşenlerine uygun seviyede dağıtılması için XL4016 DC-DC voltaj regülatörü kullanılmıştır. Bu regülatör, teleskop sisteminde yer alan farklı modüllerin gerektirdiği 5V ve 12V seviyelerini stabil şekilde sağlayarak bileşenlerin güvenli biçimde çalışmasını mümkün kılmıştır. Motor sürücüler gibi yüksek akım gerektiren parçaların çalışması sırasında voltaj dalgalanmalarının oluşmaması, sistemdeki elektronik bileşenlerin kararlılığı için önemli bir avantaj olmuştur. Regülatörün ayarlanabilir yapısı, sistem tasarımı sırasında gerekli voltaj seviye optimizasyonlarının kolayca yapılmasına olanak tanımıştır.

Bataryanın sağlıklı bir şekilde şarj edilmesi ve batarya ömrünün korunması amacıyla iMax B6 şarj cihazı tercih edilmiştir. Bu cihaz, Li-Po bataryalar için gerekli olan hassas şarj algoritmalarını desteklemekte; aşırı şarj, aşırı deşarj ve hücre dengesizliği gibi batarya sağlığını olumsuz etkileyebilecek durumları engelleyerek bataryanın uzun ömürlü kullanımını güvence altına almaktadır. Açık alanda yapılan gözlemlerde bataryanın güvenli biçimde yönetilmesi, sistemin uzun süreli çalışması için kritik bir gereklilik olmuştur.

Sistemin mekanik bileşenlerinin montajı için 3D yazıcı ile üretilmiş özel parçalar

kullanılmıştır. Bu parçalar arasında teleskop gövdesine kamera bağlamak için kullanılan adaptörler, motor bağlantı aparatları, dişli veya kayış mekanizmalarını taşıyan yapılar ve sensörlerin montajını kolaylaştıran destek bileşenleri bulunmaktadır. 3D baskı teknolojisi sayesinde sistem, prototipleme aşamasında hızlı bir şekilde uyarlanabilir hale gelmiş; farklı boyutlardaki motorlar, kameralar veya mekanik elemanların kolayca entegre edilebilmesi mümkün olmuştur. Bu esneklik, tez süresince yapılan testler sırasında ihtiyaç duyulan değişikliklerin hızlıca uygulanmasını sağlamış ve sistemin geliştirme sürecini önemli ölçüde hızlandırmıştır.

Ek donanımlar arasında kablolar, vidalar, montaj aparatları, güç dağıtım bileşenleri ve elektronik bağlantı elemanları yer almış; tüm bu bileşenler sistemin bütünlüklü ve güvenilir şekilde çalışmasını desteklemiştir. Mekanik ve elektronik parçaların uyum içinde çalışabilmesi için tüm bağlantılar dikkatle düzenlenmiş; açık alandaki sıcaklık, nem ve toz gibi çevresel etkenlere karşı yeterli dayanıklılığa sahip bir yapı oluşturulmuştur.

Sonuç olarak güç bileşenleri ve ek donanımlar, sistemin taşınabilirliğini, sürdürülebilirliğini ve uzun süreli gözlem koşullarına dayanıklılığını sağlayan kritik unsurlar olarak görev yapmış; teleskopun sahada kesintisiz ve güvenilir biçimde çalışabilmesine önemli katkı sunmuştur.

Güç bileşenleri, sistemin hem laboratuvar (sabit güç kaynağı) hem de saha (taşınabilir batarya) koşullarında çalışabilmesini sağlayacak biçimde seçilmiştir. Denetleyici kartlar, servolar ve sensörlerin farklı gerilim seviyeleri, uygun gerilim düzenleyicilerle karşılanır. Servoların hareket anında oluşturduğu akım darbeleri, denetleyici besleme hattını etkilememesi için ayrı beslenir ve dekuplaj kapasitörleriyle yumuşatılır.

Ek donanımlar arasında; bağlantı kabloları, 3B baskı parça sabitleme elemanları (vida/somun), servo ucu adaptörleri ve elektronik muhafaza kutusu yer alır. Tüm elektronik bileşenler, saha kullanımında darbe ve toza karşı korunmaları için tabana yerleştirilen bir muhafaza içinde toplanmıştır. Bu bütünleşik yerleşim, hem kablo karmaşasını azaltmakta hem de sistemin taşınabilirliğini artırmaktadır.

3.2. YAZILIM ALTYAPISI VE YAPAY ZEKÂ BİLEŞENLERİ

Sistemin otonom olarak çalışabilmesini sağlayan temel yapı, görüntü toplama, ön işleme, nesne tespiti, sınıflandırma, engel algılama, gözlemlenebilir alan oluşturma, hedef seçimi, açı hesaplama, motor sürme ve geri bildirim döngüsünü kapsayan çok katmanlı bir yazılım mimarisine dayanmaktadır. Bu mimari tasarlanırken sistemin düşük maliyetli donanımlarla uyum içinde çalışabilmesi, sensörlerin sınırlı çözünürlük ve doğruluk kapasitesinin doğru yorumlanabilmesi ve ESP32-CAM gibi kompakt bir kameranın ürettiği nispeten düşük kaliteli görüntülerin işlenebilir hale getirilmesi amaçlanmıştır. Bu nedenle yazılım altyapısının en kritik yönü, verilerin işlenme sırasının doğru belirlenmesi ve her bileşenin birbirine doğru şekilde bağlanmasıdır.

Görüntü toplama sürecinde ESP32-CAM, geniş bir açıya sahip olacak şekilde teleskop odak düzlemine yerleştirilmiş ve elde edilen kareler Raspberry Pi'ye aktarılmıştır. Bu aktarım sonrasında görüntüler, yapay zekâ modeline gönderilmeden önce mutlaka bir ön işleme sürecine tabi tutulmuştur. Özellikle gökyüzü gözlemleri sırasında parlaklık seviyeleri sürekli değiştiğinden, ham görüntüler doğrudan kullanılmaya uygun değildir.

Bu nedenle yazılım ilk aşamada her karenin parlaklık ve kontrastını dinamik olarak düzenleyerek yüzey detayının belirginleşmesini sağlar. Arka plan gürültüsünün hafifletilmesi, sensör kaynaklı parlaklık lekelerinin azaltılması, düşük ışııkta oluşan beneklenme efeklinin filtrelenmesi ve gökyüzündeki doğal gradyan geçişlerinin korunması ön işleme sürecinin önemli parçalarıdır. Böylece görüntüler hem engel algılama modülüne hem de gök cismi tespit algoritmalarına daha istikrarlı bir biçimde aktarılır.

Sistemin önemli bir bileşeni, gözlemlenebilir alanın belirlenmesi için çalışan hibrit engel tespit mekanizmasıdır. Bu mekanizma, hem parlaklık ve kenar yoğunluğu tabanlı klasik görüntü işleme yöntemlerini hem de heuristik tabanlı çözüm mantıklarını bir arada kullanır. Teleskop çevresi her zaman gökyüzüne tamamen açık olmayabileceği için yazılım, otonom gözleme başlamadan önce belirli bir aralıkta tarama yaparak engel haritası çıkarır. Kamera belirlenen açılarda döndürölür, alınan her görüntüde kenar yoğunluğu analiz edilir ve gökyüzüne özgü düşük kenar yapısı ile engellere özgü yüksek kenar yapısı arasındaki farklar yakalanır. Bazı açılarda parlaklık düşük veya görüntüde dokusal yoğunluk yüksekse sistem o açıyı engelli bölge olarak kaydeder. Böylece teleskop sadece gökyüzü açık kısımlarda çalışacak şekilde sınırlandırılır. Kullanıcı dilerse mobil uygulama üzerinden bu tarama sürecini tamamen geçebilir; özellikle açık ovada, çatısız alanlarda veya gökyüzü tamamen açık bir konumda gözlem yapıyorsa bu modül devre dışı bırakılabilir.

Engel haritasının oluşturulmasının ardından yazılım, o anda konumlanılan noktada hangi gök cisimlerinin gerçekten gözlemlenebilir olduğunu hesaplar. GPS verisiyle elde edilen enlem-boylam bilgisi, gerçek zamanlı saat verisi ve gök cisimlerinin efemeris hesaplamaları bir araya getirilerek Ay, Mars, Jüpiter, Venüs, Satürn gibi parlak hedeflerin, ayrıca bazı parlak yıldızların ve uygun şartlarda belirli derin uzay cisimlerinin o anda gökyüzündeki konumu hesaplanır. Bu konumların ufka göre yüksekliği ve engel haritasıyla kesişimi kontrol edilir; engelli açılara denk gelen cisimler gözlemlenebilir listesine dahil edilmez. Böylece hem gereksiz hareketler önlenir hem de kullanıcıya yalnızca gerçekten gözlemlenebilir cisimler sunulur.

Nesne tespiti ve sınıflandırma süreci, hafif bir CNN tabanlı yapay zekâ modeline dayanmaktadır. Model, düşük çözünürlüklü görüntülerde çalışacak şekilde sadeleştirilmiş, gereksiz katmanlar çıkarılmış ve mobil cihazlarda yaygın kullanılan kompakt mimariler örnek alınarak optimize edilmiştir. Model özellikle parlak gök cisimlerine duyarlıdır ve Ay, Jüpiter gibi geniş parlaklık farkı oluşturan objeleri hızlı şekilde tanıyabilir. Çıktılar arasında cismin sınıfı, başarı oranı, görüntüdeki konumu, parlaklık değeri ve tespit zamanı yer alır. Bu bilgiler yönlendirme kararlarının temel girdisidir; özellikle cismin görüntü merkezinden uzaklığı hesaplanarak motorlara geri bildirim sinyali oluşturulur.

Yazılım mimarisinde tüm bu sürecin arka planında gerçek zamanlı bir kontrol döngüsü çalışmaktadır. Raspberry Pi, sürekli olarak yeni görüntü alır, modeli çalıştırır, gerekli düzeltmeleri hesaplar ve motor sürme birimine gereken komutları gönderir. Tüm modüller, kendi içinde bağımsız olmasına karşın veriler ortak bir “durum yöneticisi” (state manager) üzerinden paylaşılır. Böylece hem karar alma süreci hızlandırılır hem de donanım katmanındaki gecikmeler sistem bütünlüğünü etkilemez. Otonom gözlem

sürecinin stabil ve kesintisiz çalışabilmesi için yazılım tasarımında hata tolerans mekanizmaları da eklenmiştir; beklenmeyen sensör davranışları, anlık GPS kaymaları veya anlık görüntü bozulmaları durumunda sistem bir sonraki döngüde kendini toparlayacak şekilde tasarlanmıştır.

3.2.1. YAZILIM MİMARİSİ VE KODLAMA YOL HARİTASI: YILDIZ-GEZEĞEN TANIMA VE OTONOM GÖZLEM AKIŞI

Bu projede yazılım iki katmanlı tasarlanacaktır. AWS üzerinde çalışan ana servisler gözlem planlama, hedef listesi yönetimi, veri seti biriktirme ve model eğitimi gibi yüksek kaynak isteyen işleri üstlenirken; Raspberry Pi 5 üzerinde çalışan saha ajanı motor sürme, sensör okuma, görüntü alma ve düşük gecikmeli yerel çıkarım (inference) görevlerini yerine getirecektir. Böylece internet bağlantısının zayıf olduğu sahalarda dahi temel işlevler korunurken, bağlantı mevcut olduğunda daha zengin planlama ve daha güçlü model sürümleri otomatik devreye girecektir. AWS tarafında ham görüntülerin ve etiketlerin S3'e yazılması, mobil uygulamanın tek bir API üzerinden sistemle konuşması, S3'e güvenli yükleme/indirme için presigned URL yaklaşımının kullanılması hedeflenir.

RPi 5 tarafında ana dil Python olacaktır. Picamera2/libcamera katmanı üzerinden kamera kontrolü yapılır ve bu yapı, sahada önce Raspberry Pi Kamera Modülü V1.3 ile başlayıp daha sonra IMX477'ye geçildiğinde üst seviye akışın değişmeden korunmasına imkân verir. Servo motorlar 60 kg sınıfında olacağı için Raspberry Pi'nin GPIO'sundan doğrudan PWM üretmek yerine pratikte daha kararlı olan PCA9685 tabanlı bir PWM sürücü (I2C) ile sürmek önerilir. Bu sayede titreşim ve zamanlama dalgalanmalarının etkisi azaltılır, iki eksenle (pan-tilt) pürüzsüz komut vermek kolaylaşır. NEO-7M GPS modülünden konum almak için RPi üzerinde seri port okuması pySerial ile yapılır; NMEA cümlelerini ayrıştırmak için pynmea2 kullanılır. MPU9250 IMU verisi I2C üzerinden okunur. Saha koşullarında servo hareketleri ve ortam etkileri ölçümü bozabileceği için IMU tarafında "orientation filter" uygulanır ve kontrol katmanı daha stabil bir yönelim girdisi ile beslenir [32], [33].

Bu filtreleme, yazılımın kontrol kararlılığını artırdığı için otonom hedeflemede kritik rol oynar. RPi saha ajanının iç mantığı bir durum makinesi (state machine) yaklaşımıyla kurgulanır. Ajan sensörleri okur, hedef belirleme aşamasına geçer, hedefe yönelir, görüntüyü alır, yıldız ve gezegen tanıma modülünü çalıştırır, gerekirse mikro düzeltme uygular ve veriyi kaydeder veya aktarır. Bu yapı hem manuel mod (mobil uygulama hedef seçer) hem otomatik mod (sistem görünür hedef listesinden seçer) için aynıdır; sadece hedefin kaynağı değişir. Bağlantı kesilirse görüntü ve metaveriler yerelde kuyruklanır; bağlantı geldiğinde AWS'e toplu aktarım yapılır.

Şekil 3, Şekil 4 ve Şekil 5'de gerçek kodun "nasıl görüneceğini" göstermek için doğrudan projeye kopyalanabilir örnek iskeletlerdir.

```
# RPi tarafı: GPS (NEO-6M) okuma - pySerial + pynmea2
import serial, pynmea2
ser = serial.Serial('/dev/serial0', 9600, timeout=1) # NEO-6M 9600 baud
while True:
    line = ser.readline().decode('ascii', errors='replace')
```

```

if line.startswith('$GPGGA') or line.startswith('$GPRMC'):
    msg = pynmea2.parse(line)
    if msg.latitude and msg.longitude:
        update_location(msg.latitude, msg.longitude)

```

Şekil 3. NEO-6M GPS Örnek Okuma Kodu (Python, RPi tarafı)

```

import board
import busio
from adafruit_pca9685 import PCA9685

def setup_pwm(freq_hz=50):
    i2c = busio.I2C(board.SCL, board.SDA)
    pca = PCA9685(i2c)
    pca.frequency = freq_hz
    return pca

def set_servo_us(pca, channel: int, pulse_us: int):
    # PCA9685 12-bit; çoğu servo için 500-2500us aralığı kullanılır (kalibre edilir).
    period_us = 1_000_000 / pca.frequency
    duty = int((pulse_us / period_us) * 0xFFFF)
    pca.channels[channel].duty_cycle = max(0, min(0xFFFF, duty))

```

```

# RPi tarafı: PCA9685 ile servo sürme (azimut / altitude)
from adafruit_servokit import ServoKit
kit = ServoKit(channels=16) # PCA9685 (I2C)
def goto(az_deg, alt_deg):
    kit.servo[0].angle = clamp(az_deg, 0, 180) # azimut kanali
    kit.servo[1].angle = clamp(alt_deg, 0, 90) # altitude kanali

```

Şekil 4. Örnek Servo Sürme Kodu

Bu katmanda 60 kg servo kullanımı nedeniyle güç beslemesi ayrıca tasarlanır. Yazılım tarafında önemli olan, servo komutlarının anlık sıçramalar yerine hız limitli bir rampayla uygulanmasıdır. Bu sayede titreşim azalır ve yıldız merkezleme döngüsü daha kararlı olur.

```

from picamera2 import Picamera2

def capture_frame(width=1920, height=1080):
    cam = Picamera2()
    cfg = cam.create_still_configuration(main={"size": (width, height)})
    cam.configure(cfg)
    cam.start()
    frame = cam.capture_array() # numpy array (H, W, C)
    cam.stop()
    cam.close()
    return frame

```

```

# RPi tarafı: Picamera2 / libcamera ile kare yakalama
from picamera2 import Picamera2
import numpy as np
cam = Picamera2()

```

```
cam.configure(cam.create_still_configuration())
cam.start()
frame = cam.capture_array()          # numpy dizisi (RGB)
gray = frame.mean(axis=2).astype(np.uint8)
```

Şekil 5. Örnek Kare Yakalama Kodu

Yıldız tanıma tarafında amaç ikiye ayrılır: görüntüde yıldızları güvenilir şekilde bulmak ve görüntü kalitesi/odak gibi metrikleri çıkarmak; gerekiyorsa hedef doğrultusunu iyileştirmek için küçük düzeltmeler üretmek. Bu projede yazılımsal olarak sağlam yaklaşım, yıldızları “noktasal kaynak” gibi tespit eden hazır astronomi araçlarını kullanmaktır. Photutils bu iş için uygun bir altyapı sağlar. RPi sürümünde photutils kullanımı mümkündür; performans baskısı olursa OpenCV tabanlı basit tepe tespiti (eşikleme ve blob tespiti gibi) yedek plan olarak eklenir [31].

Bu katmanda elde edilen yıldız merkez koordinatları, takip döngüsüne geri besleme sağlar. Hedeflenen parlak yıldızın görüntü merkezinden sapmasına göre servo düzeltmesi üretilir. Böylece IMU/GPS tabanlı kaba yönlendirme üzerine görüntü tabanlı ince düzeltme eklenmiş olur.

Gezegen tanıma modülü, saha ajanında hafif çalışacak şekilde tasarlanır. Uygulamada iki aşamalı bir kurgu tercih edilir: önce görüntüden hedef bölgeyi (ROI) çıkaran ön işleme katmanı, ardından ROI’yi sınıflandıran CNN modeli. ROI çıkarımı için kaba yaklaşım genellikle yeterlidir; gezegenler parlak ve belirgin disk yapısına sahip olduğundan merkezdeki parlak bölgeyi bulup kırmak çoğu senaryoda işe yarar. Ardından CNN modeli ROI’yi “Jüpiter / Satürn / Mars / Venüs / Ay / yıldız (noktasal) / boş gürültü” gibi sınıflara ayırır. Modelin veri seti, sistemin ürettiği gerçek görüntülerle iteratif olarak güçlendirilir ve bu süreçte AWS tarafı veri seti biriktirme ve eğitim otomasyonu açısından kritik rol oynar.

RPi tarafında inference için iki seçenek sürdürülebilir şekilde yürütülebilir: TensorFlow Lite ile tflite-runtime kullanmak veya ONNX Runtime ile ONNX modeli çalıştırmak. TFLite yaklaşımı edge cihazlarda kurulum ve kaynak tüketimi açısından genellikle daha hafiftir; ONNX yaklaşımı ise eğitim ve dağıtım hattında esneklik kazandırabilir. Şekil 6’da örnek TFLite iskelet kodlarını görebilirsiniz [28], [35].

```
import numpy as np
from tflite_runtime.interpreter import Interpreter

class PlanetClassifier:
    def __init__(self, model_path: str):
        self.interp = Interpreter(model_path=model_path, num_threads=4)
        self.interp.allocate_tensors()
        self.inp = self.interp.get_input_details()[0]
        self.out = self.interp.get_output_details()[0]

    def predict(self, roi_bgr: np.ndarray) -> dict:
        # OpenCV ile resize/normalize yapılır (örn. 224x224)
        x = preprocess_to_model_input(roi_bgr, target_size=(224, 224)) # float32
        self.interp.set_tensor(self.inp["index"], x)
        self.interp.invoke()
        y = self.interp.get_tensor(self.out["index"])[0] # softmax vector
        return postprocess_probs(y)
```

```
# RPi tarafı: TensorFlow Lite ile gok cismi siniflandirma iskeleti
import numpy as np
import tflite_runtime.interpreter as tflite
interp = tflite.Interpreter(model_path='sky_model.tflite')
interp.allocate_tensors()
inp = interp.get_input_details()[0]; out = interp.get_output_details()[0]
def classify(img):
    x = preprocess_to_model_input(img)          # OpenCV ile yeniden olcekleme
    interp.set_tensor(inp['index'], x[None].astype(np.float32))
    interp.invoke()
    return interp.get_tensor(out['index'])[0] # sinif olasiliklari
```

Şekil 6. RaspberryPi’de TFLite Inference Örnek İskeleti

Bu sınıfın yanında preprocess_to_model_input() fonksiyonu OpenCV ile yazılır (yeniden boyutlandırma, kanal sırası, normalize). RPi saha ajanında tipik paketler; picamera2, opencv-python, numpy, tflite-runtime veya onnxruntime, pyserial, pynmea2, servo/sürücü için adafruit_pca9685 ve IMU için seçilen sürücü kütüphanesidir [29], [30].

AWS tarafında önerilen yaklaşım bir çekirdek API (FastAPI) ve arka planda iş kuyruğu (Celery + Redis veya benzeri) mimarisidir. FastAPI; mobil uygulama, web paneli ve RPi ajanlarının konuştuğu REST uçlarını sunar. İş kuyruğu ise görüntülerin işlenmesi, model eğitimi, veri seti dengeleme, otomatik etiket önerisi ve raporlama gibi uzun süren işleri arka planda çalıştırır. Bu ayırım API’nin hızlı cevap vermesini ve eğitim/işleme süreçlerinin ölçeklenmesini kolaylaştırır.

Model eğitiminde PyTorch tercih edilirse, eğitim sonrası dağıtım için ONNX veya TFLite artefaktı üretilecek şekilde bir export hattı kurulur. Edge tarafına indirilecek model bir model deposunda sürümlendirilir; saha ajanı açılıştan en güncel stabil modeli indirir ve yerelde cache’ler. Böylece sahadaki çıkarım katmanı, cihaz üzerinde tutulan optimize edilmiş bir model ile çalışır.

AWS API tasarımında veri akışı şu şekilde kurgulanır: RPi ajanı yakaladığı kareyi veya kısa bir görüntü dizisini önce yerelde hızlıca değerlendirir; eğer etiketlenmeye değer bir örnekse S3’e yükler ve API’ye metadata bırakır. Metadata içinde konum, IMU çıktısı, hedef adı, zaman, kamera ayarları ve yerel tahmin sonucu yer alır. Uzun vadede bu kayıtlar veri seti üretimi ve model iyileştirme döngüsünü besler.

Senaryoda sistem GPS’ten enlem-boylamı ve UTC zamanı alır, IMU ile yönelim bilgisini okur, ardından internetten gözlemlenebilir gök cisimlerini çekerek otomatik veya manuel hedef seçimini destekler. Bu katmanda pratik yaklaşım; internet API’sinin “aday hedefleri ve göksel verileri” sağlaması, nihai “gözlemlenebilir mi?” filtresinin ise sahadaki koşullara göre sistem tarafından uygulanmasıdır. Bu filtreleme minimum yükseklik eşiği, kullanıcı tanımlı gözlem penceresi, basit ufuk kısıtları ve istenirse Ay etkisi gibi kriterleri içerebilir.

Gezegenler ve temel gök cisimleri için konuma ve zamana göre veri üreten bir REST API tercih edilebilir. Bunun yanında derin uzay hedefleri için katalog tabanlı bir hedef listesi sağlayan servisler kullanılabilir. Uygulamada AWS’de hedef listesi modülü şu mantıkla yazılır: “observer” verisi (lat, lon, alt, timestamp) hazırlanır; API’den adaylar çekilir; cevap tek bir JSON şemasına normalize edilir; mobil uygulamaya ve RPi ajanına aynı şema üzerinden sunulur. RPi ajanı hedef seçim komutunu aldığı anda hedefe yönelir ve takip döngüsünü başlatır.

Bütçeye göre RPi 5 yerine ESP32/Arduino kombinasyonuna kayma ihtimali olduğundan, yazılımın motor ve sensör katmanını soyutlamak kritik olur. Bunun için saha ajanında donanım sürücülerini doğrudan iş mantığına gömmek yerine bir adaptör arayüzü yaklaşımı kullanılır. Örneğin MotorController, ImuProvider, GpsProvider, CameraProvider gibi soyut sınıflar belirlenir. RPi sürümünde bu sınıflar Python kütüphaneleriyle gerçekleştirilir; ESP/Arduino sürümünde motor kontrolü ve sensör okuma mikrodenetleyici üzerinde yapılır, üst seviye komutlar seri hat veya ağ üzerinden alınır ve telemetri geri gönderilir. Böylece aynı gözlem akışı korunur, sadece sürücü katmanı değişir.

CNN tarafında sürdürülebilir başarının anahtarı, eğitim ve etiketleme hattının sistemle birlikte çalışmasıdır. AWS tarafında her gözlem oturumu bir “run” olarak kaydedilir; her run içinde ham kareler, kırılmış ROI’ler ve sistemin o anki tahmini saklanır. Mobil uygulamada basit bir doğrulama ekranı ile hızlı insan etiketi toplanır ve bu etiketler eğitim veri setine otomatik akar. Eğitim pipeline’ı periyodik çalışır veya veri sayısı eşiği dolunca tetiklenir, yeni modeli üretir ve sahaya dağıtılacak stabil sürüm olarak yayınlar. Saha ajanı bu modeli indirerek yerel çıkarımı günceller. Böylece sistem kullanım arttıkça kendi veri setini büyütür, modelini günceller ve gezegen sınıflandırmasında sahaya özgü başarıyı yükseltir.

Yazılım altyapısı, ideal tasarımda yerel çıkarım (TFLite/ONNX) [28], [35] ve bulut tabanlı görev işleme [25], [26] üzerine kurgulanmış olsa da, gerçekleştirilen prototipte yapay zekâ bileşeni pratik nedenlerle bulut tabanlı çok-kipli bir büyük dil modeli (Google Gemini) ile sağlanmıştır. Bu tercih, sınırlı gömülü kaynaklarla yüksek seviyeli anlamsal doğrulama (yakalanan görüntünün gerçekten hedeflenen gök cismi olup olmadığının değerlendirilmesi) elde etmenin hızlı ve esnek bir yolunu sunmuştur. Modelin görüntüyle birlikte metinsel bağlamı da (hedef adı, açılar) değerlendirebilmesi, doğrulamanın güvenilirliğini artırmaktadır.

Bu yaklaşımın getirisi, geliştirme süresinin kısalması ve doğrulama kalitesinin yüksek olmasıdır; bedeli ise çevrimiçi bağlantı gereksinimi ve dış servise bağımlılıktır. İdeal tasarımdaki yerel CNN çıkarımı, çevrimdışı çalışma ve düşük gecikme avantajları nedeniyle uzun vadeli hedef olarak korunmaktadır. Web platformunda biriken kullanıcı katkılı arşiv, gelecekte bu yerel modelin eğitimi için etiketlenmiş bir veri kaynağı olarak değerlendirilebilir; böylece bulut bağımlılığı kademeli olarak azaltılabilir.

3.3. OTONOM YÖNLENDİRME YÖNTEMİ

Otonom yönlendirme yöntemi, yazılım altyapısının en karmaşık ve en kritik bölümünü oluşturmaktadır. Teleskopun belirli bir gök cismini otomatik olarak bulması, hedefe kilitlenmesi, Dünya’nın dönüşünü telafi ederek sürekli takip edebilmesi ve takip boyunca görüntüyü sürekli merkeze getirmesi için çok sayıda alt bileşen ardışık şekilde çalışmaktadır. Tüm bu süreç, kullanıcının yalnızca mobil uygulama üzerinden bir hedef seçmesi veya otomatik gözlem modunu etkinleştirmesiyle başlar; bundan sonrası tamamen yazılıma bırakılır.

Otonom yönlendirme, sistemin kalbini oluşturan ve birden çok bileşenin eşgüdümlü çalışmasını gerektiren bir süreçtir. Önce hedef cismin o anki gökyüzü konumu (azimut/yükseklik), gözlemcinin coğrafi konumu ve zamanı temel alınarak hesaplanır. Ardından bu hedef açıları mikrodenetleyiciye iletilir ve IMU geri beslemesine dayalı

kapalı çevrim denetim, mevcut yönü hedefe yaklaştırır. Bu süreç boyunca sistem, dünyanın dönüşü nedeniyle yavaşça değişen hedef konumunu periyodik olarak güncelleyerek cismi kadrada tutmaya devam eder.

Yönlendirme doğruluğu, esas olarak yön kestiriminin (heading) kalitesine bağlıdır. Bu nedenle manyetometre tabanlı yön ölçümü, jiroskopla tümleyici filtre kullanılarak kararlılaştırılır ve yerel manyetik sapma ile düzeltilir. Otonom akış literatürdeki gerçek zamanlı tespit ve izleme yaklaşımlarıyla [8], [10] kavramsal olarak uyumludur; ancak bu çalışmada amaç, sönük nesnelerin tespiti yerine bilinen parlak cisimlerin güvenilir biçimde konumlandırılması olduğundan, hesaplama açısından çok daha hafif bir yöntem yeterli olmuştur.

Sistem açıldığında Raspberry Pi, GPS modülünden enlem ve boylam bilgilerini alır ve IMU sensöründen gelen verilerle teleskopun mevcut yatay ve dikey açısını belirler. Bu iki sensör verisinin birleşimi, teleskopun başlangıç referans konumudur. Yazılım bu konumu bütün yönlendirme hesaplamalarının temel noktası olarak kullanır. Eğer kullanıcı engel haritasının kullanılmasını seçmişse teleskop kısa bir tarama sürecine girer; ESP32-CAM ile belirli açılarda görüntü alınır, her bir karede kenar ve parlaklık analizi yapılarak engeller işaretlenir. Ardından gökyüzünde gözlemlenebilir bölgeler belirlenir ve yazılım yalnızca bu bölgelerde hareket etmeye izin verir.

Bu temel hazırlık sürecinin ardından sistem, gözlemlenebilir gök cisimlerini hesaplar. Uzak cisimlerinin o andaki konumu efemeris hesaplamalarıyla bulunur; gök cisminin ufka göre yüksekliği belirlenir ve engel haritası ile kesişip kesişmediği kontrol edilir. Yalnızca engelden arındırılmış, belirli bir minimum yüksekliğin üzerinde olan ve sensör doğruluk sınırları içerisinde takip edilebilecek cisimler gözlemlenebilir listeye eklenir. Kullanıcı mobil uygulamada bu listeyi görür; listeden bir gök cismini seçebilir veya otomatik gözlem modunu aktif hale getirebilir. Otomatik gözlem modunda sistem, görüntülerde en belirgin şekilde yer alan cismi genellikle Ay veya Mars gibi yüksek parlaklık kontrastı oluşturan objeler kendisi seçer.

Hedef seçildiğinde yazılım, hedefin konumunu azimut ve yükseklik cinsinden hesaplar ve teleskopun mevcut konumuyla karşılaştırır. Bu iki değer arasındaki fark, motorların kaç derece döndürülmesi gerektiğini belirler. Raspberry Pi bu açısal farkı Arduino/ESP32 kontrol birimine gönderir. Motor kontrol birimi, step motorların hangi yönde ve ne kadar hızlı hareket edeceğini hesaplar ve TMC2208 sürücülerini aracılığıyla motorlara uygun sinyalleri gönderir. Motorlar hareket ettikçe teleskop hedefe doğru döner.

Ancak süreç bununla sınırlı değildir; yönlendirme işlemi kesintisiz bir geri bildirim döngüsü ile sürekli güncellenir. Teleskop hedefe yaklaştıkça kamera görüntüsü tekrar alınır ve yapay zekâ modeli tespit edilen cismin görüntünün merkezine ne kadar yakın olduğunu analiz eder. Eğer cisim merkezden uzaksa motorlara ufak düzeltme sinyalleri gönderilir. Dünya'nın dönüş hızına bağlı olarak gök cismi konum değiştirir; yazılım bu hareketi düzenli olarak telafi eder. Takip süreci boyunca motorlara sürekli küçük adımlarla düzeltme komutları gönderilir, böylece gök cismi görüntünün merkezinde sabit tutulur.

Bu süreç, yazılıma tanımlanan gözlem tamamlanma kriterlerine göre devam eder. Sistem genellikle en az 50 görüntü kaydedene kadar, kullanıcı manuel olarak durdurana kadar

veya gök cismi gözlemlenebilir açının dışına çıkana kadar takip işlemini sürdürür. Eğer hedef bir engelin arkasına geçerse engel haritası devreye girer ve sistem takip modunu güvenli şekilde sonlandırır. Tüm takip boyunca elde edilen görüntüler, tespit edilen cisim bilgileri, motor hareket kayıtları ve konum verileri saklanır; bu veriler daha sonra analiz için kullanılabilir.

Otonom yönlendirme yöntemi, prototipte iki katmanlı bir denetim olarak gerçekleştirilmiştir. Üst katmanda, seçilen gök cisminin güncel azimut/yükseklik hedefi (efemeris hesabından) belirlenir ve ESP32-CAM üzerinden Arduino Mega'ya iletilir. Alt katmanda ise Mega, ölçtüğü gerçek yönelim ile hedef arasındaki farkı kapatacak biçimde servoları sürer. Bu ayırım, üst seviye hedef hesabının (mobil/ağ tarafı) ile alt seviye gerçek zamanlı denetimin (mikrodenetleyici tarafı) birbirinden bağımsız olarak geliştirilip iyileştirilebilmesini sağlar.

Hedefe kilitlenme (target lock), hem azimut hem de yükseklik hatasının önceden tanımlı bir tolerans bandının içine girmesiyle bildirilir. Kilit sağlandığında sistem, görüntü yakalama adımına güvenle geçebilir. Sürekli dönüş servosunun salınımını (hunting) önlemek için azimutta bir ölü bant (tolerans) tanımlanmış; küçük hatalarda servo durdurularak gereksiz titreşimin önüne geçilmiştir. Bu strateji, düşük maliyetli tahrik bileşenleriyle kararlı bir hedef kilidi elde etmenin anahtarıdır.

3.4. VERİ KAYDI VE DEPOLAMA YÖNTEMİ

Gözlem süreci boyunca elde edilen tüm görüntüler, sensör verileri ve yapay zekâ çıktıları sistemli bir biçimde kaydedilmiş; daha sonra yapılacak analizler, kalibrasyon çalışmaları ve doğrulama süreçlerinde kullanılmak üzere düzenli bir yapı altında depolanmıştır. Otonom gözlem mekanizması sürekli çalışan bir geri bildirim döngüsüne sahip olduğundan, sistem her bir kareyi yalnızca anlık yönlendirme için kullanmakla kalmamış, aynı zamanda bu kareleri zaman damgası, konum bilgisi, yönlendirme komutları, takip esnasında yapılan düzeltmeler gibi zengin meta verilerle birlikte arşivlemiştir. Böylece hem gök cismi tespit performansının geriye dönük incelenmesi hem de sistemin ilerleyen çalışmalarda daha kararlı hale getirilmesi için gerekli veri altyapısı oluşturulmuştur. Veri kaydı sürecinin temel bileşenlerinden biri, teleskop kontrol biriminde yer alan SD kart depolama sistemi olmuştur. ESP32 veya Raspberry Pi tarafından yönetilen SD kart, sistemin internet bağlantısından tamamen bağımsız şekilde çalışabilmesini ve açık alanlarda yapılan gözlemler sırasında veri kaybı yaşanmamasını sağlamaktadır. Her oturum için otomatik olarak yeni bir klasör oluşturulmuş; bu klasör içine görüntüler ve meta veri dosyaları zaman sırasına göre kaydedilmiştir. Görüntü dosyaları, işlemci yükünü artırmayacak ancak yeterli kaliteyi sağlayacak bir formatta saklanmış; meta veriler ise JSON ve CSV benzeri hafif metin tabanlı yapılar kullanılarak düzenlenmiştir. Bu yaklaşım, gelecekte aktarım işlemini hızlandırmakta ve dış sistemlerle entegrasyonu kolaylaştırmaktadır.

Veri kaydı ve depolama, bu çalışmanın yalnızca bir gözlem aracı değil; aynı zamanda bir veri üretim platformu olma vizyonunun temelini oluşturur. Her gözlemden elde edilen görüntünün yanında; çekim anının azimut/yükseklik açıları, GPS konumu, zaman damgası ve hedef bilgisi gibi meta veriler de kaydedilir. Bu yapılandırılmış meta veri, görüntülerin ileride otomatik etiketleme ve model eğitimi için kullanılabilecek değerli bir veri setine dönüşmesini sağlar.

İdeal tasarımı bu veri akışı, bulut tabanlı ve ölçeklenebilir bir mimariyle kurgulanmıştır: çekirdek bir API katmanı [26], arka planda ağır işleri yürüten bir asenkron görev kuyruğu

[25] ve büyük ortam dosyaları için nesne depolama [22]. Bu bileşenler, çok sayıda cihazdan gelen veriyi eşzamanlı olarak alıp işleyebilecek bir altyapı öngörür. Prototipte aynı işlevsellik, daha mütevazı ancak tamamen çalışır bir PHP/MySQL yığınıyla karşılanmış; böylece veri seti vizyonu pratikte de hayata geçirilmiştir.

Meta veriler, yalnızca tespit edilen cismin sınıfı ve modelin güven oranı gibi yapay zekâ çıktılarından ibaret olmayıp; gözlem sürecinin daha sonra tam olarak yeniden canlandırılabilmesini sağlayan birçok ek parametre içermektedir. Bunlar arasında tespit zamanı, GPS konumu, teleskopun o andaki yatay ve dikey açı değerleri, görüntü parlaklık seviyesi, motorlara gönderilen yönlendirme komutları, takip sırasında yapılan mikro düzeltmelerin miktarı, görüntünün merkezine göre sapma oranı, sensör sıcaklığı ve hatta engel haritasıyla ilişkili bilgiler yer almaktadır. Bu kapsamlı kayıt yapısı, sistemin yalnızca sonuçlara değil, aynı zamanda sürecin işleyişine dair tüm ayrıntıları saklayabilmesini mümkün kılmıştır.

Görüntülerin kaydedilmesi, takip performansının değerlendirilmesi açısından özellikle önemlidir. Her kare, tespit edilen gök cisminin konumu, parlaklığı, çerçeve içindeki merkezi konum bilgisi ve görüntünün genel kalitesi ile ilişkilendirilmiştir. Bu yapı sayesinde örneğin takip sırasında cisim merkezden kaymışsa bunun hangi karede gerçekleştiği, ne kadar düzeltme gerektiği ve düzeltmenin ne kadar sürede yapıldığı doğrudan incelenebilmektedir. Ayrıca ESP32-CAM'in düşük ışık performansı sebebiyle bazı karelerin gürültü oranı yüksek olabileceği için, yazılım bu kareleri işaretleyerek ayıklanabilir hale getirmiştir. Böylece veri analiz süreçlerinde yalnızca güvenilir kareler kullanılabilir.

Depolama sürecinin ikinci aşaması, sistem internet bağlantısına erişim sağladığında devreye giren sunucuya veri aktarım mekanizmasıdır. Bu mekanizma, SD kartta biriken görüntü ve meta veri dosyalarını sırayla sunucuya yüklemekte; yüklemenin başarılı olması durumunda ilgili dosyayı yerel depolamadan silerek hem bellek kullanımını optimize etmekte hem de veri tekrarını önlemektedir. Bağlantı zayıfsa yazılım yeniden deneme stratejisi uygulamakta ve aktarım sürecini gözleme zarar vermeden arka planda yürütmektedir. Bu özellik, özellikle kırsal bölgelerde yapılan gözlemlerde sistemin dayanıklılığını artırmış ve veri kayıplarını ortadan kaldırmıştır.

Sunucu tarafında veri depolama yapısı, her gözlem oturumunu ayrı bir kimlik altında saklamaktadır. Böylece kullanıcı daha sonra oturumları karşılaştırabilir, aynı gök cismini farklı tarihlerde nasıl görüntülediğini inceleyebilir ve yapay zekâ modelinin doğruluk oranındaki değişimleri takip edebilir. Bu yapı ayrıca tezin ilerleyen aşamalarında geniş ölçekli veri seti oluşturmayı ve bu veri setinin yapay zekâ modellerinin yeniden eğitilmesi veya iyileştirilmesi amacıyla kullanılmasını mümkün kılmaktadır.

Sonuç olarak veri kaydı ve depolama sistemi, hem otonom gözlem sürecinin güvenilirliğini artırmış hem de tez kapsamında yapılacak analiz, karşılaştırma ve geliştirme çalışmalarına temel oluşturmuştur. Yerel SD kart depolaması ile sunucuya aktarımın birleşimi, sistemin hem çevrim dışı hem çevrim içi koşullarda kesintisiz çalışabilmesini sağlamış; görüntülerin ve meta verilerin bütünsel bir yaklaşımla saklanması, araştırma değerini önemli ölçüde artırmıştır.

Veri kaydı ve depolama yöntemi, gözlemlerin yalnızca anlık olarak görüntülenmesini değil, kalıcı ve sorgulanabilir biçimde saklanmasını hedefler. Her gözlem; ham/işlenmiş görüntü dosyası ve buna eşlik eden yapılandırılmış meta veriyle birlikte kaydedilir. Meta veri; hedef adı, ortam türü, azimut/yükseklik açıları, GPS koordinatları, çekim zamanı ve yapay zekâ doğrulama sonucu gibi alanları içerir. Bu yapı, gözlemlerin zaman içinde tutarlı bir veri setine dönüşmesini sağlar.

Depolama mimarisi, çevrimdışı–çevrimiçi senaryolar arasında dayanıklılık gözetilerek tasarlanmıştır. Sahada ağ erişimi bulunmadığında veriler yerelde güvenli biriktirilir; bağlantı sağlandığında ise web platformuna senkronize edilir. Bu yaklaşım, geçici ağ kesintilerinin veri kaybına yol açmasını önler. İdeal tasarımda bu rol, nesne depolama (S3) ve önceden imzalı bağlantılar (presigned URL) gibi bulut servisleriyle [22] üstlenilirken; prototipte aynı işlev, kendi sunucumuzdaki PHP/MySQL tabanlı platform ve onun cihaz yükleme API'siyle karşılanmıştır.

3.4.1. KAYDEDİLEN VERİLERİN VERİ SETİNE TAŞINMASI

Bu tez kapsamında geliştirilen otonom teleskop sistemi, gözlem sırasında üretilen görüntüleri yalnızca “anlık yönlendirme” için kullanmakla kalmayıp; her kareyi zengin meta verilerle birlikte arşivleyerek ileride büyütülebilecek bir “Gökyüzü Veri Seti”nin temelini oluşturmayı hedeflemektedir. Gözlemlerden elde edilen görüntülerin; tarih, konum, nesne tipi, parlaklık ve model güven oranı gibi bilgilerin yanında, takip sürecine ilişkin motor komutları ve düzeltmeler gibi teknik ayrıntılarla birlikte saklanması, veri setinin bilimsel ve mühendislik açısından tekrar üretilebilirliğini artırır. Tezde de belirtildiği üzere bu veriler, internet bağlantısı bulunduğunda bulut tabanlı bir veri havuzuna aktarılacak ve süreç sonunda sürekli büyüyen açık bir veri seti oluşturulacaktır.

Bu alt bölümde; (i) sahada kaydedilen verilerin paketlenmesi, (ii) güvenli ve dayanıklı aktarım mekanizması, (iii) sunucu alanı/domain ve servis mimarisi, (iv) veri doğrulama–kalite kontrol–etiketleme akışı ve (v) veri seti sürümleme ve dışa aktarma adımları ayrıntılı biçimde açıklanmaktadır.

3.4.1.1. OTURUM BAZLI VERİ PAKETLEME VE STANDARTLAŞTIRMA

Gözlem sürecinde veri kaydı, cihazın internet bağlantısından bağımsız çalışabilmesi için yerel depolama üzerinde oturum (session) bazlı tutulur. Tezde önerilen yaklaşımda her gözlem oturumu için otomatik bir klasör açılması; görüntü ve meta veri dosyalarının zaman sıralı biçimde kaydedilmesi; meta verilerin ise JSON/CSV gibi hafif formatlarda saklanması, daha sonra yapılacak aktarım ve entegrasyonu kolaylaştırmaktadır.

Bu tasarım, veri kaybını azaltırken aynı zamanda “veri setine dönüşebilirlik” için gerekli disiplinli dosya yapısını baştan sağlar. Örnek klasör yapısı aşağıdaki gibi tasarlanabilir:

- session.json (oturum özeti: cihaz, lokasyon, tarih, kalibrasyon bilgileri)
- frames/ (ham görüntüler)
- meta/ (kare bazlı meta veriler)
- manifest.json (hash/CRC, dosya boyutu, yükleme durumu)
- preview/ (isteğe bağlı küçük boyutlu önizlemeler)

Kare bazlı meta veriler, tezde de vurgulandığı gibi yalnızca “tahmin edilen sınıf + güven” bilgisinden ibaret değildir; gözlemin sonradan yeniden canlandırılmasını sağlayan birçok

parametreyi içerir (tespit zamanı, GPS konumu, teleskop açıları, parlaklık, sensör sıcaklığı vb.). Bu kapsam, veri setini yalnızca bir “etiketli görüntü koleksiyonu” olmaktan çıkarıp; aynı zamanda kontrol algoritmalarının değerlendirilmesi ve sistem iyileştirme döngüsü için bir mühendislik kaydı haline getirir.

```
{
  "frame_id": "000123",
  "timestamp_utc": "2025-11-22T18:31:05Z",
  "gps": { "lat": 40.7661, "lon": 29.9408, "alt_m": 85 },
  "mount": { "az_deg": 124.22, "alt_deg": 35.10 },
  "target": { "pred_class": "Jupiter", "confidence": 0.92 },
  "image_stats": { "brightness": 0.64, "blur_score": 0.11, "noise_score": 0.27 },
  "tracking": {
    "center_offset_px": { "x": -14, "y": 6 },
    "motor_cmd": { "az_steps": 3, "alt_steps": -1 },
    "micro_corrections": 2
  }
}
```

Şekil 7. Basitleştirilmiş örnek meta verisi

Bu standart, ileride farklı cihazlardan gelen kayıtların tek bir veri setinde birleştirilmesini kolaylaştırır: farklı kamera modülleri, farklı teleskoplar, farklı motor oranları vb. değişse dahi veri sözleşmesi (schema contract) sabit tutulur.

3.4.1.2. ÇEVİRİMDIŞI-ÇEVİRİMİÇİ SENARYOLARDA DAYANIKLI VERİ AKTARIMI

Saha gözlemlerinde internet bağlantısının sürekliliği garanti edilemediğinden, veri aktarım mekanizması çevrimdışı koşullarda veri biriktiren ve bağlantı sağlandığında otomatik olarak devreye girerek yarım kalan işlemleri tamamlayabilen bir yapıda tasarlanmalıdır. Bu kapsamda sistem, gözlem sırasında SD kart üzerinde oturum bazlı birikmiş görüntü ve meta verileri bağlantı elde edildiğinde sırayla sunucuya yükler; yükleme başarılı biçimde doğrulandığında yerel kopyayı silerek hem depolama alanını verimli kullanır hem de aynı verinin tekrar tekrar taşınmasını önler.

Aktarım sürecinin pratikte güvenilir çalışması için öncelikle oturum ve dosya bazında durum takibi yapılması gerekir. Bu amaçla her oturum için bir manifest dosyası oluşturulur ve oturum içindeki her dosya için bütünlük doğrulamasına temel olacak bir özet değer, dosya boyutu ve aktarım durumu tutulur. Aktarım durumu, dosyanın henüz kuyruğa alınmadığı, yüklenmekte olduğu, başarıyla tamamlandığı veya hata aldığı gibi aşamaları ifade eder. Bu yaklaşım, özellikle cihaz yeniden başlatıldığında ya da bağlantı kopup geldiğinde, aktarım sürecinin nerede kaldığını doğru biçimde tespit ederek kaldığı yerden devam edebilmesini sağlar.

Büyük boyutlu görüntü dosyalarında bağlantı kopmalarının doğrudan başarısızlığa dönüşmemesi için parçalı yükleme yaklaşımı benimsenir. Bu yöntemde dosya tek seferde gönderilmek yerine daha küçük parçalara ayrılarak iletilir; bağlantı kesildiğinde tüm dosyayı baştan göndermek yerine yalnızca eksik kalan parçalar yeniden yüklenir. Yeniden deneme mantığı, kısa süreli ağ dalgalanmalarında veri kaybı yaşanmadan aktarımın tamamlanmasını sağlar ve saha koşullarında sistemin dayanıklılığını artırır.

Sunucu tarafında depolama katmanı olarak obje depolama kullanılacağı senaryolarda, aktarım sürecinin performans ve güvenlik açısından ölçeklenebilir olması için imzalı yükleme yaklaşımı uygulanabilir. Bu kurguda cihaz, doğrudan API üzerinden büyük dosyayı taşımak yerine, API’den bir yükleme yetkisi alır ve görüntüyü obje depolamaya yükler. API tarafına ise yalnızca meta veri ve yüklenen dosyayı işaret eden referans bilgisi iletilir. Böylece API katmanı gereksiz veri trafiği altında kalmaz; dosyalar depolama katmanında daha verimli şekilde yönetilir.

Aktarım tamamlandıktan sonra verinin kabul edilmesi için bütünlük doğrulaması yapılması kritik bir adımdır. Sunucu, manifestte kayıtlı özet değer ile kendisine ulaşan dosyanın özet değerini karşılaştırmadan dosyayı “tamamlandı” kabul etmez. Bu kontrol, özellikle bağlantı kopmaları sırasında oluşabilecek bozuk veya eksik yazılmış dosyaların veri setine karışmasını engeller. Yerel depolamadan silme adımı da doğrudan yükleme isteğinin dönmesiyle değil, sunucunun dosyayı eksiksiz aldığını doğrulayan yanıtıyla tetiklenir. Yüklemenin “tamamlandı” durumuna alınması ve sunucudan onay alınması, iki aşamalı bir güvence sağlar. Böylece sistem uzun süre çevrimdışı çalışsa bile verileri kaybetmez; bağlantı sağlandığında arka planda güvenli biçimde taşıyarak veri seti havuzuna tutarlı bir şekilde aktarır.

3.4.1.3. SUNUCU ALTYAPISI, DOMAIN PLANI VE DAĞITIM YAKLAŞIMI

Açık veri seti hedefi doğrultusunda sistemin sunucu tarafı, hem veri kabulünü hem de veri setinin görüntülenmesini ve yönetimini destekleyecek şekilde yapılandırılmalıdır. Proje kimliğini yansıtacak tek bir alan adı seçilerek servislerin alt alan adlarıyla ayrıştırılması, hem yönetimi kolaylaştırır hem de ileride ölçekleme ve güvenlik kurgularında düzen sağlar. Örnek bir kurgu olarak gokveri.org alan adı kullanılabilir. Bu durumda api.gokveri.org veri kabul ve kimlik doğrulama uçlarını, app.gokveri.org gözlem arşivi ve veri seti tarama amaçlı web arayüzünü, panel.gokveri.org yönetim ve etiketleme panelini, docs.gokveri.org dokümantasyon ve veri sözleşmesini, cdn.gokveri.org ise önizleme görselleri ve küçük boyutlu çıktıları sunacak dağıtım katmanını barındıracak şekilde planlanabilir.

Başlangıç aşamasında maliyet-etkin ve yönetilebilir bir kurulum hedeflenir. Bu kapsamda Linux tabanlı tek bir sanal sunucu üzerinde temel servislerin çalıştırılması yeterli olabilir. İşletim sistemi olarak güncel bir Ubuntu sürümü tercih edilebilir; işlemci ve bellek kapasitesi başlangıç için orta seviyede planlanır; depolama tarafında ise hem uygulama verisini hem de gerekli indeksleri taşıyabilecek SSD alanı ayrılır. Görüntü dosyalarının uzun vadeli saklanması için ayrı bir obje depolama katmanı kullanılır; bu katman yönetilen bir S3 servisi olabileceği gibi, ihtiyaç halinde aynı sunucu üzerinde konumlandırılmış bir MinIO kurulumu da olabilir. Uygulama verileri ve meta veriler için ilişkisel veritabanı gerekeceğinden PostgreSQL kullanılabilir; bu veritabanı sunucu üzerinde çalıştırılabileceği gibi yönetilen bir veritabanı servisine taşınarak bakım yükü azaltılabilir. Sık erişilen verilerin hızlandırılması ve kuyruk tabanlı işlerin taşınması için Redis kullanımı planlanır. Dış dünyaya açılan tüm servisler için Nginx ters vekil olarak konumlandırılır ve TLS sertifikaları Let’s Encrypt ile yönetilerek iletişim güvenli hale getirilir.

Veri hacmi ve kullanıcı sayısı arttıkça dağıtım yaklaşımı kademeli olarak genişletilebilir.

API katmanı konteyner olarak paketlenip yatay çoğaltılabilir; obje depolama yönetilen S3 tarafına taşınarak depolama ölçeklenmesi ayrıştırılabilir ya da MinIO küme yapısına geçilebilir. Önizleme üretimi, kalite analizi gibi işlemci yoğun görevler için worker sayısı artırılarak kuyruk üzerinden paralel işleme yapılabilir. İhtiyaç oluştuğunda konteyner yönetimi için daha gelişmiş orkestrasyon sistemleri devreye alınabilir. Bu yaklaşım, başlangıçta gereksiz karmaşıklığa girmeden, büyümeye açık ve kontrollü bir altyapı sağlar.

3.4.1.4. SUNUCU TARAFI SERVİSLER VE TEKNOLOJİ BİLEŞENLERİ

Veri seti oluşturma sürecinde sunucu tarafı, yalnızca dosyaların toplandığı bir alan değil; verinin kabulünden doğrulanmasına, işlenmesinden sunulmasına kadar tüm yaşam döngüsünü yöneten bir platform olarak ele alınır. Bu nedenle servislerin görevlerine göre ayrıştırılması, bakım kolaylığı ve ölçeklenebilirlik açısından avantaj sağlar.

Veri kabul servisi, cihaz doğrulama, oturum açma, meta veri kabulü ve yükleme yetkilendirmesi üretimi gibi çekirdek görevleri üstlenir. Bu servis Node.js tabanlı bir çatı üzerinde veya Python tabanlı bir servis olarak kurgulanabilir. Cihazların yetkilendirilmesi, cihaz anahtarı veya sertifika mantığıyla sağlanır; böylece yalnızca kayıtlı cihazların veri yükleyebilmesi güvence altına alınır. Veri kabul servisi, büyük dosya trafiğinin doğrudan kendisi üzerinden taşınmasını gerektirmeyecek şekilde tasarlandığında, uygulama daha kararlı ve sürdürülebilir hale gelir.

İşleme katmanı, sunucuya ulaşan verilerin veri setine uygun hale getirilmesi için gereken operasyonları yürütür. Bu kapsamda küçük boyutlu önizleme üretimi, format dönüştürme, kalite metriği hesaplama, tekrarlı örneklerin ayıklanması ve otomatik ön-etiket kontrol akışları bu katmanda ele alınır. İşler kuyruk üzerinden yönetilerek yoğun anlarda sistemin tıkanması önlenir. Worker tarafı Python veya Node.js ile kurulabilir; kuyruk için Redis veya RabbitMQ kullanılabilir. Bu tasarım, veri kabulü ile veri işleme yükünü birbirinden ayırarak sistemin daha stabil çalışmasını sağlar.

Kullanıcıların veri setini inceleyebilmesi için web uygulaması katmanı gerekir. Bu katman gözlem oturumlarının listelenmesi, filtrelenebilir, veri seti üzerinde arama yapılması ve indirme veya dışa aktarma isteklerinin yönetilmesi gibi işlevleri sağlar. Web arayüzü Node.js tabanlı bir yapı üzerinde geliştirilebilir. Yönetim ve etiketleme paneli ise kullanıcı yönetimi, cihaz kayıtları, veri onay akışı, etiket düzeltme ve kalite problemlerinin işaretlenmesi gibi operasyonel ihtiyaçları karşılar. Bu panelin PHP CodeIgniter tabanlı olarak geliştirilmesi, hızlı yönetim ekranları ve rol tabanlı kontrol ihtiyaçlarında pratik bir yol sunar. Bu hibrit yaklaşım, veri kabul ve modern arayüz ihtiyaçlarında Node.js ekosisteminin esnekliğini kullanırken, yönetim tarafında mevcut PHP altyapısının sağladığı hız ve alışkanlıkları koruyabilir.

Projeye ait tanıtım, lisans, yayın ve kullanım koşullarının sunulacağı bir içerik katmanı da gerekebilir. Bu katman PHP tabanlı hafif bir içerik yönetimiyle ya da statik bir sayfa yapısıyla sağlanabilir. Amaç, veri setinin nasıl kullanılacağına dair kuralları ve proje bilgisini merkezi biçimde sunmaktır.

3.4.1.5. VERİ DOĞRULAMA, KALİTE KONTROL VE ZENGİNLEŞTİRME

ADIMLARI

Veri setinin gerçek değeri, sadece çok sayıda örnek içermesiyle değil; örneklerin doğru, temiz ve izlenebilir olmasıyla ortaya çıkar. Bu nedenle sunucuya ulaşan veriler, veri seti havuzuna dahil edilmeden önce doğrulama ve kalite kontrol sürecinden geçirilir. İlk aşamada bütünlük doğrulaması yapılır. Yüklenen dosyaların manifestteki özet değerlerle eşleşmesi kontrol edilerek bozuk, eksik veya aktarım sırasında zarar görmüş dosyaların sistemde kalıcı hale gelmesi engellenir. Bu kontrol, aktarım mekanizmasının güvenilirliğini veri seti perspektifinde tamamlayan temel adımdır.

Meta verinin tutarlılığı da aynı derecede kritiktir. Meta veriler belirlenen JSON şemasına uymadığında kayıt doğrudan veri seti havuzuna alınmaz; kontrol bekleyen bir alana ayrılır. Zaman ve konum bilgileri açısından tutarlılık kontrolleri yapılır; GPS bilgisinin eksik olması durumunda kayıt işaretlenir, olağandışı koordinatlar tespit edildiğinde gözden geçirme sürecine sokulur. Bu sayede veri setine giren örneklerin bağlam bilgisi korunur ve analiz aşamalarında yanlış yorumlama riski düşer.

Görüntü kalitesi, veri setini doğrudan etkileyen bir diğer başlıktır. Bulanıklık, aşırı karanlık ya da aşırı parlak kareler ve yüksek gürültü düzeyi gibi göstergeler, kayıtların uygunluk durumunu belirlemede kullanılır. Özellikle düşük ışık koşullarında gürültü seviyesi artabileceğinden, bu tür karelerin işaretlenebilir olması ve gerektiğinde veri setinden ayrı tutulabilmesi önemlidir. Ayrıca tekrarlı kayıtların veri setinde dengesizlik oluşturmaması için tekrarlılık kontrolü yapılır. Aynı karelerin yeniden yüklenmesi veya aynı oturumun tekrar taşınması gibi durumlarda içerik özetleri üzerinden ayıklama uygulanır. Son olarak otomatik ön-etiketleme mantığında model çıktısı öneri olarak saklanır; insan doğrulamasıyla kesin etiket haline getirilir. Böylece veri seti, yalnızca model çıktısının otomatik bir yığını değil, kontrollü biçimde iyileşen bir kaynak haline gelir.

3.4.1.6. ETİKETLEME VE VERİ SETİ SÜRÜMLEME YAKLAŞIMI

Sistem ilk aşamada sınıflandırma tabanlı bir veri seti üretecek şekilde kurgulansa da, ilerleyen aşamalarda veri setinin kapsamı genişletilebilir. Sınıflandırma veri setinde her kareye bir etiket atanırken; tespit odaklı veri setinde kare üzerinde nesne konumunu ifade eden kutu bilgisi ile etiket birlikte tutulur. Daha ileri aşamada segmentasyon yaklaşımıyla piksel düzeyinde maske üretimi hedeflenebilir. Bu genişleme, veri setinin farklı yapay zekâ görevlerinde kullanılabilmesini sağlar; ancak her aşamada etiket formatının, saklama biçiminin ve doğrulama sürecinin net tanımlanması gerekir.

Etiketleme sürecinde cihazdan gelen model tahmini doğrudan kesin etiket kabul edilmez; panel üzerinde öneri olarak sunulur. Operatör veya etiketleyici, örnek kareleri doğrulayarak veya düzelterek etiket kalitesini yükseltir. Bulanık, hedefin yanlış olduğu, engel veya hareket nedeniyle uygun olmayan kareler veri seti dışında tutulacak şekilde işaretlenir. Onaylanan oturumlar veri setine hazır statüsüne alınır ve veri seti üretiminde tercih edilen kaynak haline gelir. Bu akış, veri setinin zaman içinde daha temiz ve güvenilir hale gelmesini sağlar.

Veri seti büyüdükçe sürümleme zorunlu hale gelir. Tek bir klasörde sürekli büyüyen bir

yapı yerine, sürüm mantığıyla paketlenmiş çıktıların üretilmesi hem izlenebilirliği hem de tekrarlanabilirliği artırır. Sürümler, artan numaralandırmayla ifade edilir ve her sürüm için sınıf dağılımı, örnek sayısı, kalite metriklerine dair özet ve eğitim, doğrulama, test bölünmeleri saklanır. Ayrıca hangi ham oturumlardan hangi sürümün üretildiğinin izlenebilmesi gerekir; bu izlenebilirlik manifest bilgileri ve veritabanı kayıtları üzerinden sağlanır. Böylece model eğitimi sonuçları bir sürüme bağlanabilir, ileride aynı sürüm yeniden üretilebilir ve değişikliklerin etkisi karşılaştırılabilir.

3.5. DENEYSEL GÖZLEM YÖNTEMİ

Araştırma kapsamında tasarlanan sistemin performansını, doğruluğunu ve gerçek koşullarda çalışabilirliğini değerlendirebilmek amacıyla tüm testler doğrudan açık gökyüzü altında, değişken atmosferik koşullarda ve farklı zaman dilimlerinde gerçekleştirilen deneysel gözlem oturumlarıyla yürütülmüştür. Bu gözlemler yalnızca sistemin teknik yeterliliğini doğrulamakla kalmamış; aynı zamanda yazılım bileşenlerinin, otonom yönlendirme algoritmalarının, engel tespit yöntemi ile gökyüzü görünürlük haritasının ve veri kaydı mekanizmasının gerçek kullanım senaryolarında verdiği tepkilerin anlaşılmasını sağlamıştır.

Her bir gözlem oturumu, belirli bir hazırlık aşamasıyla başlamıştır. Bu hazırlık kapsamında önce sistemin fiziksel kurulumu yapılmış, teleskop dengesi kontrol edilmiş, güç bileşenlerinin bağlantıları doğrulanmış ve GPS ile IMU sensörlerinin kalibrasyon süreçleri tamamlanmıştır. Sistem bu aşamada çevredeki engelleri taramak amacıyla kısa bir görüntü tarama döngüsüne girmiş; ESP32-CAM ile farklı açılardan alınan görüntüler analiz edilerek gözlemlenebilir gökyüzü aralığı belirlenmiştir. Eğer kullanıcı bulutların az olduğu, yüksek görüş açıklığına sahip bir bölgede gözlem yapıyorsa bu tarama aşaması mobil uygulama üzerinden atlanmış ve sistem doğrudan hedefleme sürecine geçmiştir. Bu esneklik, sistemin hem kentsel alanlarda hem de kırsal bölgelerde farklı koşullarda kullanılmasına olanak tanımıştır.

Bu aşamada kullanılan deneysel test düzeneği, sistemin gerçek saha koşullarındaki davranışını bütüncül olarak değerlendirebilmek amacıyla oluşturulmuştur. Şekil 7’de görüldüğü gibi, deney ortamında merkezi kontrol birimi olarak Arduino Mega kullanılmış; motor kontrolü, sensör okuma ve düşük seviye zamanlama işlemleri bu birim üzerinden yürütülmüştür. Görüntü tabanlı engel tespiti ve gökyüzü tarama işlemleri için ESP32-CAM, yüksek seviye veri işleme, yapay zekâ tabanlı analizler ve mobil uygulama ile haberleşme için ise Raspberry Pi Zero sisteme entegre edilmiştir. Konum ve zaman bilgisinin hassas şekilde elde edilebilmesi amacıyla NEO-7M GPS modülü, harici bir GPS anteni ile birlikte kullanılmış; yönelim ve açısal hareketlerin ölçümü için MPU9250 IMU sensörü sisteme dâhil edilmiştir. Teleskopun yatay ve düşey eksenlerde yüksek tork gerektiren hareketlerini sağlayabilmek amacıyla her biri 60 kg·cm tork kapasitesine sahip üç adet servo motor kullanılmıştır. Bu donanım bileşenlerinin birlikte çalıştığı test düzeneği, sistemin otonom yönlendirme doğruluğunun, mekanik kararlılığının ve yazılım donanım entegrasyonunun açık gökyüzü altında gerçekçi koşullarda sınanmasına olanak sağlamıştır.



ilişkilendirilmiş olarak SD karta kaydedilmiş; bu meta veriler arasında zaman damgası, GPS koordinatı, hedef cismin türü, parlaklık değeri, yapay zekâ modelinin tespit güven yüzdesi, o anki teleskop açısı, motorlara gönderilen adım miktarı ve takip sırasında yapılan düzeltme oranları yer almıştır. Böylece her bir görüntü yalnızca görsel bir veri değil, aynı zamanda yönlendirme algoritmasının nasıl davrandığını gösteren bir ölçüm niteliği taşımıştır.

Bu deneysel gözlemler sonucunda elde edilen veriler, tez kapsamında oluşturulacak açık gökyüzü veri setinin temelini oluşturmuştur. Veri setinin amacı, hem gelecekte yapılacak testlerde sistemin iyileştirilmesine olanak tanımak hem de benzer çalışmalarda araştırmacıların kullanabileceği bir kaynak sağlamaktır. Farklı atmosfer koşullarında, farklı konumlarda ve farklı parlaklık seviyelerine sahip gök cisimleriyle gerçekleştirilen bu oturumlar, sistemin gerçek kullanım senaryolarındaki davranışını ayrıntılı biçimde ortaya koymuş; yönlendirme doğruluğunun, takip süresinin, görüntü kalitesinin ve yapay zekâ tespit performansının bütünsel olarak değerlendirilmesine imkân tanımıştır.

Deneysel gözlem yöntemi, sistemin uçtan uca işlevselliğini gerçek koşullarda sınamak üzere tasarlanmıştır. Her gözlem oturumu; düzeneğin kurulumu ve tesviyesi, GPS konum kilidinin beklenmesi, yön (azimut) kalibrasyonu, hedef seçimi, otonom yönlendirme, odak/kolimasyon kontrolü, görüntü yakalama ve son olarak doğrulama ile arşivleme adımlarını içerir. Bu standartlaştırılmış akış, farklı oturumlar arasında karşılaştırılabilir sonuçlar elde edilmesini sağlamıştır.

Gözlemler ağırlıklı olarak Ay üzerinde yoğunlaştırılmıştır; bunun nedeni, Ay'ın yüksek parlaklığı ve büyük açısal boyutu sayesinde, düşük maliyetli görüntüleyiciyle dahi zengin yüzey detayı sunması ve yönlendirme doğruluğunun görsel olarak kolayca değerlendirilebilmesidir. Oturumlarda, farklı Ay evrelerinde (dolunaya yakın ve terminator bölgesinde) görüntüler alınarak, aydınlatma koşullarının detay görünürlüğü üzerindeki etkisi de gözlenmiştir. Elde edilen görüntüler, doğrudan web tabanlı arşive yüklenerek sistemin veri üretme döngüsü baştan sona test edilmiştir.

3.6. MOBİL UYGULAMA VE UZAKTAN KONTROL ALTYAPISI

Sistemin yalnızca yerel bir prototipten ibaret kalmaması, farklı lokasyonlarda konuşlandırılabilmesi ve gerçek kullanıcılar tarafından esnek bir şekilde yönetilebilmesi için, teleskop sistemiyle entegre çalışan bir Android mobil uygulaması ve bu uygulama ile teleskop sistemi arasında köprü görevi gören sunucu tabanlı bir altyapı tasarlanmıştır. Bu yapı, hem internet bağlantısının mevcut olduğu senaryolarda hem de yalnızca yerel kablosuz ağ üzerinden erişim sağlanabilen durumlarda çalışabilecek şekilde iki modlu bir mimariyle kurgulanmıştır. Böylelikle sistem, sahada gerçek gözlem senaryolarında karşılaşılabilecek bağlantı kısıtlarından minimum düzeyde etkilenmektedir.

Mobil uygulama ve uzaktan kontrol altyapısı, sistemin kullanıcıyla bulunduğu katmandır. Bu katmanın tasarımındaki temel ilke, karmaşık otonom işlevleri (efemeris hesabı, kapalı çevrim yönlendirme, görüntü doğrulama) kullanıcıdan soyutlayarak, gözlemi birkaç dokunuşla yapılabilir hâle getirmektir. Kullanıcı yalnızca bir hedef seçer; konum hesabı, yönlendirme ve doğrulama gibi adımlar arka planda otomatik olarak yürütülür.

Uzaktan kontrol, yerel ağ üzerinden düşük gecikmeli bir HTTP arayüzüyle sağlanır. Bu yaklaşım, robotik gözlemevi denetim sistemlerinin [9] temel mantığını (merkezi bir denetleyiciye komut gönderme ve durum sorgulama) küçük ölçekte ve erişilebilir biçimde yeniden üretir. Cihaz keşfinin mDNS ile otomatikleştirilmesi ve bağlantının kopması

durumunda kendiliğinden onarılması, saha kullanımında kullanıcı yükünü belirgin biçimde azaltan tasarım tercihleridir.

Mobil uygulama, öncelikle kullanıcının teleskop sistemini uzaktan kontrol edebilmesini ve gözlem sürecine ilişkin temel tüm parametreleri anlık olarak takip edebilmesini sağlayan bir arayüz sunmaktadır. Uygulama üzerinden teleskopun sağa–sola, yukarı–aşağı hareket ettirilmesi, otomatik hedefleme ve otonom takip modlarının başlatılıp durdurulması, belirli bir gök cisminin (örneğin Ay, Mars veya Jüpiter) doğrudan seçilmesi, engel tarama fonksiyonunun tetiklenmesi veya atlanması gibi işlemler yapılabilmektedir. Bunun yanı sıra uygulamada teleskopun güncel yönelim bilgisi, GPS konumu, o anki gözlem modunun durumu, sistemin açık/kapalı hali, güç durumu ve bağlantı durumu gibi telemetri bilgilerinin görüntülenebildiği durum ekranları bulunmaktadır. Böylece kullanıcı, hem sahada teleskopun yanında iken hem de uzaktan erişim sağladığında sistemin çalışma durumunu ayrıntılı biçimde görebilmektedir.

Mobil uygulama ile teleskop sistemi arasındaki iletişim iki farklı çalışma moduna göre düzenlenmiştir. Birinci mod, teleskop sisteminin (ESP32 veya Raspberry Pi) internete bağlı olduğu durumlarda devreye giren sunucu tabanlı kontrol modudur. Bu modda teleskop sistemi, belirli aralıklarla HTTP GET istekleri göndererek bağlı olduğu AWS tabanlı sunucuya “mevcut komut var mı?” şeklinde sorgular iletmektedir. Sunucu, ilgili teleskopa ait bekleyen bir komut varsa bunu JSON formatında cevap olarak döner; herhangi bir komut yoksa boş veya basit bir yanıt verilerek gereksiz işlem yükü oluşturulmamaktadır. Bu yapı sayesinde sunucu, teleskopa doğrudan istek atmak zorunda kalmamakta; NAT, port yönlendirme ve güvenlik duvarı gibi karmaşık ağ yapılandırmalarına gerek kalmadan yalnızca teleskopun sunucuyu “polling” yöntemiyle periyodik olarak yoklaması yeterli olmaktadır. Komutların kaynağı; mobil uygulama veya web tabanlı yönetim paneli olabilir; ancak her durumda komutlar önce sunucuda kayıt altına alınmakta, daha sonra teleskop sistemi tarafından çekilmektedir.

İkinci mod ise teleskop sisteminin internete erişiminin olmadığı, ancak yerel Wi-Fi üzerinden erişilebilir olduğu durumlar için tasarlanmıştır. Bu senaryoda mobil uygulama, aynı yerel ağa bağlı olduğu ESP32 veya Raspberry Pi’ye doğrudan REST API çağrıları ile bağlanmaktadır. Kullanıcı, uygulama üzerinden “yerel bağlantı” modunu seçtiğinde sistem sunucuya istek göndermemekte; tüm komutlar doğrudan yerel IP adresi üzerinden teleskop sistemine iletilmektedir. Bu yaklaşım özellikle kırsal bölgelerde, dağlık alanlarda veya mobil veri erişiminin sınırlı olduğu durumlarda sistemin kullanılabilmesini sağlamaktadır. Ayrıca bu modda canlı video akışı da doğrudan teleskop sisteminden alınmakta, böylece gecikme süresi azaltılmakta ve sunucudan bağımsız, daha hızlı bir geri bildirim döngüsü elde edilmektedir.

Canlı video akışı, hem mobil uygulamanın hem de sunucu tarafındaki web arayüzünün önemli bileşenlerinden biridir. ESP32-CAM veya Raspberry Pi üzerinde çalışan görüntü işleme bileşeni, MJPEG formatında bir video akışı üretmekte; bu akış, internet bağlantısı mevcut ise sunucu üzerinden, bağlantı yoksa doğrudan yerel ağ üzerinden mobil uygulamaya iletilmektedir. MJPEG akışı, her ne kadar bant genişliği açısından en verimli yöntem olmasa da, uygulanmasının görece basit olması, birçok istemci tarafından doğrudan görüntülenebilir olması ve düşük gecikmeli kontrol senaryoları için yeterli performansı sağlaması nedeniyle tercih edilmiştir. Mobil uygulama, bu akışı kullanıcıya

gerçek zamanlıya yakın bir hızda göstererek teleskopun yönelimi, takip başarısı ve görüntü kalitesi hakkında anlık geri bildirim sunmaktadır.

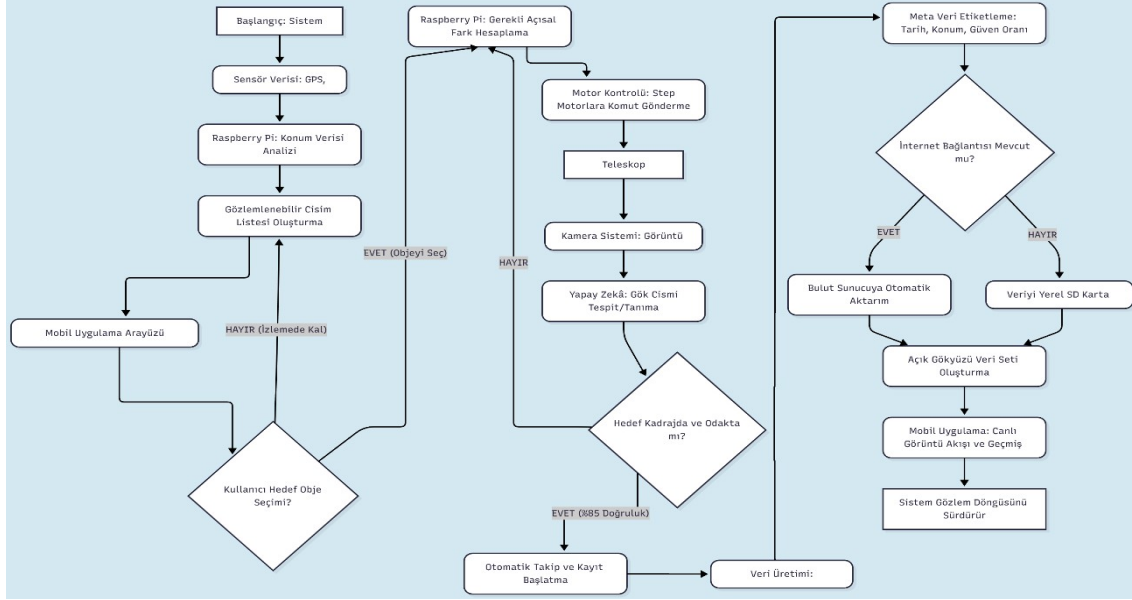
Sunucu tarafı AWS üzerinde barındırılan, PHP (CodeIgniter 3) ve Bootstrap tabanlı bir web uygulaması olarak tasarlanmıştır. Bu sunucu, iki ana rol üstlenmektedir: Birincisi, teleskop sistemlerine ait komutların ve telemetri bilgilerinin yönetildiği yönetici panelini barındırmak; ikincisi, dışarıdan gelen kullanıcıların erişebileceği, veri setine ait görüntüler ve meta verilerin bulunduğu görüntüleme arayüzünü sağlamaktır. Yönetici paneli, her teleskop sistemi için ayrı bir “kontrol alanı” sunmakta; bu alan üzerinden teleskop sahibi kullanıcı, teleskopuna komut gönderebilmekte, canlı akışı izleyebilmekte, güncel konum ve yönelim verilerini inceleyebilmekte, otonom gözlem oturumlarını başlatıp durdurabilmekte ve elde edilen verilerin özetlerini görebilmektedir. Panel, aynı zamanda sistem loglarının ve hata kayıtlarının görülebildiği, sensör bağlantı durumu ve son iletişim zamanlarının görüntülenebildiği bir izleme ekranı içermektedir.

Sunucu mimarisinin önemli bir diğer bileşeni, kullanıcı yönetim sistemi ve çoklu teleskop desteğidir. Dışarıdan gelen kullanıcılar herhangi bir hesap oluşturmadan veri setine ait belirli görüntüleri, meta verileri ve genel istatistikleri görüntüleyebilmektedir. Ancak sisteme veri eklemek, yeni bir teleskop ünitesini kaydetmek veya aktif olarak gözlem verisi göndermek için kullanıcıların kaydolmaları gerekmektedir. Kaydolan her kullanıcı, kendisine ait ayrı bir panel üzerinden kendi teleskop sistemini tanımlayabilmekte; bu tanımlama esnasında teleskopa özgü bir kimlik ve güvenlik anahtarı (API anahtarı vb.) oluşturulmaktadır. Bu sayede sistem, gelecekte farklı kullanıcıların kendi otonom teleskoplarını dahil etmelerine ve hepsinin verilerinin ortak bir veri havuzuna katkı sunmasına olanak tanımaktadır. Böylece başlangıçta tek bir sistemle sınırlı olan veri üretim süreci, farklı kullanıcıların katkılarıyla zaman içinde büyüyen bir açık gökyüzü veri setine dönüşebilmektedir.

Mobil uygulama ile sunucu arasındaki iletişim REST tabanlı servisler üzerinden yürütülmektedir. Kullanıcı uygulama üzerinden bir gök cismi seçtiğinde, otonom takip başlattığında, teleskobu manuel olarak hareket ettirdiğinde veya sistem modunu değiştirdiğinde bu komutlar sunucuya HTTP POST/GET istekleri aracılığıyla iletilmekte; sunucu bu komutları veritabanına kaydederek ilgili teleskop sistemine ait “bekleyen komut” kuyruğuna eklemektedir. Teleskop sistemi, belirli aralıklarla sunucuya gönderdiği GET istekleri ile bu kuyruğu kontrol etmekte; yeni bir komut bulduğunda bunu alarak kendi yerel kontrol döngüsüne dahil etmektedir. Bu mimari, aynı teleskop sistemine aynı anda birden fazla kaynaktan komut gönderilmesini engellemek için tek sahipli bir kontrol mantığıyla birleştirilmiş; uygulama tasarımı, aynı anda yalnızca ilgili teleskop sahibinin ya da yetkilendirilmiş kullanıcının kontrol paneline erişebileceği şekilde kurgulanmıştır.

Sonuç olarak mobil uygulama ve sunucu tabanlı uzaktan kontrol altyapısı, sistemin hem sahada pratik kullanılabilirliğini artırmakta hem de tez kapsamında hedeflenen açık veri üretimi ve çok kullanıcılı teleskop ekosistemi vizyonuna altyapı sağlamaktadır. Mobil uygulama, kullanıcıya teleskop üzerinde yüksek düzeyde kontrol imkânı sunarken; AWS üzerinde barındırılan PHP/CI3 tabanlı web uygulaması, hem bu kontrolün arka planını örgütleyen merkezi bir yapı hem de üretilen verilerin paylaşıldığı ve büyüyen bir veri setinin yönetildiği platform olarak işlev görmektedir. Bu bütünleşik mimari, ilerleyen

aşamalarda yeni teleskop sistemlerinin çalışmaya eklenmesi, gözlem kapasitesinin artırılması ve farklı araştırmacıların üretilen verilere erişerek kendi çalışmalarında kullanması için gerekli temel altyapıyı oluşturmaktadır. Şekil 8’de görüldüğü üzere, otonom teleskop sistemi sensör verilerinin toplanmasıyla başlayan; hedef seçimi, yönlendirme, görüntü yakalama, yıldız/gezegen tanıma, mikro düzeltme ve veri kaydı/aktarımı adımlarını kapsayan bütünleşik bir akış yapısı üzerine kurgulanmıştır.



Şekil 9. Otonom Teleskop Sisteminin Genel Akış Diyagramı

3.7 YÖNTEMİN DİYAGRAMLARLA AÇIKLANMASI VE DENEYSEL SONUÇLARIN DEĞERLENDİRİLMESİ

Geliştirilen otonom teleskop sisteminin işleyişi, sensör verilerinin sürekli işlenmesi, çevresel koşulların değerlendirilmesi, otomatik hedef belirleme süreçleri, motor kontrol mekanizmaları, görüntü işleme adımları, yapay zekâ temelli doğrulama döngüleri ve çok katmanlı veri kaydı yapılarının bir araya geldiği bütünleşik bir mimariye dayanmaktadır. Bu mimarinin anlaşılabilir ve bilimsel olarak ifade edilebilir hâle getirilmesi, yalnızca donanım ve yazılım bileşenlerinin tek tek açıklanmasıyla mümkün değildir. Bunun yerine sistemin davranışının, karar noktalarının, bilgi akışının, zaman bağımlı değişimlerinin ve veri organizasyonunun bir bütün olarak ele alınması gereklidir. Bu amaç doğrultusunda yöntem; akış diyagramı, sözde kod, durum davranış modeli ve varlık-ilişki diyagramı gibi dört farklı modelleme yaklaşımıyla detaylandırılmıştır.

Sistemin genel süreci bir akış diyagramı ile temsil edilmiştir. Diyagram, sistemin başlangıç noktasından veri kaydına kadar olan tüm süreçleri ardışık bloklar hâlinde göstermektedir. İlk aşamalarda sensörlerin hata payı, ısınma süreleri ve konum kararlılığı sağlanana kadar sistem başlangıç modunda kalmaktadır. GPS modülü konum verisini alıp doğruladıktan, IMU birimi eğim ve yönelim bilgisini kararlı şekilde üretmeye başladıktan ve kamera sistemi doğru şekilde görüntü sağlayabilir duruma geldikten sonra sistem konum analizine geçer. Konum analizi yalnızca gök cisimlerinin teorik konumlarının hesaplanması anlamına gelmez; aynı zamanda bulunduğu konumda gökyüzünün ne kadarının fiziksel engeller tarafından kapalı olduğunu da göz önünde bulundurur. Bu nedenle engel tespiti, akış diyagramında sensör verilerinin hemen ardından gelen kritik

bir adımdır.

Engel haritasının çıkarılması, sistemin bulunduğu çevrede hangi doğrultuların gözleme açık olduğu bilgisini üretir. Bu bilgi daha sonra gök cisimlerinin gözlemlenebilir olup olmadığını değerlendirmek için kullanılır. Böylece oluşturulan gözlemlenebilir cisim listesi, yalnızca gökyüzündeki konumlara göre değil, aynı zamanda teleskopun o anki sahadaki görüş alanına göre oluşturulur. Bu liste mobil uygulama arayüzüne aktarılır ve kullanıcı burada doğrudan bir hedef seçebilir. Kullanıcının hedef seçmediği durumda sistem otonom çalışarak en parlak ve en belirgin gök cismini seçer. Bu karar, sahada gözlemin kolaylaştırılması amacıyla tasarlanmıştır.

Hedef seçimiyle birlikte sistem otomatik hedefleme sürecine geçer. Raspberry Pi birimi, hedef gök cisminin konumunu mevcut tarih, saat ve konuma göre hesaplar. Ardından teleskopun mevcut yönelimiyle hedef arasındaki farkı belirler. Bu hesaplamalar sonucunda motor sürücüsüne gönderilecek komutlar oluşturulur. Bu komutların belirlenmesi yalnızca iki açının birbirinden çıkarılmasıyla gerçekleşen basit bir işlem değildir; mekanik kaymalar, toleranslar, step motorların çözünürlüğü ve teleskopun mekanik yapısındaki küçük hata payları da hesaba katılır. Bu nedenle sistem hedefleme işlemini tamamladıktan sonra yapay zekâ modülü aracılığıyla doğrulama yapar.

Otonom davranışın merkezinde yer alan bu doğrulama süreci, yapay zekâ modelinin görüntüyü analiz ederek hedefin gerçekten kadrain merkezine yakın olup olmadığını değerlendirmesiyle çalışır. Doğruluk oranı belirli bir eşik değerinin altına düştüğünde sistem hedefleme sürecini tekrar başlatır. Bu geri besleme döngüsü, sistemin hem dünya dönüşüne hem de motorlardan kaynaklanan küçük kaymalara sürekli uyum sağlamasını mümkün kılar. Böylece teleskop hedefi mümkün olduğunca stabilize edilmiş bir şekilde görüntü merkezinde tutar.

Otonom takip sürecine geçildiğinde sistem artık yalnızca konum hesaplaması yapmaz; görüntüyü düzenli olarak yakalar, analiz eder, değerlendirir ve kaydeder. Kamera tarafından elde edilen her görüntü veya video parçası meta verilerle birlikte zenginleştirilir. Bu meta veriler arasında GPS konumu, zaman damgası, yapay zekâ güven yüzdesi, hedef adı, motor yönelim bilgisi, engel tespit durumu ve bağlantı modu gibi bilgiler bulunur. Bu meta veri yapısı, görüntülerin bilimsel analizlerde kullanılabilmesi için zorunlu olan bilgi bütünlüğünü sağlar.

Veri kaydı süreci bağlantı durumuna göre farklı yollar izler. İnternet bağlantısının mevcut olduğu durumlarda veriler bulut sunucuya aktarılır; bu sayede merkezî bir veri seti oluşur ve diğer kullanıcılar tarafından da görüntülenebilir hâle gelir. Bağlantının olmadığı durumlarda ise veriler yerel SD karta kaydedilir. Böylece saha koşullarında bağlantı problemleri oluşsa bile veri kaybı önlenir.

Bu akışın algoritmik karşılığı sözde kod ile detaylandırılmıştır. Sözde kod, sistemin çerçevesini oluşturan ana döngüyü, sensör okuma adımlarını, hedef seçim mantığını, açısal fark hesaplamalarını, motor komutlarının gönderilmesini, yapay zekâ doğrulama mekanizmasını, veri üretimini ve veri kaydını adım adım gösterir. Sözde kodun uzun hâli, sistemin yüzlerce milisaniyede bir tekrarlanan döngüsel yapısını formalize eder; bu yönüyle algoritmanın bilgisayar bilimleri açısından net ve analiz edilebilir bir formda

sunulmasını sağlar.

3.7.1 GENİŞLETİLMİŞ PSEUDO CODE

SİSTEM BAŞLATILDIĞINDA:

```
SistemDurumu = "Başlangıç"  
// Sensörleri başlatma  
GPS.ModülAç()  
IMU.ModülAç()  
  
Kamera.ModülAç()  
MotorSürücü.ModülAç()  
  
// Sensör kararlılık kontrolü  
GPS_Kararlı = GPS.BekleKonumKilidi()  
IMU_Kararlı = IMU.Kalibre()  
  
Eğer GPS_Kararlı != True veya IMU_Kararlı != True ise  
    Hata("Sensör kalibrasyonu başarısız")  
    TekrarDeneme()  
  
SistemDurumu = "Konum Analizi"
```

ANA DÖNGÜ BAŞLAR:

```
GPS_Verisi = GPS.Oku()  
IMU_Verisi = IMU.Oku()  
Konum = KonumBirleştir(GPS_Verisi, IMU_Verisi)  
GörüntüHam = Kamera.CekFrame()  
EngelDurumu = EngelAnalizi(GörüntüHam)  
  
GörülebilirCisimler = CisimListesi(Konum, EngelDurumu)  
KullanıcıKomutu = KomutOku()  
  
Eğer KullanıcıKomutu.HedefSecildiMi():  
    Hedef = KullanıcıKomutu.Hedef  
Aksi Halde:  
    Hedef = EnParlakCisminOtomatikSecimi(GörülebilirCisimler)  
  
HedefKonumu = EfemerisHesapla(Hedef, TarihSaat, Konum)  
AçısalFark = AciFarki(MevcutAci, HedefKonumu)  
MotorSürücü.HareketEttir(AçısalFark)  
YeniGörüntü = Kamera.CekFrame()  
YZ_Sonuç = YapayZeka.TespitEt(YeniGörüntü)  
  
Eğer YZ_Sonuç.Dogruluk >= 0.85:  
    // Takip başarıyla başladı
```

```
Kayıt_Baslat()  
Meta = MetaVeriOlustur(Konum, TarihSaat, YZ_Sonuç)  
Kayıt = VeriBirleştir(YeniGörüntü, Meta)  
Eğer İnternetVarMı():  
    SunucuyaGonder(Kayıt)  
Aksi Halde:  
    SDepoula(Kayıt)
```

```
Aksi Halde:  
    MotorSürücü.KüçükDüzeltilme()  
    DevamEt  
    Uyku(50-200ms)  
    DöngüTekrar()
```

SİSTEM KAPANANA KADAR BU DÖNGÜ SÜREKLİ ÇALIŞIR

Durum diyagramı ise sistemin davranışının zaman içinde nasıl değiştiğini açıklar. Bu model, sistemin farklı çalışma hâllerini tanımlar ve hangi olayların bu hâller arasında geçişe sebep olduğunu gösterir. Örneğin başlangıç hâlinden manuel moda geçiş yalnızca kullanıcı komutuyla olur. Manuel moddan otomatik hedefleme moduna geçiş yalnızca hedef seçildiğinde gerçekleşir. Otonom takip modunda ise hedef kaybedildiği anda sistem otomatik hedeflemeye geri döner. Bu davranış sistemi hem güvenilir hem de kullanıcı dostu hâle getirir.

Verilerin nasıl saklandığını ve ilişkisel yapının nasıl düzenlendiğini gösteren varlık-ilişki diyagramı, sistemin veri mimarisinin temelini oluşturur. Bu diyagram verilerin hem cihaz üzerindeki hem de sunucu tarafındaki saklanma düzenini birbirine entegre bir şekilde ortaya koyar.

3.8. BÜTÇE KISITLARI ALTINDA GERÇEKLENEN PROTOTİP MİMARİSİ

Prototip mimarisinin seçiminde temel kriter; ideal tasarımdaki **gözlem akışının** bütçe dostu donanım ile uygulanabilir olmasıdır. Raspberry Pi 5 tabanlı saha ajanı yaklaşımı, Picamera2/libcamera [32], [33] ile yüksek kaliteli kare yakalama, PCA9685 sürücü [21] ile çok kanallı servo kontrolü ve TFLite/ONNX [28], [35] ile yerel çıkarım sunarken; toplam maliyet ve tedarik süresi lisans projesi takvimini zorlamıştır. Bu nedenle Mega+ESP32 ikilisi, literatürdeki düşük maliyetli gözlem platformları [4], [5], [6], [17] ile uyumlu bir alternatif olarak değerlendirilmiştir.

Donanım ikamesi yalnızca işlem gücünü değil, **veri yolu mimarisini** de etkilemiştir. İdeal tasarımda Python süreçleri tek kart üzerinde birleşirken; prototipte gerçek zamanlı motor döngüsü Mega'da, ağ/kamera yükü ESP32'de tutulmuştur. UART üzerinden JSON [27] ile haberleşme, pySerial tabanlı seri protokolün gömülü karşılığıdır ve iki katman arasında gevşek bağlı bir sözleşme sağlar.

Bulut tarafında AWS S3 presigned URL [22], FastAPI [26] ve Celery/Redis [25] ile tasarlanan boru hattı; prototipte PHP/MySQL ile sadeleştirilmiş, ancak oturum tabanlı kimlik doğrulama, meta veri arşivleme ve çok kullanıcı galeri işlevleri korunmuştur. Böylece ideal mimarinin **işlevsel hedefleri** korunurken maliyet **operasyonel** düzeyde düşürülmüştür.

Tanıtım videosu [GitHub deposu ve YouTube] prototipin uçtan uca çalışmasını göstermektedir: mobil uygulama üzerinden hedef seçimi, ESP32-CAM canlı yayını,

Mega tarafında servo hareketi ve web arşivine aktarım aynı oturum içinde izlenebilir. Bu bütünlük, bütçe kısıtına rağmen sistemin **kavramsal doğrulamasını** destekler.

Önceki alt bölümlerde (3.1–3.7) sistemin **hedeflenen ideal mimarisi** ayrıntılı biçimde tarif edilmiştir: Raspberry Pi 5 tabanlı bir saha ajanı, PCA9685 sürücülü yüksek torklu servolar [21] ve AWS üzerinde çalışan bulut servisleri (FastAPI [26], Celery/Redis [25], S3 [22]) ekseninde kurgulanan, uçtan uca otomatik bir gözlem platformu. Bu kurgu, akademik açıdan ulaşılması istenen referans tasarımı temsil etmektedir. Ne var ki bir lisans bitirme çalışmasının bütçe ve tedarik kısıtları; özellikle Raspberry Pi 5, yüksek çözünürlüklü IMX477 kamera, endüstriyel sınıf servolar ve sürekli çalışan bir bulut altyapısının toplam maliyeti, bu ideal kurgunun tamamının fiziksel olarak hayata geçirilmesini bu aşamada zorlaştırmıştır.

Bütçe kısıtının teknik kararlar üzerindeki etkisi, yalnızca bir bileşeni ucuzuyla değiştirmekten ibaret değildir; aynı zamanda mimarinin hangi parçasının nerede çalıştırılacağına dair bir yeniden konumlandırma gerektirir. İdeal tasarımda yerel olarak (Raspberry Pi 5 üzerinde) yapılması öngörülen yapay zekâ çıkarımı, prototipte bulut tabanlı bir servise (Google Gemini) taşınmıştır. Bu kaydırma, yerel işlem gücü gereksinimini ortadan kaldırırken sisteme internet bağımlılığı eklemiştir; ancak hedef doğrulama gibi seyrek ve gecikmeye toleranslı işlemler için bu ödünleşim kabul edilebilir bulunmuştur.

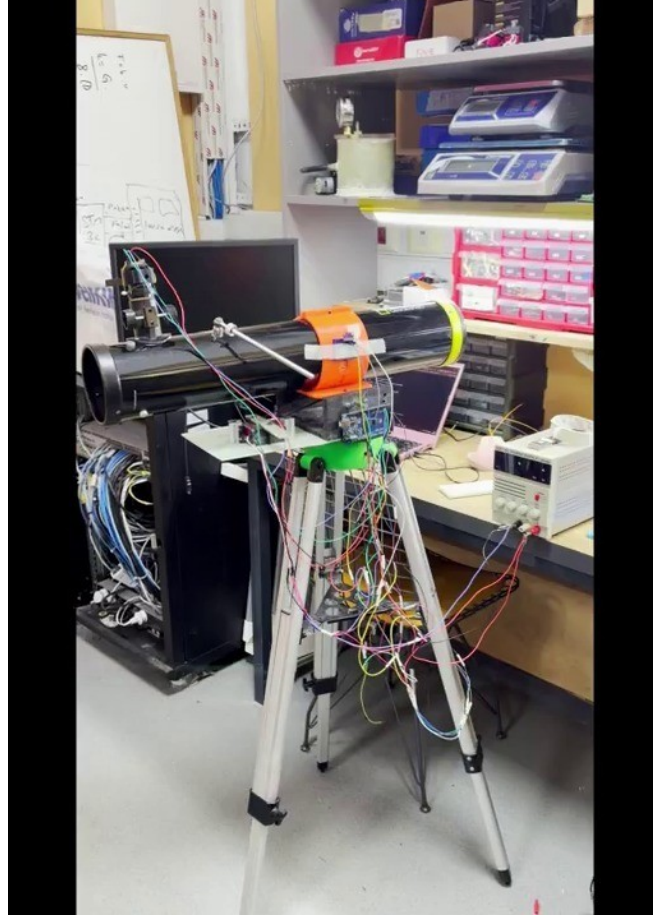
Benzer bir ödünleşim, denetleyici seçiminde de yapılmıştır. Tek bir güçlü işlemci (Raspberry Pi 5) yerine, gerçek zamanlı görevlerin Arduino Mega’ya, ağ ve kamera görevlerinin ESP32-CAM’e dağıtıldığı iki denetleyicili bir yapı tercih edilmiştir. Bu yaklaşım, maliyeti düşürmenin yanı sıra, zamanlama açısından kritik motor denetimini ağ trafiğinin değişken yükünden yalıtarak sistemin belirlenebilirliğini (determinism) artırmıştır. Dolayısıyla bütçe kısıtı, bazı durumlarda mimari açıdan daha sağlam çözümlere de yol açabilmektedir.

Sonuç olarak prototip, ideal tasarımın bir “küçültülmüş” kopyası değil; aynı kavramsal katmanları (algılama, denetim, ağ geçidi, arşiv ve istemci) koruyan, ancak her katmanı erişilebilir teknolojilerle yeniden gerçekleyen bir uyarlamadır. Bu sayede çalışmanın akademik katkısı (otonom gözlem akışı ve gökyüzü veri seti vizyonu) korunurken, fiziksel olarak çalışan ve tekrar üretilebilir bir sistem ortaya konmuştur.

Bu nedenle çalışmanın uygulama ayağında, ideal tasarımın **kavramsal mimarisi** (katmanlı yazılım, donanım soyutlama arayüzleri ve aynı gözlem akışı) korunmuş; ancak maliyet açısından çok daha erişilebilir bileşenlerle kurulan, **çalışır bir prototip** gerçekleştirilmiştir. Düşük maliyetli yıldız takip ve gözlem sistemlerinin literatürde de uygulanabilir olduğu birçok çalışmada gösterilmiştir [4], [5], [6], [17]; bu çalışma da aynı düşük maliyet–erişilebilirlik felsefesini benimsemiştir. Gerçeklenen prototipte üst seviye denetleyici olarak Raspberry Pi yerine **Arduino Mega 2560** ve **ESP32-CAM (AI-Thinker)** ikilisi kullanılmıştır. Görev paylaşımı şu şekildedir: Arduino Mega gerçek zamanlı sensör füzyonu ve servo kontrolünden (“kas + duyu”), ESP32-CAM ise Wi-Fi ağ geçidi, kamera ve Mega ile UART köprüsü görevinden (“ağ + göz”) sorumludur. Bulut tarafı (AWS) yerine kendi sunucumuzda barındırılan **PHP/MySQL** tabanlı bir web platformu; ideal tasarımdaki mobil uygulama ise **Android (Kotlin)** uygulamasıyla karıştırılmıştır.

Önemli olan nokta, bu ikamelerin sistemin **gözlem akışını değiştirmemesidir**: ideal tasarımda tanımlanan “konum belirleme → hedef seçimi → otonom yönlendirme → görüntü yakalama → doğrulama → arşivleme” zinciri, prototipte de aynen işletilmektedir. Aşağıdaki bölümler, bu prototipin mekanik tasarımını, gömülü

yazılımını, web ve mobil bileşenlerini ve gerçek saha test sonuçlarını ayrıntılı biçimde sunmaktadır. Çalışmanın kaynak kodu ve tanıtım videosu açık biçimde paylaşılmıştır.



Şekil 10. Bütçe dostu prototipin sahaya kurulmuş hâli: teleskop tüpü, turuncu 3D baskı halka kelepçeler, yeşil azimut/altitude modülü, alüminyum tripod ve sürücü elektroniği (laboratuvar güç kaynağı ve bağlantı kabloları).

İdeal tasarım ile gerçekleştirilen prototip arasındaki bileşen düzeyindeki karşılıkları Tablo 3.1 özetlemektedir.

Katman	İdeal Tasarım	Gerçeklenen Prototip
Üst denetleyici	Raspberry Pi 5	Arduino Mega 2560 + ESP32-CAM
Kamera	IMX477 (yüksek çöz.)	OV2640 (ESP32-CAM)
Servo sürücü	PCA9685 + yüksek tork	Mega GPIO PWM + hobi servo
Kamera yazılımı	Picamera2 / libcamera	esp_camera (Espressif)
Bulut/sunucu	AWS (FastAPI, Celery, S3)	Kendi sunucu: PHP + MySQL
Çıkarım (AI)	TFLite / ONNX yerel	Google Gemini (bulut, çoklu-kip)
Mobil arayüz	Mobil uygulama	Android (Kotlin) uygulaması

Tablo 3.1'in görsel karşılığı yukarıdaki Şekil 10'da sunulan fiziksel prototiptir.

3.9. MEKANİK TASARIM VE MONTAJ

Prototipin mekanik iskeleti, teleskobu iki ekseninde (azimut/yatay ve yükseklik/dikey) hareket ettirebilen bir **alt-azimut (pan-tilt)** düzeneğidir. Tüm taşıyıcı parçalar 3B yazıcıda (PLA/PETG) üretilmiş; teleskop tüpü, ticari bir alüminyum tripod üzerine oturtulan bu 3B baskı modüllerle taşınmaktadır. Bu yaklaşım, hem maliyeti düşürmüştür hem de tasarımın hızlı yenilenmesine (parçaların yeniden basımıyla) olanak tanımıştır.

3.9.1. İKİ EKSENLİ DÜZENEK VE HAREKET STRATEJİSİ

Azimut eksenini, mutlak konum bilgisi gerektirmeyen ancak 360° kesintisiz dönebilen bir **sürekli dönüş servosu** ile sürülür; bu serviste komut, açı değil **hız** anlamına geldiğinden, hedef azimuta ulaşmak için IMU yön ölçümüne dayalı **kapalı çevrim** bir denetim uygulanır (bkz. 3.10.1). Yükseklik (altitude) eksenini ise doğrudan açı yazılabilen standart bir **konum servosu** ile kontrol edilir ve üst limiti kullanıcı tarafından belirlenir. Bu görev bölüşümü, sürekli dönen bir tabla ile sınırlı açıyla eğilen bir beşiğin birlikte çalıştığı klasik bir alt-azimut montaj mantığını yansıtır.

Alt-azimut montaj, ekvatorial montaja kıyasla mekanik olarak daha basit ve daha düşük maliyetlidir; çünkü kutup hizalaması gerektirmez ve iki eksenini de yere göre dik/yatay konumda çalışır. Bunun bedeli, gök cisminin hareketini izlerken her iki eksenini de eşzamanlı ve değişken hızlarda sürülmesi gerekliliğidir (alan dönmesi). Bu çalışmada hedeflenen parlak cisimler (Ay, gezegenler) için kısa gözlem süreleri yeterli olduğundan, alt-azimut montajın getirdiği bu karmaşıklık periyodik mikro düzeltmelerle yönetilebilir düzeyde kalmıştır.

Azimut ekseninde sürekli dönüş servosunun seçilmesinin temel nedeni, 360° boyunca kesintisiz dönebilme ihtiyacıdır; standart bir konum servosu yaklaşık 180° ile sınırlı olduğundan tam tur azimut taraması için uygun değildir. Ne var ki sürekli dönüş servosunun mutlak konum geri beslemesi yoktur; bu eksiklik, IMU'dan elde edilen gerçek yön ölçümünün geri besleme olarak kullanıldığı kapalı çevrim bir denetimle giderilmiştir. Yükseklik ekseninde ise hedeflenen açı aralığı 180° içinde kaldığından, doğrudan ve kararlı konum kontrolü sunan standart servo yeterli olmuştur.

Her iki ekseninde de ani hareketlerden kaçınmak için adımlı (stepped) bir yaklaşım uygulanır: hedefe doğrudan atlamak yerine, her 300 ms'de yaklaşık 1° ilerlenerek yumuşak bir yörünge izlenir. Bu strateji; servo akım çekişindeki ani sıçramaları azaltır, mekanik titreşimi sınırlar ve özellikle hafif 3B baskı parçalarda rezonans kaynaklı bozulmaları engeller. Hedefe yaklaşıldığında ise her iki eksen için ayrı tolerans bantları (azimutta 1° , kilit için 2°) devreye girerek gereksiz salınım önlenir.

İki eksenli (alt-azimut) düzenekte hareket stratejisi, iki eksenini farklı doğalarını dikkate alır. Azimut eksenini, gök cisimlerinin görünür hareketini izlemek için sürekli ve geniş açıyla dönüş gerektirdiğinden, 360° kesintisiz dönebilen bir servoyla sürülür. Yükseklik eksenini ise sınırlı bir açı aralığında (ufuk ile zenit arasında) çalıştığından, doğrudan açı yazılabilen bir konum servosu için uygundur. Bu görev bölüşümü, klasik alt-azimut montaj mantığını düşük maliyetli bileşenlerle yeniden üretir.

Hareket planlaması sırasında, hedefe büyük açısal sıçramalar yerine kademeli yaklaşım tercih edilmiştir. Özellikle azimut ekseninde, hedefe yaklaşıldıkça dönüş komutunun kesilmesi ve tolerans bandı içinde durdurulması, salınımı (hunting) önler. Manuel kontrol kipinde ise kullanıcı, belirli adım büyüklükleriyle (örneğin 1°) ince ayar yapabilir; bu, otomatik yönlendirme sonrası kadrajın hassas biçimde ortalanmasında kullanışlıdır.

3.9.2. 3B BASKI PARÇA TASARIMLARI

3B baskı parçaların tasarımında, teleskop tüpünün ağırlığı ve uzun kol momenti dikkate alınmıştır. Halka kelepçe (Teleskop_Mount) parçası, tüp çapına göre ayarlanabilir kavrama sağlar; altitude beşiği (teleskop üstü son) ise yükseklik servosunun momentini tablaya iletir. Servolu alt plaka, azimut servosunun eksen hizasını koruyacak şekilde silindirik yatak içerir; timing belt aktarımı (Şekil 23) sürekli dönüş servosunun 360° tabla üzerindeki torkunu artırır.

Atölye videolarında görülen yeşil tabla işleme adımı (Şekil 25), FDM baskı sonrası tolerans birikiminin pratik çözümünü gösterir: parça, mil geçişinde sıkılık sağlanana

kadar mekanik olarak düzeltilmiş, ardından montaja alınmıştır. Bu yinelemeli üretim yaklaşımı, PANOPTES [17] ve benzeri düşük maliyetli teleskop projelerinde de benimsenen hızlı prototipleme felsefesiyle örtüşür.

Montaj sırasında tripod bağlantısı, sistemin sahada taşınabilirliğini sağlar. Kurulu düzenekte turuncu halka kelepçeler optik tüpü, yeşil modül azimut tablasını, siyah kutu gövde ise sürücü elektroniğini barındırır (Şekil 27–28). Elektronik entegrasyonda Mega–ESP32 UART hattı gerilim bölücü ile korunmuş; GPS anteni açık gökyüzü görüşü alacak konuma yerleştirilmiştir.

Mekanik sistemin temel parçaları, montaj öncesi bilgisayar destekli tasarım (CAD) ortamında modellenmiş ve STL formatında dilimlenerek basılmıştır. Aşağıdaki şekiller, bu parçaların üç boyutlu modellerinden alınan görüntüleri sunmaktadır.

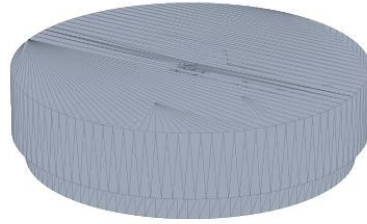
Parçaların büyük çoğunluğu PLA ve daha yüksek mekanik dayanım gereken bağlantılarda PETG filament kullanılarak masaüstü bir FDM (eriyik biriktirme) yazıcıda üretilmiştir. Baskı parametreleri, mukavemet ile üretim süresi arasında denge gözetilerek seçilmiştir: taşıyıcı parçalarda %30–%50 dolgu (infill) oranı, 0,2 mm katman yüksekliği ve yük taşıyan yönler dik yerleşim tercih edilmiştir. Halka kelepçe gibi sıkıştırma kuvveti uygulayan parçalarda ise katmanlar arası ayrılmayı önlemek için duvar (perimeter) sayısı artırılmıştır.

Azimut döner tablası (Şekil 11–12), üst ve alt olmak üzere iki parçadan oluşur; bu iki parça arasına yerleştirilen bir yatak/rulman düzeneği, yükü taşıırken sürtünmeyi azaltır ve tablanın düşük dirençle dönmesini sağlar. Tablanın dış profilindeki dişli benzeri tırtıllar, hem tutuş kolaylığı hem de gerektiğinde bir aktarım elemanı ile kavrama imkânı sağlayacak biçimde tasarlanmıştır. Halka kelepçe ve yükseklik beşiği (Şekil 13), teleskop tüpünün çapına göre ölçeklenmiş ve tüpü iki noktadan kavrayarak eğilme anında kaymayı önleyecek şekilde modellenmiştir.

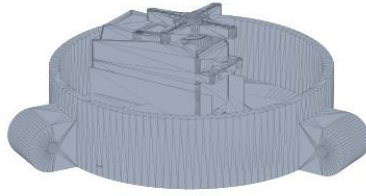
Taşıyıcı taban platformu (Şekil 14), iç kısmındaki çapraz takviyelerle ağırlığı artırmadan burulma rijitliği kazandıracak biçimde tasarlanmıştır; bu, özellikle teleskop yüksek yükseklik açılara eğildiğinde oluşan moment kuvvetlerine karşı önemlidir. Servo tutucu ve montaj braketleri (Şekil 15–16) servoların gövdeye boşluksuz oturmasını sağlar; eksen göbeği (Şekil 17) ise servo mili ile tahrik edilen parça arasındaki kavramayı, baskı toleranslarını telafi edecek sıkı geçme bir arayüzle gerçekleştirir. Tüm parçalar, gerektiğinde yeniden basılarak hızla iyileştirilebilecek modüler bir bütün oluşturur.



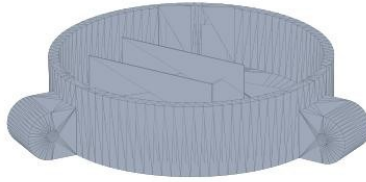
Şekil 11. 360° azimut üst tabla (Montaj) — dişli/tırtıklı dış profilli döner tabla kapak parçası (360 altın üstü plaka).



Şekil 12. 360° üst parça — azimut tablasının düz disk/tabla bileşeni.



Şekil 13. Servolu alt plaka — azimut servosunun oturduğu silindirik yatak gövdesi (alt plaka servolu).



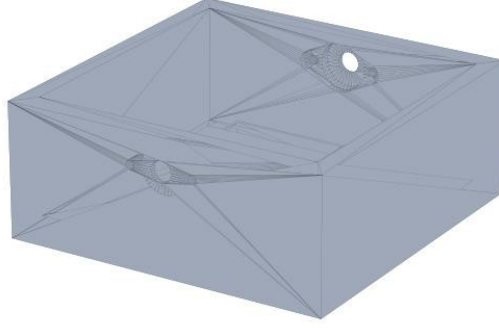
Şekil 14. 360° alt tabla varyantı (telescope alt2 360) — taban montaj modülü.



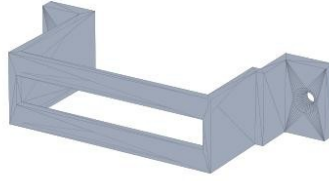
Şekil 15. Teleskop tüpü halka kelepçe montajı (Teleskop_Mount) — tüpü sabitleyen C profil kelepçe ve taban plakası.



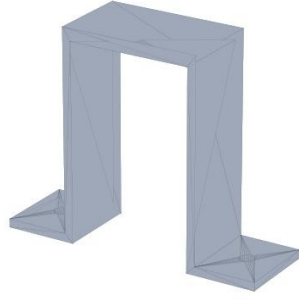
Şekil 16. Teleskop üst montajı (teleskop üstü son) — halka kelepçe + yükseklik (altitude) beşiği birleşimi.



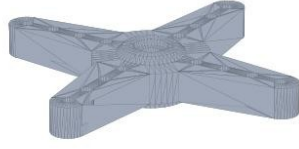
Şekil 17. Alt parça (Alt_Parça) — sürücü elektroniği ve kablolamanın yerleştiği kutu gövde.



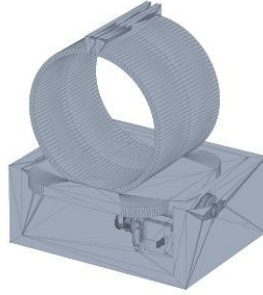
Şekil 18. Servo tutucu 1 — U profilli servo braket.



Şekil 19. Servo tutucu 2 — köprü (II) biçimli montaj braket.



Şekil 20. Servo ucu adaptörü (servo ucu 2) — mil aktarımı için haç biçimli bağlantı parçası.



Şekil 21. Teleskop montaj kesiti — halka kelepçenin alt kutu gövdeye oturtulma görünümü (teleskop montaj kesiti).

3.9.3. ATÖLYEDE MONTAJ VE ENTEGRASYON

3B baskı parçaların üretiminin ardından mekanik düzenek atölye ortamında birleştirilmiştir. Montaj sürecinde; taban plakasının hazırlanması, yatay eksen milinin yatak üzerine oturtulması, servo braketlerinin ve halka kelepçelerin sabitlenmesi ve son olarak sürücü elektroniğinin entegrasyonu adımları izlenmiştir. Bu aşamada baskı toleranslarından kaynaklanan küçük uyumsuzluklar parçaların yeniden basımıyla, servo ucu adaptörlerindeki boşluklar ise sıkı geçme bağlantılarla giderilmiştir.

Montaj sürecinde karşılaşılan başlıca zorluk, 3B baskıya özgü boyutsal toleranslardır. FDM yöntemiyle üretilen deliklerin gerçek çapı, ısı büzülme nedeniyle tasarım değerinden bir miktar küçük çıkabilmektedir; bu durum, cıvata ve mil geçişlerinde sıkışmalara yol açmıştır. Sorun, kritik deliklerin tasarımda hafifçe büyütülmesi veya raybalanmasıyla giderilmiştir. Servo ucu (horn) ile baskı parça arasındaki bağlantıda zamanla boşluk oluşma eğilimi gözlenmiş; bu boşluk, sıkı geçme adaptörler ve gerektiğinde yapıştırıcı destekli sabitlemeyle ortadan kaldırılmıştır.

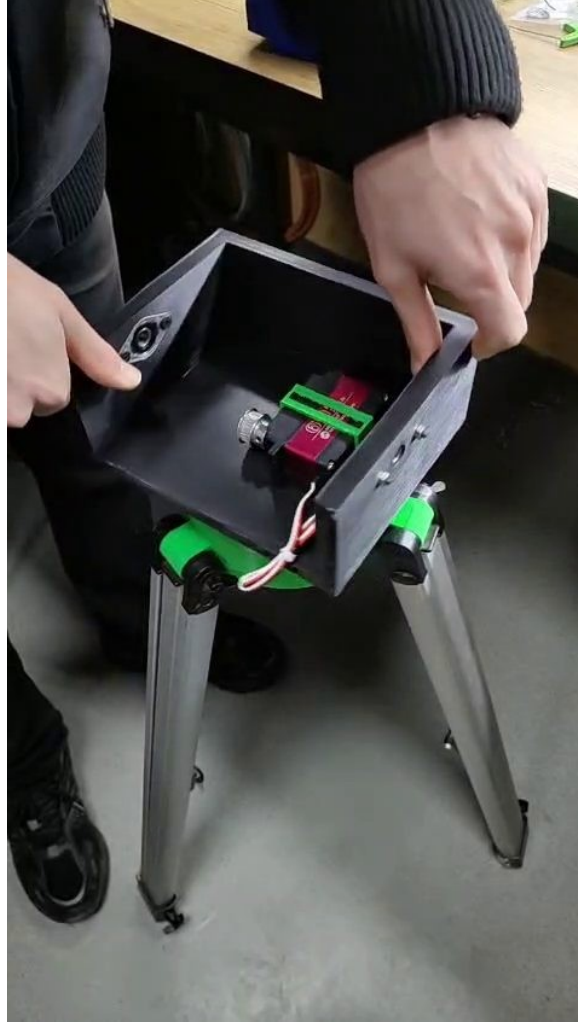
Elektronik entegrasyon aşamasında, Arduino Mega ve ESP32-CAM kartları ile güç regülasyon devresi taban gövdesine yerleştirilmiş; kablolama, eksenlerin hareket açıklığını kısıtlamayacak biçimde pay bırakılarak düzenlenmiştir. Servoların yüksek anlık akım çekişi nedeniyle, servo besleme hattı mantık (lojik) besleme hattından ayrılmış ve ortak toprak (GND) üzerinden birleştirilmiştir. Bu ayırım, servoların hareketi sırasında denetleyicilerde gözlemlenebilecek gerilim düşmesi (brown-out) ve buna bağlı yeniden başlatma sorunlarını önlemiştir.



Şekil 22. Atölyede siyah taban/muhafaza panelinin hazırlanması (WhatsApp Video, 2026-03-16).



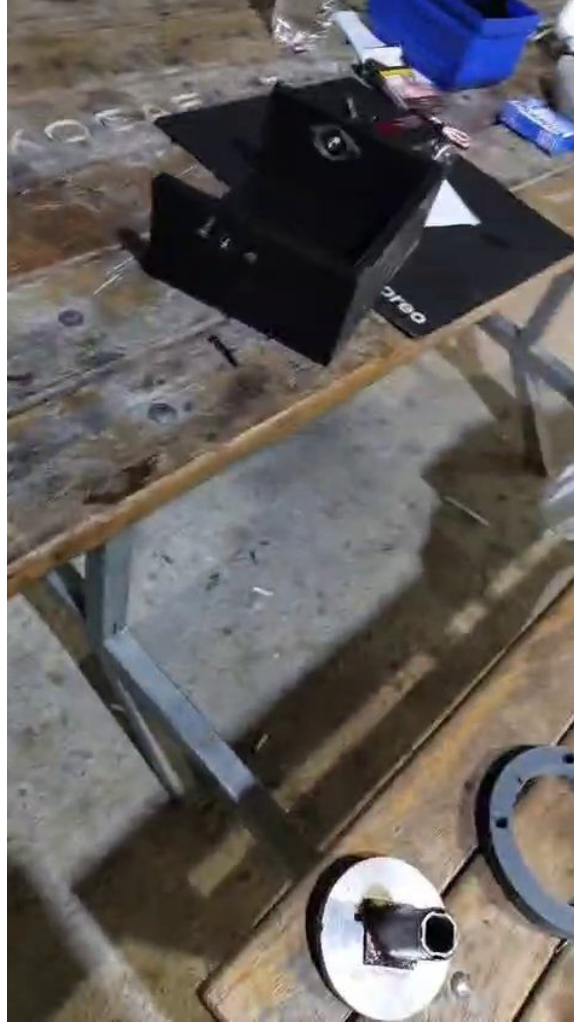
Şekil 23. Azimut eksenini timing belt (zaman kayışı) ve kasnak aktarım mekanizması — sürekli dönüş servosunun tablaya bağlandığı iç montaj.



Şekil 24. 3Baskı siyah gövde ve pan-tilt iskeletinin tripod üzerine birleştirilmesi.



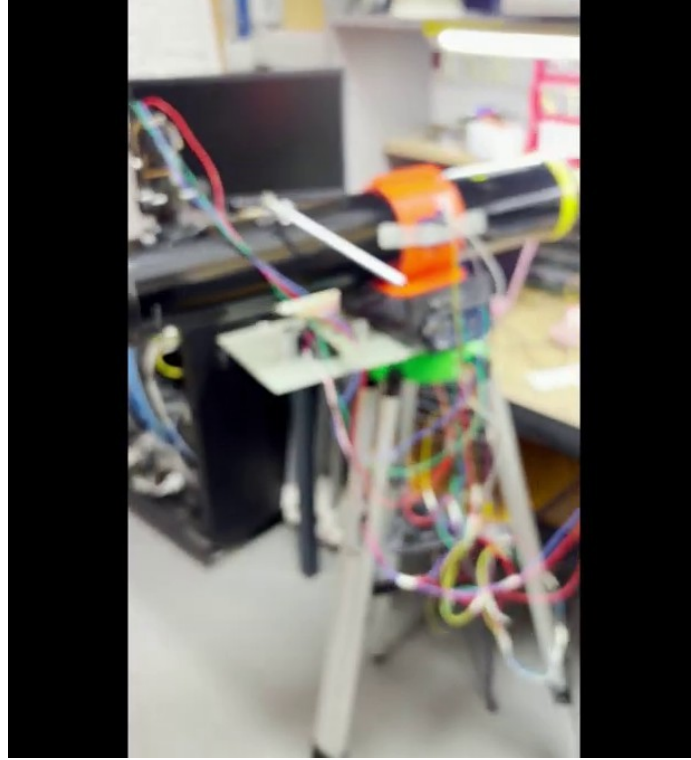
Şekil 25. Yeşil azimut tablasının Dremel ile işlenmesi (tolerans düzeltme).



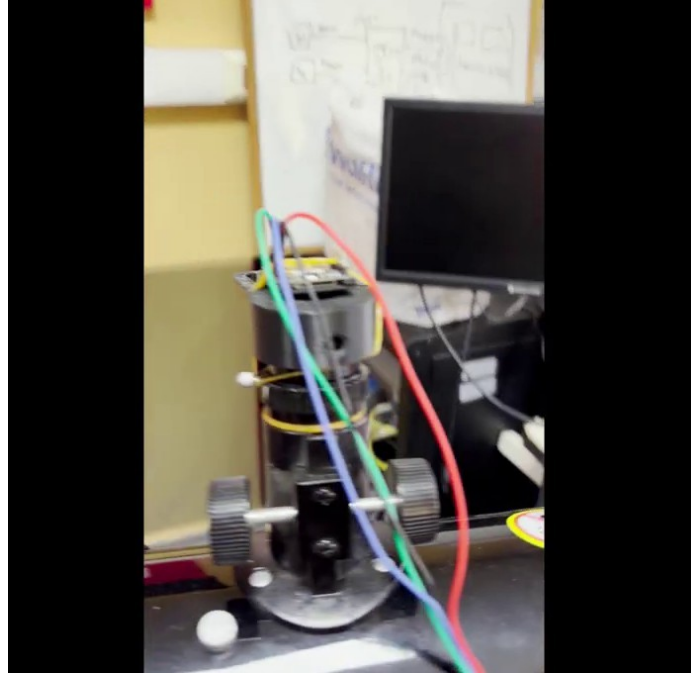
Şekil 26. Atölye tezgahında metal flanş, halka parçalar ve montaj malzemeleri.

3.9.4. KURULU SİSTEMİN GENEL GÖRÜNÜMÜ

Mekanik montaj ve elektronik entegrasyonun tamamlanmasıyla, teleskop tüpü turuncu halka kelepçelerle altitude beşiğine, beşik ise azimut tablası üzerinden tripoda bağlanmıştır. Sürücü elektronığı (Arduino Mega, ESP32-CAM, güç beslemesi ve bağlantı kabloları) tabana yerleştirilmiş; sistem hem atölyede hem de açık alanda kullanılabilir hâle getirilmiştir. Şekil 24, kurulu sistemin arka planda web tabanlı gözlem arşivinin çalıştığı izleme istasyonu ile birlikte genel görünümünü vermektedir.



Şekil 27. Kurulu prototip — teleskop tüpü, turuncu halka kelepçeler, yeşil azimut modülü ve alüminyum tripod (YouTube tanıtım videosu, ~6:12).



Şekil 28. Kurulu sistem ve arka planda web tabanlı Otokop Gözlem Arşivi arayüzünün çalıştığı izleme istasyonu.

3.9.5. MALZEME SEÇİMİ, AĞIRLIK VE DENGİ

Mekanik tasarımda malzeme seçimi, maliyet, üretilebilirlik ve dayanım arasında bir denge gözetilerek yapılmıştır. Geometrik karmaşıklığı yüksek ancak düşük yük taşıyan parçalar PLA ile; sıkıştırma, eğilme veya sürtünmeye maruz kalan kritik bağlantılar ise daha tok ve ısıl dayanımı yüksek PETG ile üretilmiştir. Bu seçim, hem baskı kolaylığını

hem de saha koşullarındaki güneş ısı ve mekanik yüke karşı dayanıklılığı birlikte gözetmektedir.

Alt-azimut bir montajda en kritik mekanik konulardan biri, teleskop tüpünün ağırlık merkezinin yükseklik ekseninde dengelenmesidir. Tüpün dengesiz olması, yükseklik servosunun sürekli bir tutma torku üretmesini gerektirir; bu da hem konum kararlılığını bozar hem de servonun ısınmasına ve erken yıpranmasına yol açar. Bu nedenle halka kelepçenin tüp üzerindeki konumu, sistem yatay konumdayken tüpün kendiliğinden dengede kalacağı noktaya ayarlanmıştır. Gerektiğinde tabana eklenen küçük denge ağırlıkları ile toplam ağırlık merkezi tripod eksenine yaklaştırılarak sistemin devrilme eğilimi azaltılmıştır.

Parça	Malzeme	Başlıca İşlev
Azimut tablası (üst/alt)	PETG	Yatay eksende dönüş, yük taşıma
Halka kelepçe	PETG	Teleskop tüpünü kavrama/sıkıştırma
Yükseklik beşiği	PETG	Dikey eksende eğme
Servo tutucu braketler	PLA	Servoları gövdeye sabitleme
Eksen göbeği	PETG	Servo mili-parça kavraması
Taban platformu	PLA	Bileşenleri taşıma, rijitlik

Tablo 3.3. Başlıca 3B baskı parçaların malzeme ve işlevleri.

3.10. GÖMÜLÜ YAZILIM (FIRMWARE) MİMARİSİ

Prototipin gömülü yazılımı iki ayrı denetleyici üzerinde, görev paylaşımına dayalı olarak çalışır. **Arduino Mega 2560**, gerçek zamanlı sensör füzyonu ve servo kontrolünden; **ESP32-CAM**, Wi-Fi bağlantısı, kamera akışı ve Mega ile köprü görevinden sorumludur. İki kart, 115200 baud hızında, satır sonu (n) ile ayrılan JSON mesajları üzerinden gevşek bağlı (loosely coupled) biçimde haberleşir. Bu ayırım, zamanlama açısından kritik motor denetimini ağ trafiğinin yükünden yalıtarak sistemin kararlılığını artırır.

3.10.1. ARDUINO MEGA: SENSÖR FÜZYONU VE SERVO KONTROLÜ

MPU9250 tabanlı yön kestirimi, manyetometre gürültüsünün çevresel demir ve motor sürücülerden etkilenebileceği gerçeğiyle tasarlanmıştır. Kalibrasyon komutu (/calibrate) gyro bias ve manyetometre hard-iron offset değerlerini EEPROM'a yazar; telefon pusulası ile hizalama (caloffset) ise kullanıcı arayüzünden uygulanabilir. Manyetik sapma (MAG_DECLINATION_DEG) İstanbul için +6° olarak kodlanmıştır; farklı gözlem noktalarında bu sabit güncellenmelidir.

Azimut kapalı çevriminde sürekli dönüş servosu, mutlak enkoder olmadığı için yalnızca IMU tabanlı geri beslemeye güvenir. LOCK_TOLERANCE_DEG (2°) içinde hedef kilitlenmiş sayılır; tracking modunda periyodik mikro düzeltmeler kadrajı korur. Bu yaklaşım, profesyonel ekvatorial mount'ların yıldız takip motorlarına kıyasla basittir; ancak Ay ve parlak gezegen gözlemleri için yeterli doğruluk sağlamıştır [18], [19], [20]. Telemetri JSON paketi (~10 Hz), uygulama tarafında /status ile tüketilir. megaAgeMs, megaBytes gibi tanılama alanları UART kopukluğunu hızla ayırt etmeyi sağlar; bu, saha kurulumunda kritik bir hata ayıklama aracıdır.

Arduino Mega; **MPU9250** (ivmeölçer + jiroskop) ve bünyesindeki **AK8963** manyetometresini I2C üzerinden, **NEO-6M GPS** modülünü ise ayrı bir donanım seri portundan (Serial2, 9600 baud) okur. GPS verisi, NMEA 0183 cümlelerinin çözümlenmesi için TinyGPSPlus kütüphanesiyle işlenir; aynı işlev ideal tasarımda Python tarafında pynmea2 [36] ve pySerial [27] ile karşılanmaktadır. Servo sürücüsü olarak ideal tasarımdaki PCA9685 [21] yerine doğrudan Mega GPIO pinleri kullanılır: azimut servosu 9 numaralı, yükseklik servosu 10 numaralı pine bağlıdır.

Sistemin doğru çalışabilmesi için sensörlerin kalibrasyonu kritik öneme sahiptir. Jiroskopun sıfır noktası (bias), cihaz hareketsizken alınan çok sayıda örneğin ortalamasıyla belirlenir ve sonraki ölçümlerden çıkarılır; aksi hâlde küçük bir sabit kayma, entegrasyon nedeniyle zamanla büyüyen bir yön hatasına dönüşür. Manyetometre ise çevredeki metal kütleler ve elektronik bileşenlerin oluşturduğu sabit manyetik bozulmalara (hard-iron) karşı kalibre edilir; cihaz farklı yönlere döndürülerek toplanan ölçümlerin merkezi kaydırılır. Bu kalibrasyon değerleri ve kullanıcı tanımlı hizalama ofsetleri, gücün kesilmesi durumunda kaybolmaması için EEPROM'a yazılır.

Yön kestirimi, üç farklı referans çerçevesinin tutarlı biçimde birleştirilmesini gerektirir: cihaz gövde çerçevesi, yerel yatay (ufuk) çerçeve ve coğrafi kuzeye dayalı azimut çerçevesi. İvmeölçer, yerçekimi vektörünü ölçerek pitch ve roll açılarını verir; bu açılar, manyetometre okumalarının yatay düzleme izdüşürülmesinde (eğim kompanzasyonu) kullanılır. Manyetik kuzey ile coğrafi kuzey arasındaki fark, gözlem bölgesine özgü manyetik sapma (declination) değeriyle düzeltilir. Böylece elde edilen azimut, gök cismi konumlarının hesaplandığı çerçeveye uyumlu hâle gelir.

Gömülü yazılım, saha koşullarındaki arızalara karşı dayanıklı olacak biçimde tasarlanmıştır. Örneğin, I2C hattındaki geçici bir kopukluk nedeniyle IMU okunamaz hâle gelirse, yazılım sensörü saniyede bir yeniden başlatmayı deneyerek bağlantı düzeldiğinde ölçümün kendiliğinden geri gelmesini sağlar. Benzer şekilde, telemetriye eklenen sağlık bayrakları (imu, gps, lock) ve zaman damgaları, hatanın kaynağının uygulama üzerinden hızlıca teşhis edilmesine olanak tanır.

```
// AZIMUT (pan): SUREKLI DONUS servosu. Komut = HIZ (aci degil).
const int AZ_STOP_US = 1500; // notr (dur) puls genisligi
const int AZ_SLEW_US = 90; // notr'den sapma = donus hizi
const int AZ_DIR = 1; // donusyonu (+1 / -1)
const float AZ_TOLERANCE_DEG = 1.0;
// ALTITUDE (tilt): NORMAL KONUM servosu (0..180 yazilir).
const float ALT_SERVO_SCALE = 1.0; // mekanik oran (1:1 ise 1.0)
// Hareket adimlama: hedefe '300ms'de 1 derece' yumusak yaklasim
const unsigned long STEP_INTERVAL_MS = 300;
const float STEP_DEG = 1.0;
servoAz.attach(9); // azimut -> pin 9 (surekli donus)
servoAlt.attach(10); // altitude-> pin 10 (konum servosu)
```

Şekil 29. Servo mimarisi ve pin atamaları (Arduino Mega).

Yükseklik (altitude) açısı, ivmeölçerden türetilen pitch değeri ile elde edilir. Azimut (yön) açısı ise iki aşamalı bir füzyonla hesaplanır: önce manyetometre verisi pitch/roll ile **eğim-kompanze** edilerek manyetik yön bulunur ve yerel manyetik sapma (İstanbul için yaklaşık +6°) eklenir; ardından jiroskopun Z eksenine entegre edilerek bir **tümleyici (complementary) filtre** ile manyetometre yönüne her döngüde %2 oranında yakınsanır. Bu yaklaşım, manyetometrenin anlık gürültüsünü jiroskopun kısa süreli kararlılığıyla dengeleyerek titreşimli saha koşullarında daha kararlı bir yön kestirimi sağlar.

```
// Tilt-kompanze manyetometre heading
if (readMag(mx, my, mz)) {
    float xh = mx*cos(pitch) + mz*sin(pitch);
    float yh = mx*sin(roll)*sin(pitch) + my*cos(roll) - mz*sin(roll)*cos(pitch);
    float h = atan2(-yh, xh)*180.0/PI + MAG_DECLINATION_DEG; // Istanbul ~+6
    if (h < 0) h += 360.0; magHeading = h;
}
// Gyro Z entegrasyonu + complementary filter (drift'i mag ile duzelt)
float gzDps = (gz - gzBias) / GYRO_SENS;
yaw += gzDps * dt;
float diff = magHeading - yaw;
while (diff > 180) diff -= 360; while (diff < -180) diff += 360;
```

```

yaw += diff * 0.02; // %2 manyetometre duzeltmesi / dongu
heading = fmod(yaw, 360.0); if (heading < 0) heading += 360.0;
azReal = normAz(heading + azOffset); altReal = altitude + altOffset;

```

Şekil 30. Eğim-kompanze pusula yönü ve jiroskop tümleyici filtresi (Arduino Mega). Servo kontrolü iki farklı stratejiyle yürütülür. Azimut ekseninde mutlak konum veremeyen sürekli dönüş servosu kullanıldığı için, hesaplanan gerçek azimut ile hedef arasındaki en kısa açısal fark sürekli izlenir; fark belirli bir toleransın ($AZ_TOLERANCE_DEG = 1^\circ$) üzerindeyse servo nötr darbe genişliğinden (1500 μs) sapacak biçimde döndürülür, tolerans içine girince durdurulur. Bu, IMU geri beslemesine dayalı kapalı çevrim bir konumlandırma değildir. Yükseklik ekseninde ise standart konum servosu doğrudan açı değerine sürülür ve kullanıcı tanımlı üst limitle sınırlandırılır. Her iki eksen de hedefin $LOCK_TOLERANCE_DEG (2^\circ)$ yakınına geldiğinde “hedef kilidi” durumuna geçer.

```

void driveServos() {
    // AZIMUT: sürekli donus servosu, MPU heading'e gore kapali dongu
    float errAz = shortestAngle(azReal, azCmd);
    if (haveTarget && fabs(errAz) > AZ_TOLERANCE_DEG) {
        int dir = (errAz > 0) ? 1 : -1;
        servoAz.writeMicroseconds(AZ_STOP_US + AZ_DIR * dir * AZ_SLEW_US);
    } else {
        servoAz.writeMicroseconds(AZ_STOP_US); // dur (notr)
    }
    // ALTITUDE: normal konum servosu (kullanici limitiyle)
    float a = constrain(altCmd * ALT_SERVO_SCALE, 0.0, 180.0);
    servoAlt.write((int)round(a));
    if (haveTarget) { // hedef kilidi tespiti
        float lockAz = fabs(shortestAngle(azReal, targetAz));
        float lockAlt = fabs(constrain(targetAlt, 0, altMaxLimit) - altCmd);
        targetLocked = (lockAz < LOCK_TOLERANCE_DEG && lockAlt < LOCK_TOLERANCE_DEG);
    }
}

```

Şekil 31. Servo sürüş döngüsü ve hedef kilidi tespiti (Arduino Mega). Mega, hesapladığı tüm durum bilgisini saniyede yaklaşık on kez (~10 Hz) tek bir JSON satırı hâlinde ESP32-CAM’e gönderir. Bu telemetri; gerçek ve hedef açılar, servo konumları, GPS kilidi ve koordinatları, IMU sağlığı, takip ve kilit bayraklarını içerir.

```

void sendTelemetry() { // ~10 Hz, Serial1 @ 115200
    StaticJsonDocument<512> doc;
    doc["az"] = round(azReal*10)/10.0; doc["alt"] = round(altReal*10)/10.0;
    doc["taz"] = round(targetAz*10)/10.0; doc["talt"] = round(targetAlt*10)/10.0;
    doc["sAz"] = round(servoAzPos*10)/10.0; doc["sAlt"] = round(servoAltPos*10)/10.0;
    doc["gps"] = gps.location.isValid();
    if (gps.location.isValid()) {
        doc["lat"] = gps.location.lat(); doc["lon"] = gps.location.lng();
    }
    doc["imu"] = imuOk; doc["trk"] = tracking; doc["lock"] = targetLocked;
    serializeJson(doc, Serial1); Serial1.print('\n');
}

```

Şekil 32. ~10 Hz telemetri JSON paketinin üretimi (Arduino Mega). Tümleyici filtrenin (complementary filter) başarımı, jiroskop ve manyetometre katkılarının ağırlıklandırılmasına bağlıdır. Bu çalışmada yön kestirimi, kısa vadede jiroskop entegrasyonuna (yüksek frekanslı, sürüklenmeye eğilimli) ve uzun vadede manyetometre okumasına (düşük frekanslı, gürültülü fakat sürüklenmesiz) güvenecek biçimde yapılandırılmıştır. Manyetometre düzeltmesi her döngüde yalnızca küçük bir katsayıyla (0,02) uygulanır; böylece anlık manyetik bozulmalar yön kestirimini ani

biçimde saptırmaz, fakat jiroskop sürüklenmesi zamanla bastırılır. Bu denge, hareketli bir düzenekte hem kararlı hem de tepkisel bir azimut ölçümü sağlamaktadır.

Eğim kompanzasyonu (tilt compensation), düzeneğin tam yatay olmadığı durumlarda manyetometre kaynaklı yön hatasını önemli ölçüde azaltır. Pusula yönü hesaplanırken ivmeölçerden elde edilen yuvarlanma (roll) ve yunuslama (pitch) açıları kullanılarak manyetik bileşenler yatay düzleme izdüşürülür. Ayrıca yerel manyetik sapma (declination) açısı eklenerek manyetik kuzey ile coğrafi kuzey arasındaki fark düzeltilir. Bu adımlar olmaksızın, özellikle sahada eğimli zeminlerde, azimut hatası onlarca dereceye ulaşabilmektedir.

Ana denetim döngüsü, telemetri üretimi ve servo sürüşü yaklaşık 10 Hz'lik bir hızda yürütülür. Bu hız, hobi servolarının mekanik tepki süresi ve UART hattının veri kapasitesi göz önüne alındığında, hem yeterince tepkisel hem de kararlı bir denetim sağlamaktadır. Daha yüksek döngü hızları, sürekli dönüş servosunda salınıma (avlanma/hunting) yol açabildiğinden, hedef toleransı (azimutta $\sim 1^\circ$) ile birlikte dikkatle ayarlanmıştır.

3.10.2. ESP32-CAM: Wİ-Fİ, HTTP API VE KAMERA

ESP32-CAM, hem bir erişim noktası (AP, “Otoskop” SSID) hem de mevcut bir ağa istemci (STA) olarak bağlanabilen ikili Wi-Fi kipinde çalışır ve kendisini ağda mDNS ile **otoskop.local** adıyla yayınlar; böylece IP adresi değişse bile mobil uygulama cihazı bulabilir. Cihaz, 80 numaralı portta REST tarzı bir HTTP API, 81 numaralı portta ise canlı **MJPEG** yayını sunar. Yayının ayrı bir portta verilmesi, uzun süreli akışın API isteklerini tıkamasını önler. Tüm yanıtlar CORS başlığı taşır.

ESP32-CAM’in ikili Wi-Fi kipinde çalışması, sahada esneklik sağlar. Mevcut bir kablolu ağ bulunduğunda cihaz bu ağa istemci (STA) olarak katılır; aksi hâlde kendi erişim noktasını (AP) yayınlayarak mobil cihazın doğrudan bağlanmasına izin verir. Bu sayede internet altyapısının bulunmadığı karanlık gözlem alanlarında dahi sistem kullanılabilir. mDNS ile yayınlanan otoskop.local adı, IP adresi DHCP tarafından değiştirilse bile bağlantının sürekliliğini güvence altına alır.

Kamera tarafında, Espressif’in esp_camera sürücüsü kullanılarak çözünürlük, JPEG kalite ve kare hızı parametreleri PSRAM kapasitesi gözetilerek yapılandırılır. Canlı yayın, performans ile bant genişliği arasında denge kurularak yaklaşık 15 kare/saniye hızında sunulur. API ve yayın sunucularının ayrı portlarda (80 ve 81) çalıştırılması, tek iş parçacıklı sunucu üzerinde uzun süreli akışın kontrol komutlarını bloke etmesini engelleyen kasıtlı bir tasarım kararıdır.

Kablolu güncelleme (OTA) yeteneği, sahaya kurulu bir sistemin fiziksel müdahale olmaksızın güncellenebilmesi açısından önemlidir. Cihaz, hem aynı ağ üzerinden ArduinoOTA ile yerel güncellemeyi hem de uzak bir sürüm dosyasını (version.json) periyodik olarak yoklayan bulut tabanlı çekme mekanizmasını destekler. Yeni bir sürüm saptandığında, firmware ikilisi indirilirken durum LED’i yanıp sönmeye geçerek kullanıcıyı bilgilendirir ve güncelleme tamamlandığında cihaz yeniden başlatılır.

```
server.on("/status", HTTP_GET, handleStatus);
server.on("/gps", HTTP_GET, handleGps);
server.on("/camera", HTTP_GET, handleCamera);
server.on("/target", HTTP_POST, handleTarget);
server.on("/move", HTTP_POST, handleMove);
server.on("/correction", HTTP_POST, handleCorrection);
server.on("/calibrate", HTTP_POST, handleCalibrate);
server.on("/limits", HTTP_POST, handleLimits);
server.begin(); // API port 80
// MJPEG yayını ayrı portta (81) -> API'yi tıkmaz
streamServer.on("/stream", HTTP_GET, handleStream);
streamServer.begin(); // stream port 81
```

Şekil 33. HTTP uç noktalarının yönlendirilmesi ve çift sunucu tasarımı (ESP32-CAM). ESP32-CAM, Mega'dan gelen telemetriyi önbellege alır ve /status uç noktası çağrıldığında bu veriyi, bağlantı teşhis bilgileriyle (son Mega satırının yaşı, satır ve bayt sayaçları) birlikte mobil uygulamaya uygun anahtar adlarıyla JSON olarak sunar. Bu teşhis alanları, fiziksel UART hattındaki kopuklukların sahada hızla saptanmasını sağlar.

```
void handleStatus() { // Mega telemetrisini JSON'a çevir
    StaticJsonDocument<512> doc;
    doc["azimuth"] = tele.az; doc["altitude"] = tele.alt;
    doc["targetAzimuth"] = tele.taz; doc["targetAltitude"] = tele.talt;
    doc["servoAz"] = tele.sAz; doc["servoAlt"] = tele.sAlt;
    doc["gpsFix"] = tele.gps;
    doc["imuOk"] = tele.imu && (millis() - tele.lastUpdate < 5000);
    doc["tracking"] = tele.trk; doc["targetLocked"] = tele.lock;
    doc["megaAgeMs"] = (lastMegaRecvMs==0)?-1L:(long)(millis()-lastMegaRecvMs);
    String out; serializeJson(doc, out);
    server.sendHeader("Access-Control-Allow-Origin", "*");
    server.send(200, "application/json", out);
}
```

Şekil 34. /status uç noktası ve UART bağlantı teşhisi (ESP32-CAM).

Cihaz ayrıca iki yöntemle kablosuz güncelleme (OTA) destekler: aynı ağdaki ArduinoOTA ile ve bir bulut dizinindeki sürüm dosyasını (version.json) beş dakikada bir yoklayan HTTP tabanlı çekme mekanizmasıyla. Yeni sürüm bulunduğunda firmware ikili dosyası indirilerek cihaz kendini günceller.

ESP32-CAM, prototipte üç ayrı görevi tek bir düşük maliyetli modülde birleştirir: Wi-Fi ağ geçidi, kamera ve Arduino Mega ile UART köprüsü. Wi-Fi tarafında modül, yerel ağa bağlanır ve kendisini mDNS üzerinden tanıtarak Android uygulamasının elle IP girişi olmadan cihazı bulmasını sağlar. HTTP API'si, durum sorgulama, GPS verisi, hedef atama ve manuel hareket gibi işlevleri ayrı uç noktalar hâlinde sunar; bu uç noktalar, tüm kaynaklardan erişime izin veren CORS başlıklarıyla döndürülür.

Kamera ve canlı görüntü akışı, ana HTTP sunucusundan ayrı, ikinci bir sunucu örneği üzerinden sağlanır. Bu çift sunucu tasarımının amacı, sürekli ve bant genişliği yoğun olan MJPEG akışının, kısa ve sık gelen durum/komut isteklerini bloke etmesini önlemektir. Böylece kullanıcı canlı görüntüyü izlerken, telemetri yoklaması ve hareket komutları gecikmeden işlenebilir. Kamera modülünün çözünürlük ve kare hızı parametreleri, ESP32'nin bellek ve işlem kısıtları gözetilerek dengeli biçimde ayarlanmıştır.

3.10.3. UART KÖPRÜSÜ VE JSON PROTOKOLÜ

İki denetleyici arasındaki tüm haberleşme, insan tarafından da okunabilen JSON satırlarıyla yapılır. Mega'dan ESP'ye telemetri, ESP'den Mega'ya ise komutlar akar. Bu metinsel protokol, hata ayıklamayı kolaylaştırır ve iki tarafın birbirinden bağımsız geliştirilmesine olanak tanır.

```
// Mega -> ESP (telemetri, ~10 Hz, her satır \n ile biter)
{"az":133.2,"alt":48.2,"taz":171.7,"talt":49.5,"gps":true,
 "lat":40.575,"lon":30.013,"imu":true,"trk":true,"lock":false}

// ESP -> Mega (komutlar)
{"cmd":"target","az":171.74,"alt":49.49} // hedefe yönel
{"cmd":"move","dir":"up","step":1.0} // manuel adım
{"cmd":"calibrate"} // jiro/mag kalibrasyonu
{"cmd":"limits","altMax":80.0} // yukseklik ust limiti
```

Şekil 35. UART köprüsündeki JSON telemetri ve komut protokolü örnekleri.

Arduino Mega ile ESP32-CAM arasındaki UART köprüsünde taşınan telemetri paketinin alanları ve anlamları Tablo 3.2'de özetlenmiştir. Paket, satır sonu karakteriyle (\n)

sonlandırılan tek satırlık bir JSON nesnesidir; bu sayede alıcı taraf paketleri güvenilir biçimde çerçeveleyebilmektedir.

Alan	Tür	Açıklama
az	float	Ölçülen azimut açısı (°)
alt	float	Ölçülen yükseklik açısı (°)
taz	float	Hedef azimut açısı (°)
talt	float	Hedef yükseklik açısı (°)
sAz	float	Azimut servo konumu (°)
sAlt	float	Yükseklik servo konumu (°)
gps	bool	GPS konum kilidi durumu
lat/lon	float	Enlem/boylam (GPS geçerliyse)
imu	bool	IMU sağlık durumu
trk	bool	Takip (tracking) etkin mi
lock	bool	Hedef kilidi sağlandı mı

Tablo 3.2. Mega → ESP32 yönündeki telemetri JSON paketinin alanları.

3.10.4. KALİBRASYON YORDAMLARI VE EEPROM YAPILANDIRMASI

Doğru yön kestirimi için iki temel kalibrasyon yordamı, mobil uygulamadan tek dokunuşla tetiklenebilir. Jiroskop kalibrasyonunda cihaz birkaç saniye hareketsiz tutulur ve bu süre boyunca toplanan örneklerin ortalaması sıfır noktası (bias) olarak belirlenir. Manyetometre kalibrasyonunda ise cihaz farklı yönlerde yavaşça döndürülür; toplanan ölçümlerden minimum ve maksimum değerler bulunarak sabit manyetik bozulmalar (hard-iron) telafi edilir. Bu yordamlar, gözlem öncesi kısa bir hazırlık adımı olarak uygulanır.

Kalibrasyon sonuçlarının yanı sıra, kullanıcı tanımlı hizalama ofsetleri (azimut/yükseklik) ve yükseklik üst limiti gibi yapılandırma değerleri, gücün kesilmesi durumunda kaybolmaması için mikrodenetleyicinin kalıcı belleğine (EEPROM) yazılır. Böylece sistem her açılışta sıfırdan kalibrasyon gerektirmez; bir önceki oturumun ayarlarıyla çalışmaya hazır hâle gelir. Yükseklik üst limiti, teleskobun mekanik olarak ulaşabileceği maksimum açıyı aşmasını ve olası çarpışmaları önleyen bir güvenlik tedbiri olarak işlev görür.

Kalibrasyon yordamları, prototipin saha koşullarında güvenilir çalışması için kritik öneme sahiptir. Sistemde iki temel kalibrasyon türü uygulanır: manyetometre (pusula) ofset/ölçek kalibrasyonu ve yön (azimut) referans kalibrasyonu. Manyetometre kalibrasyonunda, sensör tüm eksenlerde döndürülerek ham ölçümlerin oluşturduğu elipsoid merkez ve ölçek değerleri kestirilir; bu sayede sabit (hard-iron) ve ölçeğe bağlı (soft-iron) bozulmalar telafi edilir. Yön referans kalibrasyonunda ise kullanıcı, teleskobu bilinen bir azimuta (örneğin telefon pusulasıyla belirlenen yöne) yönlendirir ve sistem bu noktayı sıfır referansı olarak kaydeder.

Kalibrasyon ve yapılandırma parametreleri, güç kesintilerinde kaybolmaması için mikrodenetleyicinin kalıcı belleğinde (EEPROM) saklanır. EEPROM'a yazılan başlıca değerler; azimut ofseti, yükseklik ofseti, manyetometre kalibrasyon katsayıları ve kullanıcı tanımlı yükseklik üst limitidir. Açılışta bu değerler okunarak sistem, bir önceki oturumun kalibrasyon durumuyla başlatılır. Geçerli bir kalibrasyon kaydının bulunup bulunmadığı, EEPROM'un başına yazılan bir imza (magic) baytıyla denetlenir; imza yoksa varsayılan değerlerle güvenli bir başlangıç yapılır.

3.10.5. GÜÇ MİMARİSİ VE GÜVENLİK

Sistemin güç mimarisi, servoların yarattığı yüksek ve ani akım taleplerinin denetleyicilerin kararlılığını bozmamasını sağlayacak biçimde tasarlanmıştır. Servo besleme hattı, mantık (lojik) besleme hattından ayrılmış; iki hat yalnızca ortak toprak (GND) üzerinden birleştirilmiştir. Bu ayrım, servoların hareketi sırasında oluşan gerilim

dalgalanmalarının Arduino Mega ve ESP32-CAM kartlarında yeniden başlatmaya (brown-out) yol açmasını engeller.

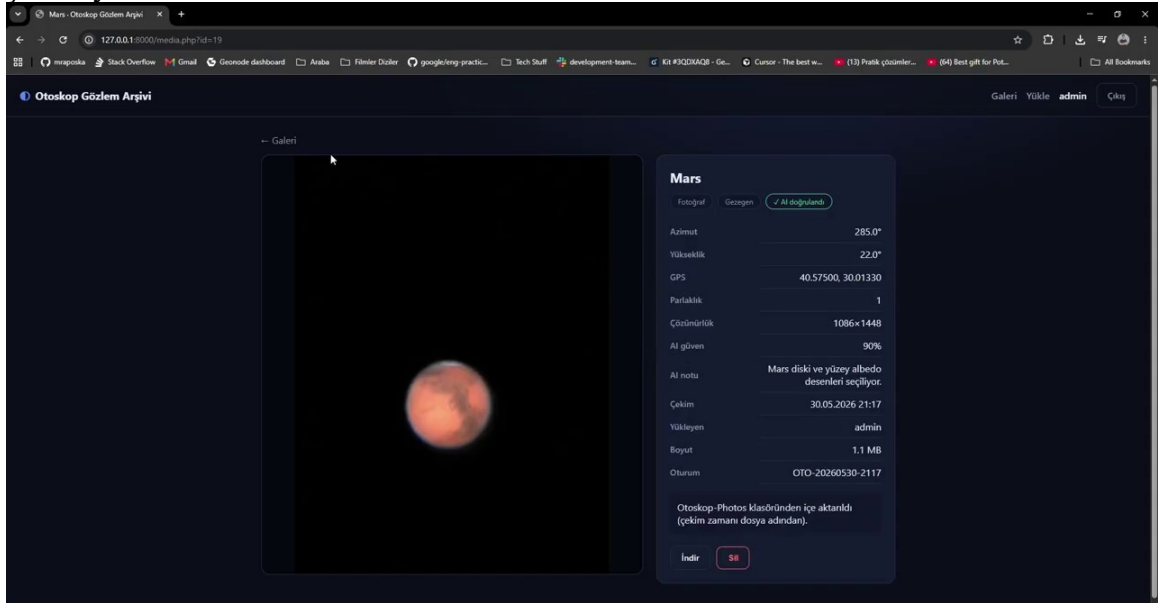
Saha kullanımında sistem, taşınabilir bir güç kaynağından (taşınabilir akü veya güç bankası) beslenebilecek şekilde planlanmıştır; bu, şebeke elektriğinin bulunmadığı karanlık gözlem alanlarında bağımsız çalışmayı mümkün kılar. ESP32-CAM'in görece yüksek anlık tüketimi (özellikle Wi-Fi iletimi ve flaş LED kullanımı sırasında) göz önünde bulundurularak, güç düzenleme devresi yeterli akım marjıyla seçilmiştir. Güvenlik açısından, eksen hareketleri yazılımsal limitlerle sınırlandırılmış ve kablolama, hareketli parçalara takılmayacak biçimde sabitlenmiştir.

Güç mimarisi, Arduino Mega, ESP32-CAM, servolar ve sensörlerin farklı gerilim ve akım gereksinimlerini güvenli biçimde karşılayacak şekilde tasarlanmıştır. Servolar, özellikle hareket anında yüksek akım çekebildiğinden, denetleyici kartların lojik beslemesinden ayrı bir hat üzerinden beslenir; bu ayrım, servo akım darbelerinin denetleyicide gerilim düşmesine ve beklenmedik yeniden başlatmalara (brown-out) yol açmasını önler. ESP32-CAM, Wi-Fi iletimi sırasında ani akım talepleri oluşturduğundan, beslemesi yeterli akım kapasitesine sahip bir hat üzerinden ve uygun dekaplaj kapasitörleriyle sağlanır.

Güvenlik açısından, yükseklik eksenini için yazılımsal bir üst limit tanımlanmıştır; bu limit, teleskobun mekanik olarak çarpışabileceği veya dengesini kaybedebileceği açılara ulaşmasını engeller. Ayrıca telemetri akışı kesildiğinde veya UART hattı zaman aşımına uğradığında, servolar güvenli (nötr) konuma alınarak kontrolsüz hareket riski azaltılır. Bu önlemler, hem donanımın hem de yakın çevredeki kullanıcının korunmasına yöneliktir ve sistemin gözetimsiz çalıştırılabilmesinin ön koşuludur.

3.11. WEB TABANLI GÖZLEM ARŞİVİ (PHP/MYSQL)

İdeal tasarımı AWS üzerinde FastAPI [26], Celery/Redis [25] ve S3 [22] ile kurgulanan bulut altyapısı, prototipte kendi sunucumuzda barındırılan, çerçeve kullanmayan sade bir **PHP** uygulaması ve **MySQL/MariaDB** veritabanıyla gerçekleştirilmiştir. “Otoskop Gözlem Arşivi” adlı bu platform; çok kullanıcıli bir gökyüzü veri seti vizyonunu hayata geçirir: kullanıcılar gözlem görüntülerini zengin meta verilerle yükleyebilir, filtreleyebilir ve arşivi büyütebilir.



Şekil 36. Web tabanlı gözlem arşivinde bir Mars gözleminin detay sayfası: azimut/yükseklik, GPS, parlaklık, yapay zekâ doğrulama güveni ve diğer meta veriler.

3.11.1. VERİTABANI ŞEMASI VE KİMLİK DOĞRULAMA

Web platformu, çerçevesiz PHP ve MySQLi ile geliştirilmiştir. Oturum tabanlı giriş kapısı (lib.php), CSRF koruması ve hazırlıklı ifadeler (prepared statements) SQL enjeksiyonuna karşı savunma sağlar. uploads/ dizini .htaccess ile doğrudan dizin listelemeye kapatılmış; dosya adları zaman damgası + rastgele hex ile çakışmasız üretilir. Galeri görünümü (Şekil 38), her gözlemin hedef adı, azimut/yükseklik, GPS koordinatı, parlaklık ve AI doğrulama skorunu listeler. Harita görünümü (Şekil 36 bağlamında) gözlemlerin coğrafi dağılımını gösterir; bu, çok kullanıcıli büyüyen bir **Gökyüzü Veri Seti** vizyonunun pratik adımıdır.

CLI betikleri (import_photos.php, update_meta.php) toplu içe aktarım ve meta veri güncellemesine izin verir. Ideal tasarımdaki Celery kuyruğu [25] yerine senkron PHP işlemleri kullanılmış olsa da, oturum paketleme ve standart meta veri şeması korunmuştur (Şekil 7 ile uyumlu).

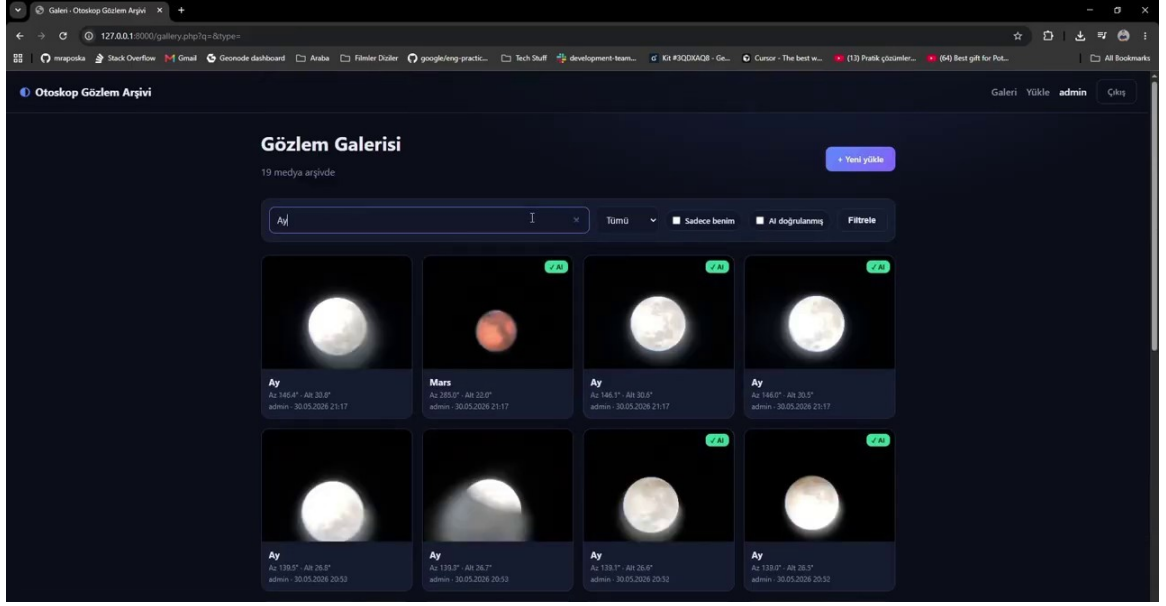
Veritabanı, uygulamanın ilk çalıştırılmasında otomatik olarak oluşturulur ve iki ana tablodan oluşur: kullanıcıları tutan **users** ve gözlem ortamlarını meta verileriyle saklayan **media**. Kimlik doğrulama, PHP oturumları üzerinden yürütülür; parolalar password_hash() ile özetlenir, tüm sorgular hazır ifadelerle (prepared statements) çalıştırılır ve POST formları CSRF jetonlarıyla korunur. İlk kaydolun kullanıcı yönetici (admin) rolünü alır.

Web platformunun güvenlik modeli, sade ancak yaygın saldırı yüzeylerini kapatacak biçimde tasarlanmıştır. Tüm veritabanı sorguları, SQL enjeksiyonunu önlemek için hazır ifadelerle (prepared statements) parametrelili olarak çalıştırılır. Kullanıcı parolaları asla düz metin olarak saklanmaz; PHP'nin password_hash() işleviyle (varsayılan olarak bcrypt) özetlenir ve doğrulama password_verify() ile yapılır. Oturum yönetimi, PHP'nin yerleşik oturum mekanizmasıyla sağlanır ve her POST isteği, siteler arası istek sahteciliği (CSRF) saldırılarına karşı tek kullanımlık bir jetonla korunur.

Yetkilendirme tarafında basit fakat işlevsel bir rol modeli benimsenmiştir: sisteme kaydolun ilk kullanıcı otomatik olarak yönetici (admin) rolünü alır, sonraki kullanıcılar ise standart kullanıcı olarak kaydedilir. Yönetici, arşivdeki tüm kayıtları görüntüleyip yönetebilirken, standart kullanıcılar kendi yükledikleri ortamlar üzerinde işlem yapabilir. Veritabanı şeması, uygulama ilk kez çalıştırıldığında otomatik olarak oluşturulduğundan, platformun yeni bir sunucuya kurulumu ek bir el ile yapılandırma gerektirmez.

```
function migrate(mysqli $c): void {
    $c->query("CREATE TABLE IF NOT EXISTS users (
        id INT AUTO_INCREMENT PRIMARY KEY,
        username VARCHAR(50) NOT NULL UNIQUE,
        email VARCHAR(190) NOT NULL UNIQUE,
        password_hash VARCHAR(255) NOT NULL,
        role ENUM('user','admin') NOT NULL DEFAULT 'user',
        created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP");
    $c->query("CREATE TABLE IF NOT EXISTS media (
        id INT AUTO_INCREMENT PRIMARY KEY, user_id INT NOT NULL,
        type ENUM('photo','video') NOT NULL, file_name VARCHAR(255) NOT NULL,
        target_name VARCHAR(120), object_type VARCHAR(40),
        azimuth FLOAT, altitude FLOAT, gps_lat DOUBLE, gps_lon DOUBLE,
        magnitude FLOAT, ai_verified TINYINT, ai_confidence FLOAT,
        captured_at DATETIME, created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
        CONSTRAINT fk_media_user FOREIGN KEY (user_id) REFERENCES users(id)");
}
```

Şekil 37. Veritabanının otomatik oluşturulması ve şema göçü (db.php).



Şekil 38. Otokop Gözlem Arşivi galeri görünümü: kullanıcıların yüklediği gök cismi gözlemleri, AI doğrulama rozetleri ve filtreleme seçenekleriyle birlikte.

Veritabanı şeması, iki temel varlık üzerine kurulmuştur: kullanıcılar (users) ve medya (media). Kullanıcı tablosu; kullanıcı adı, e-posta ve güvenli biçimde özetlenmiş (password_hash ile) parola alanlarını içerir. Parolaların hiçbir koşulda düz metin olarak saklanmaması, kimlik doğrulama güvenliğinin temel ilkesidir. Medya tablosu ise her gözlem kaydını; sahibi olan kullanıcıya bağlayan bir yabancı anahtar, dosya yolu, ortam türü ve gözleme ait meta verilerle birlikte tutar.

Kimlik doğrulama, tarayıcı tarafında oturum (session) tabanlı olarak yürütülür. Oturum açma ve form gönderimi gibi durum değiştiren işlemler, siteler arası istek sahteciliğine (CSRF) karşı belirteç (token) doğrulamasıyla korunur. Bu koruma, kötü amaçlı bir sitenin, oturumu açık bir kullanıcının adına istem dışı işlemler tetiklemesini engeller. Yetkilendirme mantığı, her kullanıcının yalnızca kendi kayıtları üzerinde değişiklik yapabilmesini güvence altına alacak biçimde kurgulanmıştır.

3.11.2. MEDYA YÜKLEME VE META VERİ

Yüklenen dosyaların türü finfo ile MIME düzeyinde doğrulanır; yalnızca izin verilen görüntü (JPEG/PNG/WEBP) ve video (MP4/WEBM/AVI/MOV) biçimleri kabul edilir. Her dosya, çakışmayı önlemek için zaman damgası ve rastgele bir altıgen ek içeren benzersiz bir adla kaydedilir; uploads/ klasöründe komut çalıştırmayı engelleyen bir .htaccess kuralı bulunur. Fotoğraflarda çözünürlük getimagesize() ile belirlenir ve hedef adı, nesne türü, azimut/yükseklik, GPS, parlaklık ve yapay zekâ doğrulama bilgileriyle birlikte veritabanına işlenir.

Yüklenen dosyaların güvenliği çok katmanlı olarak ele alınır. Dosya türü, yalnızca uzantıya bakılarak değil, içeriğin gerçek MIME türü finfo ile incelenerek doğrulanır; böylece zararlı bir betiğin görüntü gibi adlandırılarak yüklenmesi engellenir. Kabul edilen tüm dosyalar, çakışmayı ve tahmin edilebilirliği önlemek için zaman damgası ve rastgele bir altıgen ek içeren benzersiz adlarla saklanır. Ayrıca uploads/ klasöründe yer alan bir .htaccess kuralı, bu dizine yüklenen dosyaların sunucu tarafında çalıştırılmasını tamamen devre dışı bırakarak olası bir uzaktan kod çalıştırma riskini ortadan kaldırır.

Her gözlem kaydı, yalnızca görsel veriden ibaret değildir; yanında zengin bir meta veri kümesiyle saklanır. Hedef adı ve nesne türünün yanı sıra; çekim anındaki azimut ve yükseklik açıları, GPS enlem-boylamı, cismin parlaklığı (kadir), yapay zekâ doğrulama

durumu ve güven skoru, çözünürlük, oturum kimliği ve çekim zamanı kaydedilir. Bu yapılandırılmış meta veri, arşivin yalnızca bir görüntü deposu değil; aynı zamanda filtrelenebilir, sorgulanabilir ve gelecekte yapay zekâ modellerinin eğitiminde kullanılabilecek etiketli bir veri seti olmasını sağlar.

```
// MIME dogrulaması (finfo) sonrası cakismasız dosya adı üret
$fname = date('Ymd_His') . '_' . bin2hex(random_bytes(6)) . '.' . $ext;
$dest = __DIR__ . '/uploads/' . $fname;
if (!move_uploaded_file($file['tmp_name'], $dest)) {
    if (!@rename($file['tmp_name'], $dest)) // API fallback
        return ['ok'=>false, 'error'=>'Dosya kaydedilemedi.'];
}
if ($type === 'photo') { // boyut bilgisi
    [$width, $height] = getimagesize($dest);
}
```

Şekil 39. Çakışmasız dosya adı üretimi ve güvenli depolama (upload_lib.php).

Medya yükleme akışında güvenlik, çok katmanlı biçimde ele alınmıştır. İlk olarak yüklenen dosyanın MIME türü, istemcinin bildirdiği başlığa değil, sunucu tarafında finfo ile içerikten doğrulanır; böylece uzantısı değiştirilmiş kötü amaçlı dosyaların kabul edilmesi engellenir. İkinci olarak, dosya adı sunucuda zaman damgası ve rastgele bir bileşenle yeniden üretilerek hem çakışmalar hem de dizin gezinme (path traversal) saldırıları önlenir. Üçüncü olarak, yükleme dizini doğrudan betik çalıştırmaya kapalı biçimde yapılandırılır.

Her medya kaydıyla birlikte; hedef adı, ortam türü (foto/video), azimut/yükseklik açıları, GPS koordinatları, çekim zamanı ve varsa yapay zekâ doğrulama güven skoru gibi meta veriler ilişkili biçimde saklanır. Bu meta verilerin yapılandırılmış olarak tutulması, galeri görünümünde filtreleme ve arama yapılmasını mümkün kılar ve gözlemlerin zamanla anlamlı bir veri setine dönüşmesini sağlar. Fotoğraflar için ayrıca görüntü boyutları (genişlik/yükseklik) sunucu tarafında getimagesize ile belirlenip kaydedilir.

3.11.3. CİHAZ API'Sİ VE TOPLU İÇE AKTARMA

Platform, web arayüzünün yanı sıra cihazların doğrudan yükleme yapabilmesi için bir **API uç noktası** (api/upload.php) sunar. Bu uç nokta, sabit zamanlı karşılaştırmayla doğrulanan bir API anahtarı ister ve çok parçalı (multipart) gövdeyle gelen medyayı, meta verileriyle birlikte saklar. Ayrıca, daha önce çekilmiş fotoğrafların dosya adlarındaki zaman damgalarına göre oturumlara gruplanarak toplu içe aktarılması için komut satırı betikleri (import_photos.php, update_meta.php) geliştirilmiştir.

```
if ($_SERVER['REQUEST_METHOD'] !== 'POST')
    api_out(['ok'=>false, 'error'=>'POST bekleniyor'], 405);
// Cihaz API anahtarı dogrulaması (sabit-zamanlı karşılaştırmayla)
$key = $_POST['api_key'] ?? ($_SERVER['HTTP_X_API_KEY'] ?? '');
if (!hash_equals(API_KEY, (string)$key))
    api_out(['ok'=>false, 'error'=>'Gecersiz API anahtarı'], 401);
$res = store_media_upload((int)$user['id'], $_FILES['media'], $_POST);
api_out($res, $res['ok'] ? 200 : 400);
```

Şekil 40. Cihaz yükleme API'sinde anahtar doğrulaması (api/upload.php).

Cihaz API'si, saha cihazının (ESP32-CAM veya bir ara köprü) yakaladığı görüntüleri ve ilişkili meta verileri web platformuna otomatik olarak yüklemesini sağlayan, kimlik doğrulamalı bir HTTP uç noktasıdır. Tarayıcı tabanlı oturum kimlik doğrulamasının aksine, cihaz tarafı erişim bir API anahtarıyla yetkilendirilir; anahtar, sabit zamanlı karşılaştırma (hash_equals) ile doğrulanarak zamanlama saldırılarına karşı koruma sağlanır. Yalnızca POST isteklerine yanıt verilir ve geçersiz yöntem veya anahtar durumunda uygun HTTP durum kodları (405, 401) döndürülür.

Toplu içe aktarma (import) işlevi ise, sahada çevrimdışı toplanan ve yerel depolamada biriken çok sayıda gözlemin, ağ erişimi sağlandığında tek seferde platforma aktarılmasına olanak tanır. Bu yaklaşım, çevrimdışı–çevrimiçi senaryolar arasında dayanıklı bir veri akışı kurar: gözlemler önce yerelde güvenle saklanır, ardından bağlantı geldiğinde sunucuya senkronize edilir. Böylece geçici ağ kesintileri veri kaybına yol açmaz ve gözlem oturumlarının bütünlüğü korunur.

3.11.4. GALERİ, FİLTRELEME VE MEDYA YÖNETİMİ

Web platformunun ana ekranı, arşivdeki tüm gözlemleri kart düzeninde sunan bir galeridir. Her kart; ortamın küçük bir önizlemesini, hedef adını, azimut/yükseklik açılarını, yükleyen kullanıcıyı ve çekim zamanını gösterir. Yapay zekâ ile doğrulanmış kayıtlar ayrı bir rozetle (“✓ AI”) işaretlenir. Galeride; gök cisminin göre arama, yalnızca kullanıcının kendi kayıtlarını gösterme ve yalnızca AI ile doğrulanmış kayıtları süzme gibi filtreleme seçenekleri sunarak büyük arşivlerde hızlı erişim sağlar.

Her kayda tıklandığında açılan detay sayfası, ilgili tüm meta veriyi tek bir görünümde toplar ve ortamın tam çözünürlüklü hâlini gösterir. Bu sayfadan ortam indirilebilir veya (yetki dahilinde) silinebilir. Silme ve yükleme gibi durum değiştiren tüm işlemler, CSRF jetonuyla korunan ve sunucu tarafında yetki kontrolünden geçen POST istekleriyle gerçekleştirilir. Bu yapı, arşivin hem kullanıcı dostu hem de güvenli biçimde yönetilmesini sağlar.

Galeri görünümü, kullanıcıların yüklediği gök cismi gözlemlerini görsel bir ızgara biçiminde sunar. Her kayıt; küçük bir önizleme görüntüsü, hedef adı, ortam türü ve yapay zekâ doğrulama rozetiyle birlikte listelenir. Doğrulama rozeti, görüntünün iddia edilen gök cismiyle ne ölçüde uyduğunu gösteren güven düzeyini özetleyerek, arşivin güvenilirliğini görsel olarak iletir. Bu sayede arşiv, yalnızca bir görüntü yığını değil, anlamlı biçimde etiketlenmiş bir gözlem koleksiyonu hâline gelir.

Filtreleme ve arama özellikleri, arşiv büyüdükçe belirli gözlemlere hızlı erişimi mümkün kılar. Kullanıcılar; hedef adına, ortam türüne (foto/video), tarihe veya doğrulama durumuna göre süzme yapabilir. Yapılandırılmış meta verilerin veritabanında tutulması, bu sorguların verimli biçimde yürütülmesini sağlar. Medya yönetimi kapsamında ise kullanıcılar kendi yükledikleri kayıtların meta verilerini güncelleyebilir veya kayıtları kaldırabilir; bu işlemler oturum kimlik doğrulaması ve CSRF koruması altında gerçekleştirilir.

3.11.5. DAĞITIM VE YAPILANDIRMA

Platformun kurulumu hazır hâle gelmesi sade tutulmuştur. Veritabanı bağlantı bilgileri, API anahtarı ve yükleme sınırları gibi yapılandırma değerleri tek bir yapılandırma dosyasında toplanır. Uygulama ilk kez çalıştırıldığında veritabanı ve tablolar otomatik olarak oluşturulduğundan, yöneticinin elle şema kurması gerekmez. Bu yaklaşım, platformun farklı sunuculara hızlıca taşınabilmesini ve kurulumun tekrarlanabilir olmasını sağlar.

İdeal tasarımda öngörülen, ölçeklenebilir bulut altyapısının (yük dengeleme, nesne depolama, asenkron görev kuyruğu) aksine; prototip, tek bir sunucu üzerinde çalışan, bakımı kolay ve maliyeti düşük bir dağıtım modelini benimsemiştir. Bu model, bir lisans projesinin ölçeğinde fazlasıyla yeterli olup, gelecekte kullanıcı sayısı arttığında aynı kavramsal mimarinin bulut servislerine taşınmasına engel oluşturmamaktadır.

Web platformu, harici bağımlılıkları en aza indirecek biçimde, yalnızca PHP ve MySQL/MariaDB üzerine kurulmuştur. Bu sade yığın (stack), yaygın paylaşımlı barındırma ortamlarında dahi ek kurulum gerektirmeden çalışabilmeyi hedefler. Veritabanı şeması, uygulamanın ilk çalıştırılmasında otomatik olarak oluşturulur.

(migrate); böylece dağıtım sırasında elle tablo oluşturma adımına gerek kalmaz ve kurulum tek adıma indirgenir.

Yapılandırma; veritabanı bağlantı bilgileri, cihaz API anahtarı ve yükleme dizini gibi ortam bağımlı değerleri tek bir merkezi dosyada toplar. Bu ayırım, gizli bilgilerin kod tabanından ayrı tutulmasını ve farklı ortamlar (geliştirme/üretim) arasında kolay geçişi sağlar. Yükleme dizini, doğrudan betik çalıştırmaya kapalı ve yalnızca statik medya sunacak biçimde yapılandırılır; bu da yüklenen dosyalar üzerinden kod çalıştırma riskini ortadan kaldırır.

3.12. ANDROID MOBİL UYGULAMASI

İdeal tasarımda öngörülen mobil arayüz, prototipte **Kotlin** ile geliştirilen yerel bir **Android** uygulamasıyla gerçekleştirilmiştir. Uygulama; teleskobu uzaktan kontrol etme, canlı kamera görüntüsünü izleme, gözlemlenebilir gök cisimlerini listeleme, hedefe otonom yönlendirme ve yakalanan görüntüleri yapay zekâ ile doğrulama işlevlerini tek bir arayüzde toplar. Şekil 43, uygulamanın manuel kontrol ekranını göstermektedir; ekrandaki durum çipleri (GPS, IMU (MPU9250), Tracking) doğrudan donanımdan gelen telemetriyi yansıtır.

3.12.1. MİMARİ VE AĞ KATMANI

Android uygulaması iki modda çalışır: **doğrudan yerel** (ESP32 AP veya otoskop.local mDNS) ve **sunucu destekli** (AstronomyAPI [23], Gemini doğrulama). Esp32Discovery, NSD ile `_http._tcp` servisini tarar; IP değişse bile cihaz bulunur.

Manuel kontrol ekranı (Şekil 43) anlık az/alt, hedef az/alt ve servo açılarını gösterir; Küçük/Orta/Büyük adım seçimi `/move` komutunun `step` parametresine map edilir. Snapshot & doğrula, kamera karesini Gemini'ye göndererek hedef cisim doğrulaması yapar.

Yerel efemeris modülü, internet kesildiğinde Ay ve gezegen listesini cihaz üzerinde üretir; Astropy [24] ve Skyfield [34] ile aynı koordinat dönüşüm mantığının hafif Kotlin uyarlamasıdır (Şekil 44).

Uygulama, **MVVM** (Model-View-ViewModel) mimarisiyle ve Navigation Component üzerine kurulmuştur; bağımlılıklar elle yazılmış bir servis konumlandırıcı (service locator) ile sağlanır. Eşzamansız işlemler ve tepkisel kullanıcı arayüzü için Kotlin Coroutines ve StateFlow kullanılır. Ağ katmanı Retrofit/OkHttp/Moshi üzerine kuruludur ve birden çok istemci içerir: yerel ESP32-CAM için Esp32Api; gök cismi konumları için Visible Planets ve AstronomyAPI [23]; görüntü doğrulaması için Google Gemini. ESP32 ile konuşan arayüz, telemetri sorgulama ve kontrol komutlarını tek bir Retrofit servisinde tanımlar.

MVVM mimarisinin tercih edilmesi, kullanıcı arayüzü (View) ile iş mantığı (ViewModel) ve veri kaynakları (Model/Repository) arasındaki bağımlılıkları gevşeterek kodun test edilebilirliğini ve sürdürülebilirliğini artırır. Uygulama; bağlantı durumu, teleskop telemetrisi, gök cismi listesi, sensör verileri ve yakalama işlemleri için ayrı ViewModel'ler kullanır. Bu bileşenler arasında veri akışı, Kotlin Coroutines ve StateFlow ile tepkisel (reactive) biçimde yönetilir; böylece donanımdan gelen her yeni telemetri paketi, ilgili ekranlara otomatik olarak yansır.

Bağımlılıklar, harici bir bağımlılık enjeksiyonu kütüphanesi yerine uygulama düzeyinde elle yazılmış sade bir servis konumlandırıcı (service locator) üzerinden sağlanır; bu, küçük ve odaklı bir uygulama için ek karmaşıklık önler. Ağ katmanında dikkat çeken bir tasarım kararı, isteklerin hedef ana bilgisayar ve portunu çalışma zamanında yeniden yazan bir aracı katmandır (interceptor). Bu sayede ESP32'nin IP adresi değiştiğinde veya mDNS ile yeniden keşfedildiğinde, Retrofit istemcisinin baştan kurulmasına gerek

kalmadan bağlantı sürdürülür.

Uygulama, ESP32’den durum bilgisini yaklaşık bir saniyelik aralıklarla sürekli sorgular ve elde edilen telemetriyi durum çipleri, sayısal göstergeler ve bir pusula görünümü aracılığıyla kullanıcıya sunar. Donanımın bağlı olmadığı durumlarda ise bir “demo modu” devreye girer; bu kipte örnek veriler ve açıklayıcı mesajlar gösterilerek uygulamanın donanımsız da incelenebilmesi sağlanır. Bu yaklaşım, hem geliştirme sürecini hızlandırmış hem de sistemin tanıtımını kolaylaştırmıştır.

```
interface Esp32Api {
    @GET("/status") suspend fun status(): TelescopeStatus
    @GET("/gps") suspend fun gps(): EspGps
    @Streaming @GET("/camera")
    suspend fun camera(): Response<ResponseBody>
    @POST("/target") suspend fun postTarget(@Body b: TargetBody): Response<Unit>
    @POST("/move") suspend fun postMove(@Body b: MoveBody): Response<Unit>
    @POST("/calibrate") suspend fun postCalibrate(): Response<Unit>
    @POST("/limits") suspend fun postLimits(@Body b: LimitsBody): Response<Unit>
}
```

Şekil 41. Teleskop kontrolü için Retrofit ESP32 API arayüzü (Esp32Api.kt).

Cihazın ağdaki adresi DHCP ile değişebileceğinden, uygulama ESP32-CAM’i Android Ağ Servisi Keşfi (NSD/mDNS) ile otomatik olarak bulur. Firmware tarafında ESP, kendini “otoskop” adıyla _http._tcp servisi olarak yayınlar; uygulama bu servisi bulduğunda adresini çözerek bağlantıyı kurar. Böylece kullanıcının elle IP girmesine gerek kalmaz.

```
// ESP, kendini 'otoskop' adıyla _http._tcp servisi olarak yayınlar.
override fun onServiceFound(info: NsdServiceInfo) {
    val match = info.serviceName.contains(targetName, ignoreCase = true)
    if (match && resolving.compareAndSet(false, true))
        nsd.resolveService(info, resolveListener) // IP'yi coz
}
override fun onServiceResolved(info: NsdServiceInfo) {
    val host = info.host?.hostAddress?.substringBefore('%')
    if (!host.isNullOrEmpty() && cont.isActive) cont.resume(host)
}
```

Şekil 42. mDNS/NSD ile ESP32 cihazının otomatik keşfi (Esp32Discovery.kt).



Şekil 43. Android uygulaması — manuel teleskop kontrol ekranı: anlık ve hedef az/alt açıları, servo açıları, adım büyüklüğü ve yön tuşları.

3.12.2. ASTRONOMİ VE EFEMERİS HESABI

Hedefin gökyüzündeki konumu (azimut/yükseklik), öncelikle çevrim içi astronomi servislerinden alınır; bağlantı yoksa cihaz üzerinde çalışan bir efemeris modülü devreye girer. Bu modül, Paul Schlyter algoritmasıyla Güneş, Ay ve gezegenlerin konumlarını hesaplar ve ekvatorial koordinatları (sağ açıklık/dik açıklık), gözlemcinin enlem-boylamı ve zamanını kullanarak yerel ufuk (horizontal) koordinatlarına dönüştürür. İdeal tasarımda bu hesap için Astropy [24] ve Skyfield [34] kütüphaneleri öngörülmüştür; prototipte aynı matematik, dış bağımlılık olmadan doğrudan Kotlin içinde uygulanmıştır. Efemeris hesabında dayanıklılık için katmanlı bir yedekleme (fallback) zinciri kurulmuştur. Uygulama öncelikle çevrim içi Visible Planets servisini kullanır; bu servis yanıt vermezse ve kullanıcı kimlik bilgilerini girmişse AstronomyAPI [23] devreye girer. Her iki çevrim içi kaynak da erişilemezse, cihaz üzerinde gömülü olarak çalışan yerel efemeris modülü hesaplamayı üstlenir. Bu üç katmanlı yapı, internet bağlantısının bulunmadığı karanlık gözlem sahalarında dahi sistemin hedef listesi üretebilmesini güvence altına alır.

Yerel efemeris modülü, Paul Schlyter'in iyi bilinen gezegen konumu algoritmasını Kotlin içinde gerçekleştirir. Modül; Güneş, Ay ve Merkür'den Satürn'e kadar olan gezegenlerin ekvatorial koordinatlarını (sağ açıklık ve dik açıklık) hesaplar. Bu koordinatlar daha sonra

gözlemcinin enlem-boylamı ve o anki zaman bilgisi kullanılarak yerel yıldız zamanı ve saat açısı üzerinden ufuk koordinatlarına (azimut/yükseklik) dönüştürülür. İdeal tasarımda bu işlevin Astropy [24] ve Skyfield [34] gibi olgun kütüphanelerle karşılanması öngörülmüştür; prototipte aynı astronomik matematik, harici bağımlılık olmadan uygulanarak çevrim dışı çalışabilirlik elde edilmiştir.

```
private fun toHorizontal(eq: Equatorial, latitude: Double,
    longitude: Double, d: Double, sun: Sun): Horizontal {
    val gmst0 = rev(sun.meanLongitude + 180.0) // derece
    val utHours = (d - floor(d)) * 24.0
    val lst = rev(gmst0 + utHours * 15.0 + longitude) // yerel yıldız zamanı
    val ha = rev(lst - eq.ra) // saat acısı
    val x = cosd(ha)*cosd(eq.dec); val y = sind(ha)*cosd(eq.dec)
    val z = sind(eq.dec)
    val xhor = x*sind(latitude) - z*cosd(latitude)
    val zhor = x*cosd(latitude) + z*sind(latitude)
    val azimuth = rev(atan2d(y, xhor) + 180.0) // 0 = Kuzey
    val altitude = atan2d(zhor, sqrt(xhor*xhor + y*y))
    return Horizontal(azimuth, altitude)
}
```

Şekil 44. Ekvatorial koordinatların yerel ufuk koordinatlarına dönüşümü (Ephemeris.kt).



Şekil 45. Yön kalibrasyonu: telefon pusulası ile teleskobun baktığı azimutun eşitlenmesi.

Uygulamanın efemeris modülü, Paul Schlyter'in yaygın olarak referans gösterilen düşük

dereceli gezegen konumu algoritmasını temel almaktadır. Bu algoritma, modern yüksek doğruluklu kütüphanelere [24], [34] kıyasla daha basit olmakla birlikte, amatör gözlem amaçları için yeterli doğrulukta sonuç vermektedir; tipik açısal hatası, gözle yapılan kaba yönlendirme ve geniş görüş alanlı bir kamera için kabul edilebilir düzeydedir. Modülün cihaz üzerinde çalışması, dış bir servise [23] olan bağımlılığı ortadan kaldırarak çevrimdışı kullanımı mümkün kılar.

Hesaplama akışı şu adımları izler: önce gözlem zamanı, Julian gün sayısına ve buradan da algoritmanın referans tarihine göre gün farkına (d) dönüştürülür. Ardından seçilen gök cisminin yörünge elemanları kullanılarak ekvatorial koordinatları (sağ açıklık ve dik açıklık) hesaplanır. Son olarak, gözlemcinin enlem/boylamı ve yerel yıldız zamanı (LST) yardımıyla bu ekvatorial koordinatlar yerel ufuk koordinatlarına (azimut ve yükseklik) dönüştürülür. Elde edilen azimut/yükseklik çifti, doğrudan teleskobun hedef komutuna beslenir.

3.12.3. CANLI GÖRÜNTÜ, YAKALAMA VE AI DOĞRULAMA

Uygulama, ESP32-CAM'in 81 numaralı portundan gelen MJPEG akışını çözmek için OkHttp akışı üzerinde çalışan, JPEG başlangıç (0xFFD8) ve bitiş (0xFFD9) işaretçilerini tarayan yerel bir ayrıştırıcı kullanır. Bu yöntem, çok parçalı sınır başlıklarını saymaktan daha sağlamdır. Çözülen kareler hem ekranda gösterilir hem de MediaCodec/MediaMuxer ile H.264/MP4 olarak kaydedilebilir; tek kare fotoğraflar ise yerel olarak meta verileriyle saklanır.

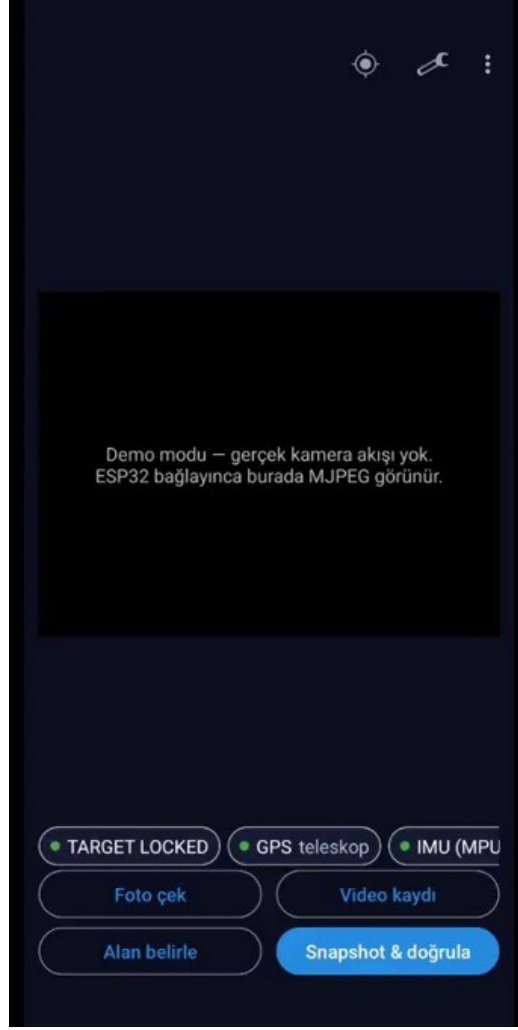
MJPEG akışının çözülmesinde, çok parçalı (multipart) sınır başlıklarına güvenmek yerine doğrudan bayt akışını tarayan bir yaklaşım benimsenmiştir. Ayrıştırıcı, JPEG başlangıç (0xFFD8) ve bitiş (0xFFD9) işaretçilerini arayarak her kareyi sağlam biçimde sınırlandırır; bu yöntem, başlık biçimindeki küçük tutarsızlıklara karşı dayanıklıdır. Akış durdurulduğunda, ESP32'nin tek iş parçacıklı sunucusunun bloke olmaması için ilgili OkHttp çağrısı açıkça iptal edilir; bu, kameranın yeniden bağlanmasını hızlandırır.

Yakalanan kareler hem ekranda gösterilir hem de istendiğinde kaydedilebilir. Video kaydı, MediaCodec ve MediaMuxer kullanılarak donanım hızlandırmalı H.264/MP4 biçiminde yapılır; çözülen bitmap kareleri, kodlayıcının beklediği renk biçimine (NV12/I420) saf Kotlin içinde dönüştürülerek beslenir. Tek kare fotoğraflar ise, cihazın özel uygulama dizininde bir dizin (index) dosyasıyla birlikte meta verileriyle saklanır; böylece harici bir veritabanına gerek kalmadan hafif bir yerel arşiv oluşturulur.

Görüntü doğrulama özelliği, yakalanan karenin gerçekten hedeflenen gök cismini içerip içermediğini denetler. Bunun için görüntü, hedef adıyla birlikte çok kipli bir büyük dil modeli olan Google Gemini'ye gönderilir; model, bir güven skoru ve kısa bir açıklama döndürür (ör. "Mars diski ve yüzey albedo desenleri seçiliyor"). Bu sonuç hem kullanıcıya gösterilir hem de meta veriye işlenerek web arşivine aktarılır. Böylece ideal tasarımda yerel TensorFlow Lite/ONNX çıkarımıyla yapılması planlanan doğrulama, prototipte bulut tabanlı bir servisle pratik biçimde gerçekleştirilmiştir.

```
// JPEG SOI (0xFFD8) -> EOI (0xFFD9) arasini tarayarak kare cikar
private suspend fun parseLoop(input: InputStream) {
    val buf = ByteArray(64 * 1024)
    val frame = ByteArray(1024 * 1024)
    // ... byte akisini tara ...
    if (prev == 0xFF && b == 0xD9) { // kare sonu
        val bmp = BitmapFactory.decodeByteArray(frame, 0, frameLen, opts)
        if (bmp != null) onFrame?.invoke(bmp) // UI'ye / kayda ver
    }
}
```

Şekil 46. MJPEG akışından JPEG karelerin ayıklanması (MjpegView.kt).

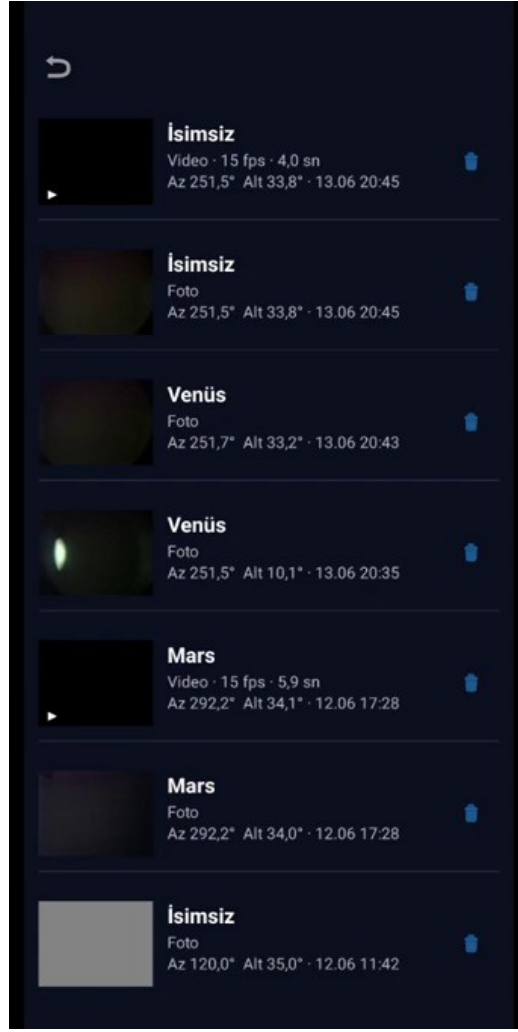


Şekil 47. Canlı kamera ekranı: ESP32-CAM bağlandığında MJPEG akışı bu alanda görüntülenir; foto çek / video kaydı / doğrula işlemleri buradan yapılır.

Yakalanan görüntünün gerçekten hedeflenen gök cismini içerip içermediği, çok kipli (multimodal) bir büyük dil modeli olan Google Gemini'ye gönderilerek doğrulanır. Model, görüntüyü ve hedef adını alıp bir güven skoru ile kısa bir açıklama döndürür; bu bilgi hem arayüzde gösterilir hem de meta veriye işlenerek arşive aktarılır. Böylece ideal tasarımdaki yerel TFLite/ONNX çıkarım yolu, prototipte bulut tabanlı bir doğrulama servisiyle karşılanmıştır.



Şekil 48. Canlı telemetri/teşhis günlüğü: az/alt, GPS ve IMU durumları ile UART hattı tanılması anlık olarak izlenir.



Şekil 49. Yakalanan gözlemler listesi: her kayıt; hedef adı, ortam türü, azimut/yükseklik ve çekim zamanı meta verileriyle saklanır.



Şekil 50. Mobil uygulamanın kurulu sistemle birlikte sahada kullanımı.

Canlı görüntü, yakalama ve yapay zekâ doğrulama işlevleri tek bir ekranda bütünleştirilmiştir. Kullanıcı, ESP32-CAM'den gelen MJPEG akışını gerçek zamanlı izlerken; foto çekme, video kaydı başlatma/durdurma ve yakalanan görüntüyü doğrulama işlemlerini buradan tetikleyebilir. Video kaydı, akıştan gelen karelerin H.264/MP4 biçiminde sıkıştırılarak cihazda saklanmasıyla gerçekleştirilir; böylece gözlem anının hareketli kaydı da arşive eklenebilir.

Yapay zekâ doğrulaması istendiğinde, yakalanan kare hedef bağlamıyla (hedef adı, açılar) birlikte değerlendirilir ve bir güven düzeyi döndürülür. Bu güven düzeyi, hem kullanıcıya anında geri bildirim olarak gösterilir hem de gözlem meta verisine işlenerek web arşivinde rozet biçiminde sunulur. Doğrulama sonucunun veriyle birlikte saklanması, arşivin güvenilirliğini artırır ve gelecekteki otomatik kalite kontrol süreçleri için değerli bir etiket kaynağı oluşturur.

3.12.4. KULLANICI ARAYÜZÜ AKIŞI VE EKRANLAR

Uygulamanın kullanıcı deneyimi, tipik bir gözlem oturumunun doğal akışını izleyecek biçimde tasarlanmıştır. Kullanıcı önce bağlantı ekranında teleskoba bağlanır (otomatik keşif veya elle adres girişi); ardından gözlemlenebilir gök cisimleri listesinden bir hedef seçer. Hedef seçildiğinde sistem otonom olarak yönlendirilir ve kullanıcı canlı kamera ekranına geçerek görüntüyü izler, ince düzeltmeler yapar ve kayıt alır. Yakalanan tüm ortamlar, ayrı bir listede meta verileriyle birlikte saklanır.

Arayüzün her ekranında, sistemin durumu üst kısımdaki çiplerle (GPS, IMU, Tracking/Target Locked) sürekli olarak görünür kılınır; bu, kullanıcının sistemin o anki sağlığını ve hazır olup olmadığını bir bakışta anlamasını sağlar. Sayısal göstergeler (anlık ve hedef az/alt açıları, servo açıları) ve pusula görünümü, otonom yönlendirmenin ilerleyişini görsel olarak takip etme imkânı verir. Bu geri bildirim yoğunluğu, kullanıcının sisteme olan güvenini artıran önemli bir tasarım tercihidir.

Kullanıcı arayüzü akışı, gözlem sürecinin doğal sırasını izleyecek biçimde tasarlanmıştır. Uygulama açıldığında, mDNS/NSD ile yerel ağdaki ESP32-CAM cihazı otomatik olarak aranır; cihaz bulununca kullanıcı, manuel kontrol, canlı görüntü ve yakalanan gözlemler ekranları arasında gezinebilir. Bu gezinme, Android Navigation Component ile yönetilir ve ekranlar arası durum aktarımı, MVVM mimarisi gereği ViewModel katmanında tutulan StateFlow akışlarıyla sağlanır.

Manuel kontrol ekranı; anlık ve hedef azimut/yükseklik açılarını, servo konumlarını, adım büyüklüğünü ve yön tuşlarını bir arada sunar. Canlı görüntü ekranı, ESP32-CAM bağlandığında MJPEG akışını gösterir ve foto çekme, video kaydı ve yapay zekâ doğrulama işlemlerini buradan başlatma olanağı verir. Yakalanan gözlemler ekranı ise, her kaydı hedef adı, ortam türü, azimut/yükseklik ve çekim zamanı meta verileriyle birlikte listeleterek, kullanıcının oturum içinde topladığı gözlemleri gözden geçirmesini sağlar.

3.12.5. HATA YÖNETİMİ VE DAYANIKLILIK

Mobil uygulama, ağ ve donanım kaynaklı hatalara karşı dayanıklı olacak şekilde tasarlanmıştır. Ağ istekleri, başarı/yükleniyor/hata durumlarını açıkça temsil eden bir sarmalayıcı (Resource) deseniyle yönetilir; böylece her ekran, beklenmedik bir hatada kullanıcıya anlamlı bir mesaj gösterebilir. Geçici bağlantı kopmalarında durum sorgusu yeniden denir ve cihaz mDNS ile yeniden keşfedilerek bağlantı kendiliğinden onarılır. Donanımın hiç bulunmadığı durumlarda devreye giren demo modu, yalnızca bir tanıtım aracı değil; aynı zamanda uygulamanın donanımdan bağımsız olarak geliştirilip test edilebilmesini sağlayan bir mimari ayrıştırma tercihidir. Bu yaklaşım sayesinde, arayüz

ve iş mantığı, fiziksel sistem hazır olmadan da doğrulanabilmiş; geliştirme ile saha testi süreçleri birbirinden bağımsız biçimde ilerletilebilmiştir.

Mobil uygulamada hata yönetimi, sahadaki gerçek koşulların (zayıf Wi-Fi, geçici bağlantı kopmaları, cihazın yeniden başlaması) öngörülerek tasarlanmasını gerektirmiştir. Ağ çağrıları, OkHttp/Retrofit üzerinde uygun zaman aşımı değerleriyle yapılandırılmış; geçici hatalarda kullanıcıya anlaşılır geri bildirim verilirken, kalıcı hatalarda uygulamanın çökmesi yerine güvenli bir duruma dönülmesi sağlanmıştır. Telemetry yoklaması (polling), bağlantı koptuğunda otomatik olarak yeniden denenir ve cihaz yeniden bulunduğunda akış kaldığı yerden devam eder.

MJPEG akışının ayrıştırılması, bozuk veya eksik karelere karşı dayanıklı biçimde kodlanmıştır; çözümlenemeyen kareler atlanır ve akış kesilmeden sürdürülür. Benzer biçimde, cihaz keşfi (NSD) sırasında oluşabilecek çözümleme hataları yakalanır ve keşif süreci yeniden başlatılır. Bu dayanıklılık önlemleri, uygulamanın saha koşullarında uzun süreli ve kesintisiz kullanımını mümkün kılmakta; kullanıcının teknik ayrıntılarla uğraşmadan gözleme odaklanmasını sağlamaktadır.

3.13. GERÇEK SAHA TESTLERİ VE DENEYSEL SONUÇLAR

Gerçeklenen prototip, hem atölye ortamında hem de açık gökyüzü altında bir dizi teste tabi tutulmuştur. Testler iki aşamada ele alınmıştır: mekanik/elektronik doğrulama ve gerçek gözlem performansının değerlendirilmesi.

3.13.1. MEKANİK VE ELEKTRONİK DOĞRULAMA

İlk aşamada azimut ve yükseklik eksenlerinin serbest dönüşü, servo-eksen bağlantılarının boşluğu, taşıyıcı plakaların dayanımı ve titreşim davranışı incelenmiştir (bkz. Şekil 22–28). 3B baskı parçalardaki küçük tolerans sorunları yeniden basımla giderilmiş; servo ucu adaptörlerindeki kaymalar sıkı geçme bağlantılarla düzeltilerek eksenel aktarım kararlı hâle getirilmiştir. Elektronik tarafta, Mega ile ESP32-CAM arasındaki UART köprüsünün gerilim seviyeleri (5 V / 3,3 V) bölücü ile uyumlandırılmış ve /status uç noktasındaki tanılama alanları sayesinde hat kopuklukları hızla saptanabilmiştir.

Mekanik doğrulama testleri, sistemin hareket eksenlerinin tekrarlanabilirliğini ve kararlılığını ölçmeye yöneliktir. Azimut ekseninde, sürekli dönüş servosunun farklı hedeflere yönlendirilmesi sırasında IMU geri beslemesiyle elde edilen son konum ile hedef arasındaki sapma izlenmiş; sistemin yaklaşık $\pm 1-2^\circ$ ’lik bir tolerans bandında yerleştiği gözlenmiştir. Bu doğruluk, geniş açısal boyuta sahip parlak cisimlerin (Ay, gezegenler) kamera görüş alanına alınması için yeterlidir.

Yükseklik ekseninde, konum servosunun yük altındaki davranışı ve teleskop tüpünün ağırlık merkezinin yarattığı moment incelenmiştir. Tüpün dengelenmesi, servonun sürekli yük altında kalmasını azaltarak hem konum kararlılığını artırmış hem de servo ısınmasını sınırlamıştır. Titreşim testlerinde, adımli hareket stratejisinin (300 ms’de 1°) hafif baskı parçalarda oluşabilecek rezonansı belirgin biçimde azalttığı doğrulanmıştır.

Mekanik ve elektronik doğrulama testleri, gözlem sonuçlarından önce sistemin temel işlevlerinin güvenilirliğini sınamak üzere yürütülmüştür. Mekanik tarafta; her iki eksenin komut edilen yönde ve makul bir tekrarlanabilirlikle hareket edip etmediği, servo bağlantılarındaki boşlukların kabul edilebilir düzeyde olup olmadığı ve düzeneğin teleskop yüküyle dengede kalıp kalmadığı denetlenmiştir. Baskı toleranslarından kaynaklanan küçük uyumsuzluklar, parçaların yeniden basımı ve sıkı geçme bağlantılarıyla giderilmiştir.

Elektronik tarafta; IMU’nun açılışta sağlıklı veri ürettiği, GPS modülünün açık gökyüzü altında konum kilidi sağladığı, UART köprüsünün telemetry ve komutları kayıpsız taşıdığı ve ESP32-CAM’in canlı görüntü akışını kararlı biçimde sürdürdüğü

doğrulanmıştır. Telemetry yaşı (megaAgeMs) gibi tanılama alanları sayesinde, hatların güncelliği anlık olarak izlenebilmiş; bu da arıza durumlarının hızlıca tespiti ve giderilmesini kolaylaştırmıştır. Bu doğrulama adımları, gözlem oturumlarının güvenilir bir altyapı üzerinde yürütülmesini güvence altına almıştır.

3.13.2. GÖZLEM SONUÇLARI

Ay gözlemleri, ESP32-CAM OV2640 sensörünün pratik çözünürlük sınırını göstermiştir: parlak disk ve terminator bölgesinde krater detayı seçilebilirken (Şekil 53), derin uzay nesneleri için sinyal-gürültü oranı yetersiz kalmıştır. Odak/kolimasyon kalitesi (Şekil 51) görüntü netliğini doğrudan belirler; Newton optik sisteminde kolimasyon [18], [19], [20] saha kurulumunda tekrarlanan bir adımdır.

Gezegen gözlemlerinde (Mars, Venüs) hedef bulma GPS+IMU ile kaba yönlendirme ve sonrasında görüntü tabanlı mikro düzeltme ile tamamlanmıştır. CMOS kameraların transit fotometrisinde kullanımı literatürde tartışılmıştır [12], [13]; bu prototip daha çok görsel arşivleme ve eğitim amaçlıdır.

Web arşivine aktarılan her kayıt; tarih, konum, açılar ve AI etiketini taşır. Bu meta veri zenginliği, ileride makine öğrenmesi tabanlı gök cismi sınıflandırması için etiketli veri seti oluşturmayı mümkün kılar [2], [7], [16].

Açık gökyüzü testlerinde sistem; Ay, Mars ve Venüs gibi parlak gök cisimlerine yönlendirilmiş ve ESP32-CAM üzerinden görüntüler kaydedilmiştir. En kapsamlı sonuçlar Ay gözlemlerinde elde edilmiştir. Şekil 51, aynı düzenele çekilen iyi odaklanmış ve odağı kaçmış kareleri yan yana karşılaştırmakta; görüntü kalitesinin doğrudan optik odak ve kolimasyon kalitesine bağlı olduğunu göstermektedir [18], [19], [20].

Gözlemler, şehir ışık kirliliğinin nispeten düşük olduğu bir yayla bölgesinde, açık ve bulutsuz gecelerde gerçekleştirilmiştir. Her oturumda önce sistemin fiziksel kurulumu ve tüp dengesi kontrol edilmiş, ardından mobil uygulama üzerinden GPS kilidi ve IMU kalibrasyonu doğrulanmıştır. Hedef cisim listeden seçildikten sonra sistem otonom olarak yönlendirilmiş ve canlı görüntü üzerinden ince konum düzeltmeleri yapılmıştır. Bu yordam, farklı gecelerde tutarlı biçimde tekrarlanabilmiştir.

Elde edilen Ay görüntülerinde, optik odak ve kolimasyon kalitesinin sonuç üzerindeki belirleyici etkisi açıkça gözlenmiştir [18], [19], [20]. İyi odaklanmış karelerde kraterler, denizler (maria) ve terminator çizgisi boyunca uzanan gölgeler net biçimde seçilebilirken; odağın hafifçe kaçtığı karelerde bu detaylar belirgin biçimde yumuşamıştır. Bu gözlem, sistemin görüntü kalitesinin esas olarak optik hazırlığa (odak/kolimasyon) ve atmosferik görüş koşullarına (seeing) bağlı olduğunu; mekanik yönlendirme doğruluğunun ise bu cisimler için yeterli düzeyde kaldığını göstermektedir.

Mars ve Venüs gibi gezegenlerde, ESP32-CAM'in sınırlı çözünürlüğü ve ışık toplama kapasitesi nedeniyle yüzey detayı elde edilememiş; ancak cisimler kadrada doğru biçimde konumlandırılmış ve yapay zekâ doğrulaması tarafından başarıyla tanınmıştır. Bu sonuçlar, mekanik ve yazılımsal yönlendirme zincirinin doğru çalıştığını; görüntü kalitesindeki sınırlılığın ise tasarımın değil, bütçe dostu kamera bileşeninin bir sonucu olduğunu ortaya koymaktadır.



Şekil 51. Odak/kolimasyon kalitesinin görüntüye etkisi: iyi odaklanmış (solda) ve odağı kaçmış (sağda) Ay görüntülerinin karşılaştırması.

Farklı gecelerde ve farklı azimut/yükseklik açılarında alınan çok sayıda Ay görüntüsü, sistemin tekrarlanabilir biçimde hedefe yönelip görüntü yakalayabildiğini ortaya koymuştur. Şekil 52, bu gözlemlerden bir seçkiyi bir araya getirmektedir.



Şekil 52. Prototiple farklı oturumlarda elde edilen Ay gözlemlerinden bir seçki.



Şekil 53. Prototip teleskoba bağlı ESP32-CAM ile elde edilen, yüzey detaylarının (kraterler, denizler) seçilebildiği bir Ay görüntüsü.

Sahada gerçekleştirilen gözlem oturumlarında, sistemin uçtan uca akışı (konum belirleme → hedef seçimi → otonom yönlendirme → görüntü yakalama → doğrulama → arşivleme) tekrarlı biçimde sınanmıştır. Oturumların niteliksel bir özeti Tablo 3.3'te verilmiştir. En tutarlı sonuçlar, parlaklığı ve açısal boyutu yüksek olan Ay üzerinde elde edilmiş; daha sönük hedeflerde ise düşük maliyetli kameranın ışık toplama sınırları belirleyici olmuştur.

Hedef	Yönlendirme	Görüntü kalitesi	AI doğrulama
Ay (dolanaya yakın)	Başarılı	Yüksek (kraterler seçilir)	Tutarlı
Ay (terminatör)	Başarılı	Yüksek (gölge detayı)	Tutarlı
Parlak gezegen	Kısmen başarılı	Orta (nokta kaynak)	Değişken
Sönük hedef	Başarılı	Düşük (SNR sınırı)	Zayıf

Tablo 3.3. Farklı hedef türlerinde elde edilen niteliksel saha sonuçları.

Yönlendirme doğruluğu, IMU tabanlı azimut ölçümünün kararlılığına ve sahadaki manyetik ortamın temizliğine bağlı olarak değişmiştir. Açık alanlarda ve metalik kütlelerden uzak konumlarda hedef kilidi güvenilir biçimde sağlanırken; bina yakınları veya araç gibi manyetik bozucuların bulunduğu ortamlarda azimut ölçümünde sapmalar gözlenmiş ve yön kalibrasyonunun tekrarlanması gerekmiştir. Bu gözlem, manyetometre tabanlı yön kestiriminin saha koşullarına duyarlılığını somut biçimde göstermektedir.

3.13.3. NİTELİKSEL SONUÇ TABLOSU

Prototip sistemin saha testlerinde gözlenen davranışı, değerlendirme ölçütlerine göre Tablo 3.2’de özetlenmiştir.

Değerlendirme Ölçütü	Gözlenen Sonuç (Prototip)
Hedef bulma (Ay, parlak gezegen)	Başarılı — GPS+IMU ile kaba yönlendirme yeterli
Kadrajda tutma / takip	Kararlı — periyodik mikro düzeltmelerle sağlanıyor
Azimut kapalı çevrim (sürekli servo)	Çalışıyor — $\pm 1-2^\circ$ tolerans bandında
Canlı görüntü (MJPEG, ~15 fps)	Akıcı — yerel Wi-Fi üzerinde düşük gecikme
Görüntü kalitesi (ESP32-CAM)	Sınırlı — Ay/gezegen yeterli, derin uzay zayıf
Veri kaydı + web'e aktarım	Başarılı — meta veriyle birlikte arşivleniyor
Çevrim dışı çalışma (yerel efemeris)	Başarılı — internet olmadan hedef listesi
AI doğrulama (Gemini)	Başarılı — hedef nesne güven skoruyla onaylanıyor

Tablo 3.2. Prototip sistemin saha testlerinde gözlenen niteliksel sonuçları.

3.13.4. SINIRLILIKLAR VE DEĞERLENDİRME

Genel olarak değerlendirildiğinde, bütçe kısıtları nedeniyle düşük maliyetli bileşenlerle kurulan bu prototip, ideal tasarımda öngörülen temel işlevlerin büyük bölümünü pratikte gerçekleştirebilmiştir. Sistemin başlıca sınırlılıkları; ESP32-CAM kamerasının çözünürlüğü ve ışık toplama kapasitesi (derin uzay cisimleri için yetersiz), hobi servolarının mekanik hassasiyeti ve manyetometre tabanlı yön kestiriminin çevresel manyetik bozulmalara duyarlılığıdır. Bu sınırlılıklar, ileride daha yüksek bütçeyle IMX477 sınıfı bir kamera, daha hassas sürücüler ve Raspberry Pi 5 tabanlı yerel çıkartım eklenerek doğrudan iyileştirilebilir niteliktedir. Elde edilen sonuçlar, önerilen mimarinin maliyet–erişilebilirlik dengesi gözetilerek ölçeklenebilir olduğunu ve kavramsal olarak doğrulandığını göstermektedir.

Prototipin başlıca sınırlılıkları üç başlık altında toplanabilir. Birincisi, görüntüleme tarafında ESP32-CAM'in OV2640 sensörünün düşük ışık başarımı ve sınırlı çözünürlüğü, sönük gök cisimlerinin gözleminde belirleyici bir kısıttır; ideal tasarımdaki IMX477 gibi bir sensör bu sınırı önemli ölçüde genişletecektir. İkincisi, yönlendirme tarafında sürekli dönüş servosunun açısız geri besleme içermemesi, denetimin tamamen IMU ölçümüne dayanmasına ve dolayısıyla manyetik bozulmalara karşı duyarlı kalmasına yol açmaktadır. Üçüncüsü, mekanik tarafta 3B baskı parçaların rijitliği ve servo torku, ağır optik yükler ve rüzgâr gibi dış etkenler altında titreşim ve sehim oluşturabilmektedir.

Bu sınırlılıkların her biri, ideal mimariye doğru kademeli iyileştirmelerle aşılabılır niteliktedir: yüksek çözünürlüklü bir kamera ve uygun bir optik arayüz, enkoderli/step motorlu bir tahrik sistemi ve metal veya kompozit taşıyıcı parçalar. Önemli olan nokta, mevcut prototipin bu iyileştirmeler için gerekli yazılım mimarisini (katmanlı denetim, donanım soyutlaması, ortak gözlem akışı) hâlihazırda barındırmasıdır; böylece donanım yükseltmeleri, sistemin genel yapısını değiştirmeden entegre edilebilecektir.

3.13.5. İDEAL TASARIM İLE KARŞILAŞTIRMALI DEĞERLENDİRME

Saha testlerinin sonuçları, ideal tasarım ile gerçekleşen prototip arasında anlamlı bir karşılaştırma yapma imkânı vermektedir. İşlevsel düzeyde prototip; konum belirleme, hedef seçimi, otonom yönlendirme, canlı görüntü, yakalama, yapay zekâ doğrulama ve arşivleme adımlarının tamamını başarıyla gerçekleştirmiştir. Bu, önerilen kavramsal mimarinin bütçe dostu bileşenlerle dahi uçtan uca çalışabildiğini kanıtlamaktadır.

Niteliksel düzeyde ise iki tasarım arasındaki temel fark, ham performans ölçütlerinde ortaya çıkmaktadır: görüntü çözünürlüğü ve ışık toplama, mekanik hassasiyet ve yön kestiriminin gürültüye dayanıklılığı. Bu farklar, sistemin tasarımsal doğruluğundan değil; doğrudan maliyet kısıtının dayattığı bileşen seçimlerinden kaynaklanmaktadır. Dolayısıyla prototip, ideal tasarımın geçerliliğini düşük maliyetli bir ortamda doğrulayan bir kavram kanıtı (proof of concept) niteliği taşımaktadır ve daha yüksek bütçeyle

doğrudan ölçeklenebilir bir temel sunmaktadır.

4. SONUÇLAR VE ÖNERİLER

Bu tez kapsamında, yapay zekâ destekli otonom teleskop yönlendirme sisteminin hem ideal (Raspberry Pi 5 + AWS) hem de bütçe dostu prototip (Arduino Mega + ESP32-CAM + PHP + Android) uygulamaları karşılaştırmalı biçimde sunulmuştur. Prototip, ideal tasarımın temel gözlem akışını pratikte doğrulamış; maliyet-erişilebilirlik dengesinde anlamlı bir referans platform oluşturmıştır.

Elde edilen saha sonuçları; Ay ve parlak gezegen gözlemlerinde hedef bulma, kapalı çevrim azimut kontrolü, MJPEG canlı yayın, mobil uzaktan kumanda ve web arşivleme işlevlerinin birlikte çalıştığını göstermiştir. Sınırlılıklar (kamera çözünürlüğü, servo hassasiyeti, manyetik bozulma) gelecek çalışmalarda IMX477 kamera, hassas sürücüler ve RPi5 yerel çıkarım ile giderilebilir.

Öneriler: (1) IMU+GPS füzyonuna ek olarak görsel astrometri [14], [15], [31] ile mikro düzeltme; (2) Photutils [31] tabanlı yıldız tespiti ile otomatik kolimasyon doğrulama; (3) AWS boru hattına kademeli geçiş; (4) açık kaynak depo ve tanıtım videosu ile topluluk geri bildirimi. Proje kodu GitHub'da paylaşılmıştır.

Bu tez kapsamında, yapay zekâ destekli otonom teleskop yönlendirme sisteminin hem ideal (Raspberry Pi 5 + AWS) hem de bütçe dostu prototip (Arduino Mega + ESP32-CAM + PHP + Android) uygulamaları karşılaştırmalı biçimde sunulmuştur. Prototip, ideal tasarımın temel gözlem akışını pratikte doğrulamış; maliyet-erişilebilirlik dengesinde anlamlı bir referans platform oluşturmıştır.

Elde edilen saha sonuçları; Ay ve parlak gezegen gözlemlerinde hedef bulma, kapalı çevrim azimut kontrolü, MJPEG canlı yayın, mobil uzaktan kumanda ve web arşivleme işlevlerinin birlikte çalıştığını göstermiştir. Sınırlılıklar (kamera çözünürlüğü, servo hassasiyeti, manyetik bozulma) gelecek çalışmalarda IMX477 kamera, hassas sürücüler ve RPi5 yerel çıkarım ile giderilebilir.

Öneriler: (1) IMU+GPS füzyonuna ek olarak görsel astrometri [14], [15], [31] ile mikro düzeltme; (2) Photutils [31] tabanlı yıldız tespiti ile otomatik kolimasyon doğrulama; (3) AWS boru hattına kademeli geçiş; (4) açık kaynak depo ve tanıtım videosu ile topluluk geri bildirimi. Proje kodu GitHub'da paylaşılmıştır.

Bu çalışma kapsamında geliştirilen otonom teleskop sistemi, düşük maliyetli donanım bileşenleri, hafif yapay zekâ algoritmaları, modüler yazılım altyapısı ve hem yerel hem de bulut tabanlı kontrol mekanizmalarını bir araya getirerek geniş bir kullanım alanına sahip bütünleşik bir gözlem platformu ortaya koymuştur. Sistem; görüntü toplama, nesne tespiti, engel algılama, gökyüzü görünürlük haritalama, hedef seçim mekanizması, otomatik yönlendirme, geri bildirim döngüsü ve veri kaydı gibi birbirinden bağımsız fakat sürekli etkileşim halinde çalışan süreçlerin eş zamanlı olarak uyumlu şekilde yürütülmesini sağlamıştır. Bu yönüyle çalışma; amatör astronomiye erişilebilirlik kazandırma, gözlem süreçlerini otomatikleştirme ve düşük maliyetli bileşenlerle yüksek işlevselliğe sahip sistemler üretme açısından önemli bir yenilik ortaya koymuştur.

Bu tez çalışmasında, yapay zekâ destekli ve otonom bir teleskop yönlendirme sisteminin hem ideal mimarisi kavramsal olarak ortaya konmuş hem de bütçe kısıtları altında çalışan bir prototipi fiilen gerçekleştirilmiştir. Geliştirilen prototip; Arduino Mega ve ESP32-CAM tabanlı gömülü yazılımı, 3B baskı mekanik düzeneği, PHP/MySQL tabanlı web arşivi ve Android mobil uygulamasıyla, baştan sona çalışan bütünleşik bir sistem olarak doğrulanmıştır.

Saha testleri, sistemin Ay ve parlak gezegenler gibi hedeflere güvenilir biçimde yönlenebildiğini, canlı görüntü sağlayabildiğini, görüntüleri yapay zekâ ile doğrulayabildiğini ve sonuçları meta verileriyle birlikte arşivleyebildiğini göstermiştir. Elde edilen sonuçlar, önerilen kavramsal mimarinin düşük maliyetli bileşenlerle dahi uygulanabilir olduğunu kanıtlamaktadır. Bu yönüyle çalışma, erişilebilir astronomi ve otonom gözlem alanındaki literatüre [4], [6], [17] pratik bir katkı sunmaktadır.

Sistemin başlıca sınırlılıkları; kamera çözünürlüğü ve ışık toplama kapasitesi, hobi servolarının mekanik hassasiyeti ve manyetometre tabanlı yön kestiriminin çevresel bozulmalara duyarlılığıdır. Gelecek çalışmalarda bu sınırlılıklar; Raspberry Pi 5 ve IMX477 sınıfı bir kamera ile yerel yapay zekâ çıkarımı [28], [35], daha hassas servo sürücüler [21] ve gelişmiş sensör füzyonu yöntemleri eklenerek doğrudan giderilebilir. Ayrıca web platformunun ölçeklenebilir bir bulut altyapısına taşınması, çok kullanıcılı bir gökyüzü veri seti vizyonunun büyütülmesine olanak tanıyacaktır.

Sonuç olarak bu çalışma, sınırlı bir bütçeyle dahi otonom ve akıllı bir gözlem sisteminin kurulabileceğini somut biçimde göstererek; hem eğitim hem de vatandaş-bilim uygulamaları için tekrarlanabilir, açık ve genişletilebilir bir temel sunmaktadır.

Sistemin pratik gözlem ortamlarında gösterdiği performans değerlendirildiğinde, özellikle gezegen gözlemlerinde başarılı sonuçlar elde edildiği görülmüştür. Teleskop, başlangıç hizalamasının ardından hedef gök cismini yüksek doğrulukla bulmuş, takip süresince sürekli mikro düzeltmeler yaparak cismin görüntü merkezinde kalmasını sağlamış ve bu işlemleri kesintisiz bir geri bildirim döngüsüyle sürdürmüştür. Bu davranış, sistemin düşük maliyetli motor ve sensör bileşenlerine rağmen yüksek kararlılık gösterebildiğini, yazılım tabanlı hata telafi mekanizmalarının fiziksel sınırlılıkları belirli ölçüde dengeleyebildiğini ve görüntü işleme yaklaşımlarının pratik koşullarda uygulanabilir olduğunu ortaya koymaktadır. Ayrıca sistemin gerçek zamanlı görüntü akışını hem mobil uygulama hem de sunucu üzerinden sağlayabilmesi, kullanıcı deneyimini güçlendirmiş ve gözlem sürecini daha interaktif hale getirmiştir.

Veri kaydı yöntemi açısından sistem, hem yerel depolama hem de bulut tabanlı aktarım modlarıyla esnek bir yapı sunmuştur. Gözlem sırasında üretilen tüm görüntülerin, meta verilerin ve hareket kayıtlarının düzenli biçimde saklanabilmesi, gözlemin daha sonra yeniden analiz edilmesine, performans karşılaştırmalarının yapılmasına ve model geliştirme süreçlerinde kullanılabilecek bir veri havuzu oluşturulmasına olanak tanımıştır. Bu veri altyapısı aynı zamanda, açık gökyüzü veri setlerinin oluşturulmasına katkı sağlayarak gelecekteki araştırmacıların bu veriler üzerinde kendi modellerini eğitmelerine veya farklı analiz teknikleri geliştirmelerine imkân tanıyacak bir temel oluşturmuştur. Sistemin birden fazla kullanıcı tarafından genişletilebilir olacak şekilde tasarlanmış olması, ilerleyen dönemlerde farklı konumlardaki teleskopların ortak bir platform altında veri üretmesine ve böylelikle hem zamansal hem de mekânsal açıdan daha çeşitli bir gökyüzü veri setinin oluşmasına zemin hazırlamaktadır.

Bu çalışma, bilime ve teknolojiye birkaç farklı açıdan katkı sunmaktadır. Öncelikle gök bilimi gözlemlerini otomatikleştirmek için yüksek maliyetli teleskop sistemlerine erişimi olmayan araştırmacılara, öğrencilere ve meraklılara düşük maliyetli fakat işlevsel bir alternatif sunmuştur. İkinci olarak, yapay zekânın gök cismi tespiti gibi geleneksel yöntemlerle yürütülmesi zor olan işlemlere entegre edilmesi, modern görüntü işleme tekniklerinin astronomi alanındaki kullanımını yaygınlaştıracak bir örnek teşkil

etmektedir. Üçüncü olarak ise sistem, IoT tabanlı gözlem yöntemlerinin astronomiyle birleştiği yeni bir uygulama alanı ortaya koymuş ve teleskopların sadece yerel değil, küresel ölçekte yönetilip koordine edilebileceği bir altyapının temelini atmıştır.

Ancak elde edilen sonuçlara rağmen sistemin bazı sınırlılıkları da mevcuttur. Öncelikle ESP32-CAM gibi düşük çözünürlüklü bir kameranın kullanılması, derin uzay gözlemlerinde sistemin kapasitesini belirli ölçüde sınırlamaktadır. Sistemin gezegen gözlemlerinde oldukça başarılı olmasına rağmen, soluk yıldızlar ve düşük kontrastlı derin uzay cisimleri için daha gelişmiş sensörlerin kullanılması gerekebilir. Ayrıca kullanılan motor ve sürücü bileşenlerinin belirli bir hassasiyet sınırı bulunmakta; çok uzun süreli takiplerde küçük titreşimler veya mekanik oynaklıklar zaman zaman birikerek görüntü merkezlemesinde ufak sapmalara yol açabilmektedir. Engel algılama sistemi, pratik kullanımda oldukça başarılı olsa da kamera çözünürlüğünün düşük olduğu durumlarda bazı sahnelerde yanılmalar görülebilir. Mobil uygulamanın bütünleşik yapısına rağmen çevrim dışı kullanımda bazı gelişmiş özellikler devre dışı kaldığından, daha güçlü yerel işleme yöntemleri bu sınırlılığı azaltabilir.

Bu sınırlılıkların giderilmesine yönelik gelecekte yapılabilecek çalışmalar oldukça geniş bir yelpazeye yayılmaktadır. Öncelikle daha yüksek çözünürlüklü kameraların sisteme uyarlanması ve bu kameraların ürettiği veri miktarına uygun yeni görüntü işleme algoritmalarının geliştirilmesi, sistemin derin uzay gözlemlerindeki performansını artıracaktır. Sürücülerinin daha yüksek hassasiyet sağlayan modellerle değiştirilmesi ve mekanik yapının daha rijit bir tasarımla yenilenmesi, uzun süreli takiplerde oluşabilecek hataları azaltabilir. Engel algılama mekanizmasının geliştirilebilmesi için gökyüzü-segmentasyonuna yönelik hafif derin öğrenme modelleri veya geometrik analiz yöntemleri sisteme entegre edilebilir. Mobil uygulama tarafında gelişmiş kullanıcı modları, çoklu teleskop yönetimi, gözlem planı oluşturma, geçmiş gözlem karşılaştırmaları ve yeni kullanıcılar arasında veri paylaşımı gibi özellikler eklenebilir.

Bunun yanında sistemin büyük ölçekli bir gözlem ağına dönüşebilmesi için, farklı kullanıcıların kendi teleskoplarını ekleyip çalıştırabildiği bir hesap yönetim altyapısı daha da genişletilebilir; verilerin merkezi bir veri gölü yapısında işlenmesi, analiz edilmesi ve araştırmacılara standart bir formatla sunulması için API tabanlı geliştirmeler yapılabilir. Ayrıca yapay zekâ modelinin yeniden eğitilmesi veya farklı astronomik objelerle genişletilmesi, sistemin algılayabileceği hedef yelpazesini artıracak ve daha kapsamlı gözlem görevlerinin otonom şekilde yürütülmesine olanak sağlayacaktır.

Sonuç olarak, bu çalışma düşük maliyetli bileşenlerle yüksek işlevselliğe sahip bir otonom teleskop sistemi geliştirmenin mümkün olduğunu göstermiş; yapay zekâ, IoT ve astronomi disiplinlerini bir araya getirerek hem uygulama hem de araştırma alanında geniş bir potansiyel sunmuştur. Sistemin modüler yapısı, hem mevcut kullanım senaryolarının sağlıklı bir şekilde desteklenmesini hem de ilerleyen dönemlerde yapılacak geliştirmelere açık olmasını sağlamaktadır. Gelecekte yapılacak çalışmalarla sistemin duyarlılığının artırılması, veri kapasitesinin genişletilmesi ve daha iyi yönlendirme mekanizmalarının eklenmesi, bu platformu yalnızca amatör gözlem aracı olmaktan çıkarp bilimsel araştırmalara katkı sunabilecek daha güçlü bir gözlem ünitesine dönüştürme potansiyeli taşımaktadır.

Gelecek çalışmalar açısından birkaç yönelim öne çıkmaktadır. İlk olarak, bu çalışmada bulut tabanlı bir büyük dil modeliyle gerçekleştirilen gök cismi doğrulaması, ideal tasarımdaki gibi cihaz üzerinde çalışan hafif bir evrişimli sinir ağı (TFLite/ONNX) ile değiştirilebilir [1], [2], [28], [35]; bu, çevrimdışı çalışmayı ve gecikmenin azaltılmasını sağlayacaktır. İkinci olarak, web platformunda biriken kullanıcı katkılı gözlem arşivi, etiketlenmiş bir veri setine dönüştürülerek bu yerel modelin eğitiminde kullanılabilir; böylece sistem, kullandıkça kendini iyileştiren bir döngüye kavuşur.

Üçüncü olarak, yönlendirme doğruluğunu artırmak için manyetometre tabanlı yön kestirimi, yıldız tanıma (plate solving) tabanlı bir kapalı çevrim düzeltmeyle desteklenebilir; bu yaklaşım, sahadaki manyetik bozulmalardan bağımsız, çok daha yüksek hassasiyette bir hedef kilidi sağlayacaktır [14]. Dördüncü olarak, mekanik tahrik sisteminin enkoderli step motorlara taşınması, hem sürekli dönüş servosunun kapalı çevrim gereksinimini ortadan kaldıracak hem de daha ağır optik yüklerin taşınmasına olanak tanıyacaktır.

Son olarak, sistemin ölçeklenebilirliği açısından, ideal tasarımda öngörülen bulut tabanlı görev kuyruğu ve nesne depolama mimarisi [22], [25], [26] hayata geçirilerek, çok sayıda saha cihazının merkezi bir platform üzerinden koordine edilmesi mümkün kılınabilir. Bu sayede, dağıtık ve düşük maliyetli gözlem düğümlerinden oluşan, topluluk temelli bir gözlem ağı kurma vizyonu gerçeğe dönüştürülebilir. Bu tezde ortaya konan çalışır prototip, bu vizyonun teknik olarak uygulanabilir olduğunu somut biçimde göstermektedir.

KAYNAKLAR

1. Su, Y., Chen, X., Liu, G., Cang, C., & Rao, P. (2023). Implementation of real-time space target detection and tracking algorithm for space-based surveillance. *Remote Sensing*, 15(12), 3156.
2. Zhou, Y., Zhang, T., Li, Z., & Qiu, J. (2025). Improved Space Object Detection Based on YOLO11. *Aerospace*, 12(7), 568.
3. Schanche, N., Cameron, A. C., Hébrard, G., Nielsen, L., Triaud, A. H. M. J., Almenara, J. M., ... & Wheatley, P. J. (2019). Machine-learning approaches to exoplanet transit detection and candidate validation in wide-field ground-based surveys. *Monthly Notices of the Royal Astronomical Society*, 483(4), 5534–5547.
4. Maulana, F., Soegijoko, W., & Yamani, A. (2016, November). Utilising Raspberry Pi as a cheap and easy do it yourself streaming device for astronomy. In *Journal of Physics: Conference Series* (Vol. 771, No. 1, p. 012025). IOP Publishing.
5. Vida, D., Mazur, M., Šegon, D., Zubović, D., Kukić, P., Parag, F., & Macan, A. (2018). First results of a Raspberry Pi based meteor camera system. *WGN, Journal of the International Meteor Organization*, 46(2), 71–78.
6. Gutiérrez, S. T., Fuentes, C. I., & Díaz, M. A. (2020). Introducing SOST: An ultra-low-cost star tracker concept based on a Raspberry Pi and open-source astronomy software. *IEEE Access*, 8, 166320–166334.
7. Calvi, J., Panico, A., Cipollone, R., De Vittori, A., & Di Lizia, P. (2021). Machine learning techniques for detection and tracking of space objects in optical telescope images. In *Aerospace Europe Conference*, Warsaw, Poland.
8. Baranova, V. S., Saetchnikov, V. A., & Spiridonov, A. A. (2021). Autonomous streaming space objects detection based on a remote optical system. *Приборы и*

- методы измерений, 12(4), 272–279.
9. Husser, T. O., Hessman, F. V., Martens, S., Masur, T., Royen, K., & Schäfer, S. (2022). pyobs—An observatory control system for robotic telescopes. *Frontiers in Astronomy and Space Sciences*, 9, 891486.
 10. De Vittori, A., Cipollone, R., Di Lizia, P., & Massari, M. (2022). Real-time space object tracklet extraction from telescope survey images with machine learning. *Astrodynamics*, 6(2), 205–218.
 11. Tsapralis, K., Choumos, G., Lappas, V., & Kontoes, C. (2024). Survey Mode: A Review of Machine Learning in Resident Space Object Detection and Characterization. In *AIAA SCITECH 2024 Forum* (p. 2065).
 12. Murawski, G. (2018, September). Use of CMOS cameras in exoplanet transit photometry. In *European Planetary Science Congress* (pp. EPSC2018–66).
 13. Littlefield, C. (2010). Observing exoplanet transits with digital SLR cameras. *J. Am. Assoc. Var. Star Obs.*, 38, 212–212.
 14. Felt, V., & Fletcher, J. (2024). Seeing stars: Learned star localization for narrow-field astrometry. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision* (pp. 8297–8305).
 15. Jia, P., Liu, Q., Sun, Y., Zheng, Y., Liu, W., & Zhao, Y. (2020). Smart observation method with wide field small aperture telescopes for real time transient detection. *arXiv preprint arXiv:2011.10407*.
 16. Jiang, Y., Tang, Y., & Ying, C. (2023). Finding a needle in a haystack: Faint and small space object detection in 16-bit astronomical images using a deep learning-based approach. *Electronics*, 12(23), 4820.
 17. Guyon, O., Walawender, J., Jovanovic, N., Butterfield, M., Gee, W. T., & Mery, R. (2014, July). The PANOPTES project: discovering exoplanets with low-cost digital cameras. In *Ground-based and Airborne Telescopes V* (Vol. 9145, pp. 1399–1406). SPIE.
 18. Gracia Témich, F., Cornejo Rodríguez, A., Granados Agustín, F., & Caulfield, H. J. (2005). Apodization of the flat mirror support of a newton telescope. *Revista mexicana de fisica*, 51(1), 104-109.
 19. Seronik, G. (2020). A beginner's guide to collimation. Gary Seronik. <https://garyseronik.com/a-beginners-guide-to-collimation/>
 20. The Editors of Sky & Telescope. (n.d.). How to align your Newtonian reflector telescope. Sky & Telescope. <https://skyandtelescope.org/astronomy-resources/how-to-align-your-newtonian-reflector-telescope>
 21. Adafruit Industries. (n.d.). Adafruit PCA9685 library documentation. Adafruit CircuitPython Documentation.
 22. Amazon Web Services, Inc. (n.d.). Amazon S3 User Guide: Presigned URLs.
 23. AstronomyAPI. (n.d.). AstronomyAPI documentation: Bodies endpoint.
 24. Astropy Collaboration. (n.d.). Astropy documentation (time, coordinates). Astropy Documentation.
 25. Celery Project. (n.d.). Celery documentation (Redis broker/backends). Celery Documentation.
 26. FastAPI. (n.d.). FastAPI documentation. FastAPI Documentation.
 27. Liechti, C. (n.d.). pySerial documentation. Read the Docs.
 28. Microsoft. (n.d.). ONNX Runtime documentation (Python API). ONNX Runtime Documentation.

29. NumPy Developers. (n.d.). NumPy documentation. NumPy Documentation.
30. OpenCV Team. (n.d.). OpenCV documentation. OpenCV Documentation.
31. Photutils Developers. (n.d.). Photutils documentation (source detection / DAOStarFinder). Photutils Documentation.
32. Raspberry Pi Ltd. (n.d.). Picamera2 documentation. Raspberry Pi Documentation.
33. Raspberry Pi Ltd. (n.d.). libcamera / rpicas-apps documentation. Raspberry Pi Documentation.
34. Rhodes, B. (n.d.). Skyfield documentation. Rhodes Mill.
35. TensorFlow Developers. (n.d.). TensorFlow documentation (Keras CNN layers, TensorFlow Lite converter & interpreter). TensorFlow Documentation.
36. Wankhede, K. (n.d.). pynmea2: Python library for parsing the NMEA 0183 protocol [Source code repository]. GitHub.

EKLER

Bu bölümde, metin içinde anlatılan prototipin teknik ayrıntılarını tek noktada toplayan donanım bağlantı tabloları, ESP32-CAM HTTP API uç noktaları ve ek görseller bir araya getirilmiştir. Ekler, sistemin yeniden üretilebilirliğini artırmak amacıyla hazırlanmıştır.

EK A. DONANIM BİLEŞENLERİ VE BAĞLANTI ŞEMASI

Aşağıdaki tablolar, prototipte kullanılan başlıca donanım bileşenlerini ve Arduino Mega 2560 ile ESP32-CAM kartlarının bağlantı (pin) düzenini özetlemektedir. Önemli bir güvenlik notu olarak, her iki kartın toprak (GND) hatları ortak olmalı ve Mega'nın 5 V TX hattı, ESP32-CAM'in 3,3 V RX girişine doğrudan değil, bir gerilim bölücü üzerinden bağlanmalıdır.

Bileşen	Açıklama / İşlev
Arduino Mega 2560	Gerçek zamanlı sensör füzyonu ve servo kontrolü
ESP32-CAM (AI-Thinker)	Wi-Fi ağ geçidi, kamera ve Mega ile UART köprüsü
MPU9250 (+AK8963)	9 eksenli IMU: ivmeölçer, jiroskop, manyetometre
NEO-6M GPS	Enlem/boylam ve zaman bilgisi (TinyGPSPlus)
Azimut servosu	Sürekli dönüş servosu (kapalı çevrim, pin 9)
Yükseklik servosu	Konum servosu (0–180°, pin 10)
Güç / regülatör	Servo ve kart beslemesi, DC–DC düzenleme
3B baskı parçalar	Halka kelepçe, tabla, servo tutucu, beşik

Tablo A.1. Prototipte kullanılan başlıca donanım bileşenleri.

Mega 2560 Bağlantısı	Pin / Ayar
MPU9250 (I2C)	VCC 3.3V, GND, SDA → 20, SCL → 21
GPS (Serial2)	GPS-TX → RX2 (17), GPS-RX → TX2 (16), 9600 baud
Azimut servosu	Sinyal → pin 9 (sürekli dönüş)
Yükseklik servosu	Sinyal → pin 10 (konum)
ESP bağlantısı (TX)	Mega TX1 (18) → gerilim bölücü → ESP RX2 (GPIO13)
ESP bağlantısı (RX)	Mega RX1 (19) ← ESP TX2 (GPIO15)

Tablo A.2. Arduino Mega 2560 bağlantı düzeni.

EK B. ESP32-CAM HTTP API UÇ NOKTALARI

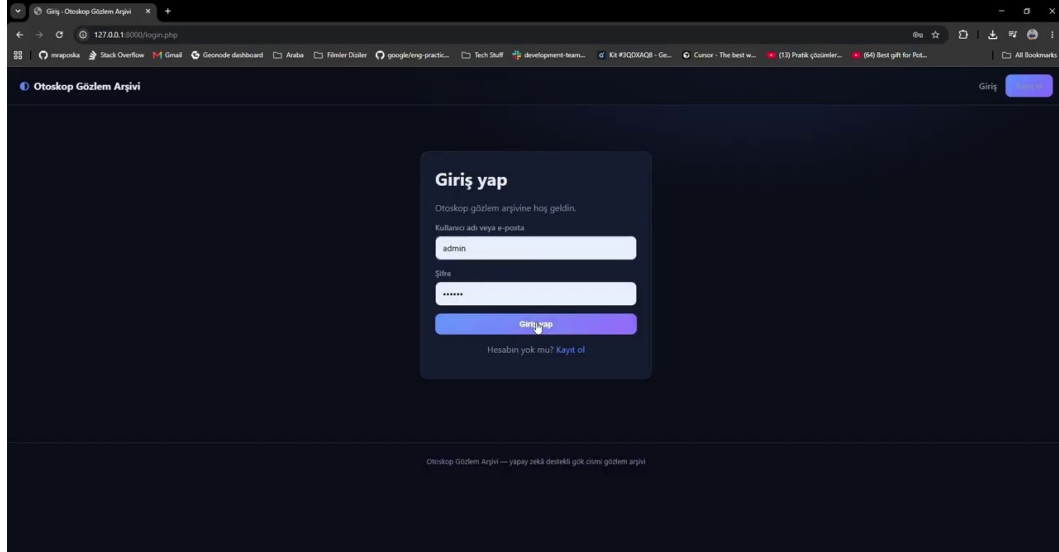
ESP32-CAM, 80 numaralı portta REST tarzı bir API, 81 numaralı portta ise canlı MJPEG yayını sunar. Tüm yanıtlar CORS başlığı taşır. Android uygulaması bu uç noktaları kullanarak teleskobu sorgular ve kontrol eder.

Metot	Uç Nokta	Açıklama
GET	/status	Telemetri JSON (uygulama sürekli sorgular)
GET	/camera	Tek JPEG kare (anlık görüntü)
GET	/stream	Canlı MJPEG yayını (port 81)
GET	/gps	{lat, lon, fix}
POST	/target	{name, azimuth, altitude} → hedefe yönel
POST	/move	{direction, step} → manuel adım
POST	/correction	{azimuthCorrection, altitudeCorrection}
POST	/calibrate	Jiro + manyetometre kalibrasyonunu tetikler
POST	/caloffset	Yön/yükseklik hizalama ofseti (EEPROM)
POST	/limits	{altMax} yükseklik üst limiti

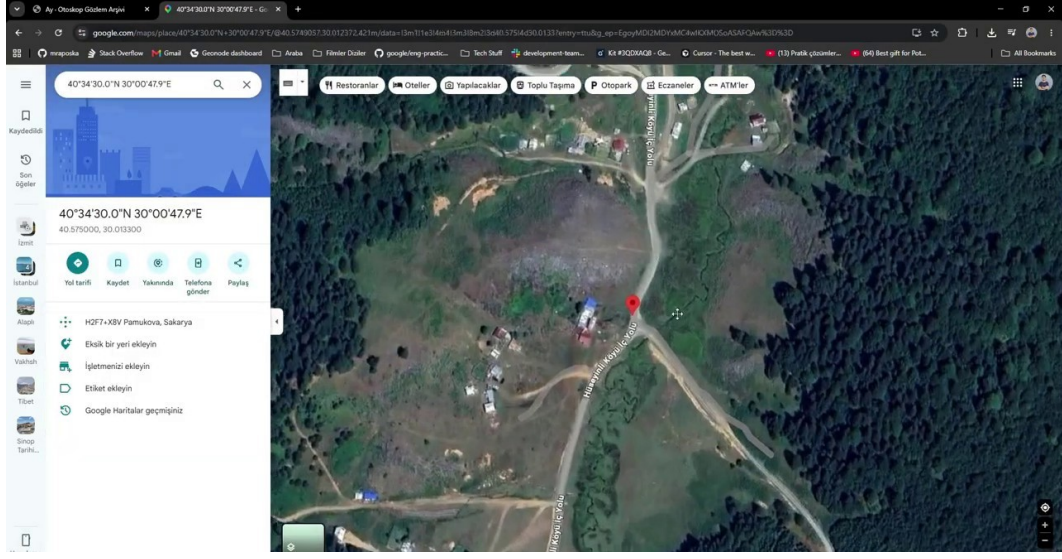
Tablo B.1. ESP32-CAM HTTP API uç noktaları.

EK C. EK GÖRSELLER

Web tabanlı gözlem arşivine ve prototip bileşenlerine ilişkin ek görseller aşağıda sunulmuştur.



Şekil C.1. Web platformu oturum açma ekranı (oturum tabanlı kimlik doğrulama).



Şekil C.2. Gözlem konumunun harita üzerinde gösterimi (GPS meta verisi).



Şekil C.3. Prototiple elde edilen ek Ay gözlemleri seçkisi.